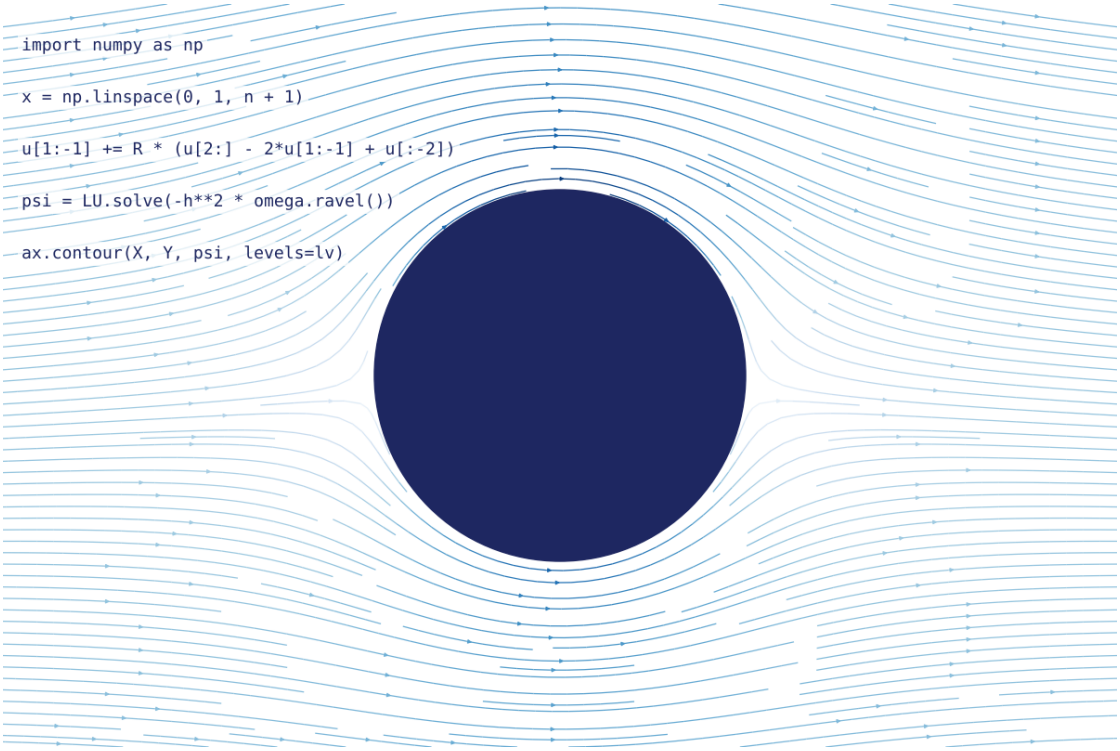
Costruzioni Aeronautiche

ITIS “E. Majorana” — Fluidodinamica numerica

# Python per la fluidodinamica numerica

## Il linguaggio, NumPy, SciPy, Matplotlib e SymPy per leggere e scrivere i programmi del corso




Monografia didattica di fluidodinamica numerica
Modulo 0: il linguaggio e gli strumenti

**Prof. Ing. Biagio Raucci**

Esempi numerici in Python (NumPy, SciPy, Matplotlib)

# Indice

**Presentazione** **vi**

**1 Primi passi: l'interprete, i numeri, le stringhe** **1**
1.1 Perché Python per la fluidodinamica numerica . . . . . 1
1.2 Variabili e tipi . . . . . 2
1.3 La virgola mobile: che cosa un calcolatore sa fare con i numeri reali . . . . . 2
1.4 Stringhe e formattazione . . . . . 3
1.5 Il Programma 0.1: l'atmosfera standard . . . . . 3
1.6 Approfondimenti . . . . . 7
1.6.1 Lo standard IEEE 754 e l'arrotondamento . . . . . 7
1.6.2 Notebook, ambienti e versioni . . . . . 7

**2 Controllo del flusso e funzioni** **9**
2.1 Decisioni e ripetizioni . . . . . 9
2.2 Funzioni . . . . . 10
2.2.1 Ricorsione . . . . . 11
2.3 Moduli e il blocco principale . . . . . 11
2.4 Il Programma 0.2: velocità limite e caduta di una sfera . . . . . 11
2.5 Approfondimenti . . . . . 17
2.5.1 Quando usare `for` e quando `while` . . . . . 17
2.5.2 L'ordine di un metodo e la regolarità del problema . . . . . 17
2.5.3 Argomenti per nome, `*args` e `**kwargs` . . . . . 17

**3 Liste, tuple, dizionari e stringhe** **18**
3.1 Le collezioni di Python . . . . . 18
3.2 Comprehension . . . . . 19
3.3 Dizionari e insiemi . . . . . 19
3.4 Stringhe come sequenze . . . . . 19
3.5 Il Programma 0.3: profili NACA a quattro cifre . . . . . 20
3.6 Approfondimenti . . . . . 24
3.6.1 Mutabile e immutabile . . . . . 24
3.6.2 Complessità delle operazioni . . . . . 24

3.6.3 Spacchettamento e assegnazioni multiple . . . 24

**4 NumPy: gli array e le differenze finite 25**
4.1 Perché gli array . . . 25
4.2 Indici, slicing e viste . . . 25
4.3 Broadcasting . . . 26
4.4 Differenze finite scritte con lo slicing . . . 27
4.5 Il Programma 0.4 . . . 27
4.6 Approfondimenti . . . 31
4.6.1 Perché la vettorializzazione è veloce . . . 31
4.6.2 Memoria e ordine degli indici . . . 31
4.6.3 Indicizzazione avanzata e funzioni universali . . . 32

**5 Le figure con Matplotlib 33**
5.1 Figura, assi, grafico . . . 33
5.2 Scale, legende, annotazioni . . . 34
5.3 Campi bidimensionali . . . 34
5.4 Lo stile della casa: `stile_grafici.py` . . . 34
5.5 Il Programma 0.5: la corrente intorno al cilindro . . . 35
5.6 Approfondimenti . . . 39
5.6.1 Colori e leggibilità . . . 39
5.6.2 Formati di uscita e dimensioni . . . 39
5.6.3 Grafici interattivi e animazioni . . . 40

**6 La cassetta degli attrezzi di SciPy 41**
6.1 Quando usare SciPy . . . 41
6.2 Radici e minimi: `scipy.optimize` . . . 41
6.3 Integrali ed equazioni differenziali: `scipy.integrate` . . . 42
6.4 Sistemi lineari: `scipy.linalg` e `scipy.sparse` . . . 42
6.5 Interpolazione e funzioni speciali . . . 43
6.6 Il Programma 0.6 . . . 43
6.7 Approfondimenti . . . 47
6.7.1 Tolleranze e messaggi . . . 47
6.7.2 Quando il risolutore “non converge” . . . 48
6.7.3 Che cosa SciPy non sostituisce . . . 48

**7 Calcolo simbolico con SymPy 49**
7.1 Simboli invece di numeri . . . 49
7.2 Derivate, integrali, serie . . . 49
7.3 Le formule alle differenze finite con i coefficienti indeterminati . . . 50
7.4 Equazioni, sistemi, `lambdify` . . . 50

7.5 Il Programma 0.7 . . . 51
7.6 Approfondimenti . . . 54
7.6.1 Limiti del calcolo simbolico . . . 54
7.6.2 Simbolico e numerico insieme . . . 54

**8 Anatomia di un programma del corso** **55**
8.1 Il programma modello . . . 55
8.2 Lettura guidata 1: la caduta della sfera (Modulo 1) . . . 58
8.3 Lettura guidata 2: il problema di Rayleigh (Modulo 3) . . . 61
8.4 Lettura guidata 3: la cavità MAC (Modulo 4) . . . 62
8.5 Approfondimenti . . . 65
8.5.1 Stile e leggibilità: PEP 8 . . . 65
8.5.2 Moduli comuni e riuso . . . 65
8.5.3 Argomenti da riga di comando . . . 65

**9 Errori, verifiche, file e tempi** **66**
9.1 Gli errori che il calcolatore segnala . . . 66
9.2 Gli errori che il calcolatore non segnala . . . 67
9.3 Verifiche automatiche con `assert` . . . 68
9.4 File . . . 68
9.5 Tempi e profilo . . . 69
9.6 Il Programma 0.9 . . . 69
9.7 Approfondimenti . . . 73
9.7.1 Riproducibilità . . . 73
9.7.2 Debug: come trovare un errore . . . 73
9.7.3 Quando Python è troppo lento . . . 74
9.7.4 Archivio dei programmi del corso . . . 74

**A Sintassi essenziale, listati di supporto e istruzioni** **75**
A.1 Sintassi essenziale di riferimento . . . 75
A.2 Nomi ricorrenti nei programmi del corso . . . 78
A.3 Listati di supporto . . . 78
A.4 Come riprodurre i risultati . . . 91
A.5 Codici in rete e QR code . . . 92
A.6 Le figure . . . 92

**B Esercizi svolti** **93**
B.1 Esercizio 1 – Conversioni di unità e tabella formattata . . . 93
B.2 Esercizio 2 – Mach e pressione dinamica in quota . . . 94
B.3 Esercizio 3 – Virgola mobile: somma compensata e radici stabili . . . 94
B.4 Esercizio 4 – Serie di Taylor con un ciclo `while` . . . 95

B.5 Esercizio 5 – Schema di Horner e metodo di Newton . . . . . 96
B.6 Esercizio 6 – La polare come dizionario . . . . . 96
B.7 Esercizio 7 – Un file di coordinate . . . . . 97
B.8 Esercizio 8 – Vettorializzare . . . . . 98
B.9 Esercizio 9 – Broadcasting . . . . . 98
B.10 Esercizio 10 – Nodi fittizi e condizioni di Neumann . . . . . 99
B.11 Esercizio 11 – Una figura a due pannelli . . . . . 100
B.12 Esercizio 12 – `fsolve` sul sistema di Galerkin della sfera . . . . . 100
B.13 Esercizio 13 – Il pendolo con `solve_ivp` ed eventi . . . . . 101
B.14 Esercizio 14 – Lo schema implicito con l'algoritmo di Thomas . . . . . 102
B.15 Esercizio 15 – La formula a cinque punti con SymPy . . . . . 102
B.16 Esercizio 16 – Il Programma 1.1 con l'integratore generico . . . . . 103
B.17 Esercizio 17 – Il problema di Rayleigh in forma implicita . . . . . 103
B.18 Esercizio 18 – Il Programma 4.5 come libreria . . . . . 104
B.19 Esercizio 19 – Un programma completo: convezione lineare . . . . . 105
B.20 Esercizio 20 – Caccia agli errori . . . . . 106

**Bibliografia** **107**

# Elenco dei listati


1.1 Programma 0.1 – primi passi . . . . . 4
2.1 Programma 0.2 – controllo del flusso e funzioni . . . . . 12
3.1 Programma 0.3 – liste, tuple, dizionari e stringhe . . . . . 20
4.1 Programma 0.4 – NumPy . . . . . 27
5.1 Programma 0.5 – Matplotlib . . . . . 35
6.1 Programma 0.6 – SciPy . . . . . 43
7.1 Programma 0.7 – SymPy . . . . . 51
8.1 Programma 0.8 – il programma modello . . . . . 56
8.2 Lettura guidata 1 – `prog1_1_caduta_sfera.py` . . . . . 59
8.3 Lettura guidata 1 – `ode_solvers.py` . . . . . 60
8.4 Lettura guidata 1 – `ode_solvers.integra` . . . . . 60
8.5 Lettura guidata 2 – `prog3_4_rayleigh.py` . . . . . 61
8.6 Lettura guidata 3 – `prog4_5_cavita_mac.py`, prima parte . . . . . 62
8.7 Lettura guidata 3 – `prog4_5_cavita_mac.py`, seconda parte . . . . . 63
9.1 Programma 0.9 – errori, verifiche, file e tempi . . . . . 69
A.1 `metodi_numerici.py` – modulo comune . . . . . 78
A.2 `stile_grafici.py` – impostazioni di Matplotlib . . . . . 81
A.3 `esercizi_svolti.py` – codice degli esercizi . . . . . 82

# Presentazione

## Di che cosa parla questa monografia

Le quattro monografie del corso di fluidodinamica numerica contengono ottantadue programmi in Python: dalla caduta di una sfera integrata con Runge–Kutta (Modulo 1) alla cavità con coperchio mobile risolta in variabili primitive su griglia sfalsata (Modulo 4). Ogni programma è commentato e discusso, ma presuppone che chi legge sappia *leggere* Python: che cosa significa `u[1:-1]`, perché `np.roll` realizza una condizione periodica, come una funzione riceve un'altra funzione come argomento, che cosa fa `if __name__ == "__main__"`. Questa monografia, il Modulo 0, colma quel presupposto: insegna il linguaggio e le librerie *nella misura e nella forma in cui servono per la fluidodinamica numerica*, e lo fa con esempi tratti dalla stessa disciplina (atmosfera standard, profili alari, caduta di una sfera, corrente intorno a un cilindro, equazione della diffusione), in modo che ogni costrutto venga introdotto per risolvere un problema reale e non in astratto.

Non è un manuale completo di Python: non parla di programmazione a oggetti, di interfacce grafiche, di reti. È invece completo rispetto a ciò che i programmi del corso usano, perché il suo indice è stato costruito proprio censendo quei programmi: i tipi numerici e la virgola mobile; il controllo del flusso e le funzioni; liste, tuple, dizionari e stringhe; gli array di NumPy con lo slicing, le viste e il broadcasting, che sono il cuore di ogni codice alle differenze finite; le figure con Matplotlib; gli strumenti di SciPy (radici, integrazione, sistemi lineari, matrici sparse, interpolazione); il calcolo simbolico con SymPy, usato nelle monografie per verificare le formule; infine la *struttura* di un programma di calcolo (docstring, costanti, funzioni, blocco principale, verifica, figure), gli errori più frequenti e il modo di scovarli, i file e la misura dei tempi. Tre *letture guidate* di programmi reali del corso (la caduta della sfera del Modulo 1, il problema di Rayleigh del Modulo 3, la cavità MAC del Modulo 4) mostrano come tutto questo si combina in un codice completo.

Il percorso è progressivo e ogni capitolo poggia sul precedente, ma chi ha già familiarità con il linguaggio può partire dal Cap. 4; chi vuole soltanto capire un programma del corso può usare l'Appendice A, che raccoglie la sintassi essenziale in due pagine, e il Cap. 8. Come nelle altre monografie, tutti i programmi sono eseguibili, stampano risultati verificati contro valori noti ed esercizi interamente svolti chiudono il volume. La bibliografia raccoglie le fonti: il testo di riferimento del corso [1] (i cui programmi, in MATLAB, sono stati tradotti in Python nelle monografie), il manuale introduttivo libero di OpenStax [4], la documentazione ufficiale di Python, NumPy, SciPy, Matplotlib e SymPy, e il corso "CFD Python" di Barba e Forsyth [3], che ha ispirato l'impostazione dell'intero corso.

## Come è organizzata

**Capitolo 1 – Primi passi: l'interprete, i numeri, le stringhe.**
Installare ed eseguire; variabili e tipi; interi, virgola mobile (epsilon di macchina, cancellazione, non associatività) e numeri complessi; stringhe e formattazione con le f-string; l'atmosfera standard come primo programma. Approfondimenti: IEEE 754, arrotondamento, notebook e ambienti.

**Capitolo 2 – Controllo del flusso e funzioni.**
`if`, `for`, `while`, `break`; `range`, `enumerate`, `zip`; funzioni, argomenti predefiniti, docstring, funzioni come argomenti e `lambda`; ricorsione; ambito delle variabili; moduli e `__main__`; bisezione, Newton e Runge–Kutta come esempi. Approfondimenti: quando la ricorsione conviene, l'ordine di un metodo e la regolarità del problema.

**Capitolo 3 – Liste, tuple, dizionari e stringhe.**
Le collezioni di Python e le loro operazioni; slicing; comprehension; copie e riferimenti; i profili NACA come esempio. Approfondimenti: complessità delle operazioni, quando usare una lista e quando un array.

**Capitolo 4 – NumPy: gli array e le differenze finite.**
Creazione, forma, tipo; slicing e viste; broadcasting; funzioni universali e riduzioni; le differenze finite scritte con lo slicing e la vettorializzazione dei cicli; algebra lineare. Approfondimenti: memoria e ordine degli indici, indicizzazione avanzata, generatori di numeri casuali.

**Capitolo 5 – Le figure con Matplotlib.**
Figure e assi; grafici, scale, legende, etichette in LaTeX; campi bidimensionali (contour, streamplot, quiver); pannelli multipli; lo stile della casa. Approfondimenti: colori e leggibilità, formati di uscita.

**Capitolo 6 – La cassetta degli attrezzi di SciPy.**
Radici e minimi; integrali, problemi ai valori iniziali e ai limiti; sistemi lineali densi, a banda e sparsi; interpolazione e funzioni speciali; ogni strumento su un problema della fluidodinamica con soluzione nota. Approfondimenti: tolleranze e messaggi di errore, quando il risolutore "non converge".

**Capitolo 7 – Calcolo simbolico con SymPy.**
Simboli, derivate, integrali, serie; derivazione automatica delle formule alle differenze; `lambdify`; verifica simbolica dei risultati delle monografie. Approfondimenti: limiti del calcolo simbolico.

**Capitolo 8 – Anatomia di un programma del corso.**
Il programma modello (docstring, costanti, funzioni, blocco principale, verifica, figure); tre letture guidate di programmi dei Moduli 1, 3 e 4; come passare dal metodo all'algoritmo al programma. Approfondimenti: stile e leggibilità (PEP 8), moduli comuni.

**Capitolo 9 – Errori, verifiche, file e tempi.**
Eccezioni e traceback; gli errori silenziosi tipici del calcolo numerico; test con `assert`; file di dati e parametri; cronometri e profilo. Approfondimenti: riproducibilità, versioni, archivio dei programmi del corso.

**Appendici.**
Sintassi essenziale di riferimento, simboli e istruzioni; i venti esercizi svolti passo per passo.

I riquadri colorati hanno il significato consueto del corso, adattato a un testo di programmazione:

- **teoria** (viola): definizioni e regole del linguaggio, con la spiegazione del *perché*;
- **fasi** (blu): procedure passo per passo (come eseguire, come costruire un programma);
- **ipotesi** (giallo): le convenzioni adottate nel corso;
- **dati** (grigio): i dati dei programmi;
- **risultati** (verde): esiti dei programmi e loro verifica;
- **esempi** (arancio): applicazioni ed esempi aeronautici;
- **avvisi** (rosso): errori frequenti e trappole.

I frammenti di codice nel testo sono riquadrati con una barra blu; l'uscita stampata dai programmi è riportata su fondo grigio. I listati completi hanno le righe numerate e, nel margine, il QR code che porta al file in rete.

## I programmi

I nove programmi del modulo (`prog0_1` ... `prog0_9`) e il file degli esercizi usano soltanto Python 3 con `numpy`, `scipy`, `matplotlib` e `sympy`; i moduli comuni `metodi_numerici.py` e `stile_grafici.py` sono gli stessi delle altre monografie. I programmi delle letture guidate sono copiati nella cartella `codice/letture/` e il QR nel margine rimanda alla monografia di origine. Tutti i programmi sono disponibili all'indirizzo `https://braucci.github.io/CFD/monografia0/codici/`.

# Capitolo 1

# Primi passi: l'interprete, i numeri, le stringhe

## 1.1 Perché Python per la fluidodinamica numerica

Un programma di calcolo numerico fa tre cose: organizza numeri in tabelle (i valori di velocità, pressione, temperatura nei nodi di una griglia), applica a quelle tabelle formule ripetute migliaia di volte (gli schemi alle differenze finite), e presenta i risultati (tabelle stampate e grafici). Per decenni lo strumento è stato il FORTRAN, poi MATLAB; i programmi del testo di riferimento del corso [1] sono in MATLAB. Le monografie li hanno tradotti in **Python**, per tre motivi: è libero e installato ovunque; ha una sintassi leggibile, vicina alla scrittura matematica, che rende il codice un buon *testo* per imparare gli algoritmi; e con le librerie NumPy, SciPy e Matplotlib offre tutto ciò che serve (array, algebra lineare, risolutori, grafici) con prestazioni adeguate, purché si scrivano le formule "per vettori" e non per singoli elementi, come insegnerà il Cap. 4.

Python è un linguaggio *interpretato*: le istruzioni sono eseguite una per una da un programma, l'interprete, senza una fase di compilazione. Si può usare in modo interattivo (si scrive un'istruzione, si vede il risultato) o eseguendo un file di testo con estensione `.py`, uno *script*: tutti i programmi del corso sono script.

**Installare ed eseguire**

1. Installare Python 3 (dal sito `python.org`, oppure con la distribuzione Anaconda che include già le librerie scientifiche). Nel corso si usa Python $\geq$ 3.9.
2. Installare le librerie: dal terminale, `pip install numpy scipy matplotlib sympy`.
3. Scrivere il programma in un editor di testo (Visual Studio Code, Spyder, Thonny, o un semplice editor) e salvarlo come `nome.py`.
4. Eseguirlo dal terminale, nella cartella in cui si trova: `python3 nome.py`. L'uscita appare nel terminale; le figure sono salvate nei file indicati dal programma.
5. In alternativa, l'interprete interattivo (`python3` senza argomenti, o meglio IPython, o un *notebook* Jupyter) permette di provare un'istruzione alla volta: utilissimo per imparare e per esplorare, meno adatto ai programmi completi del corso, che vanno eseguiti e riprodotti per intero.

## 1.2 Variabili e tipi

Una **variabile** è un nome legato a un valore; si crea con l'assegnazione = e non va dichiarata: il suo tipo è quello del valore assegnato.

```
n_passi = 400          # int
g = 9.80665            # float
z = 3 + 4j             # complex
aperto = True          # bool
nome = "Costruzioni"   # str
```

Il testo dopo # è un *commento*, ignorato dall'interprete. I nomi distinguono maiuscole e minuscole; per convenzione (PEP 8, Cap. 8) le variabili sono in minuscolo con parole separate da _, le costanti in maiuscolo.

**I quattro tipi numerici**

**`int`** interi di lunghezza arbitraria: $2^{100}$ è un intero esatto, non c'è overflow. Operatori: `+ - *`, `//` (quoziente intero), `%` (resto), `**` (potenza). Attenzione: `7 / 2` vale `3.5` (divisione vera, sempre `float`), `7 // 2` vale `3`.

**`float`** numeri in virgola mobile a 64 bit secondo lo standard IEEE 754 ("doppia precisione"): circa 16 cifre significative, valori fino a $1{,}8 \times 10^{308}$. È il tipo di tutti i campi fisici del corso.

**`complex`** coppie di float, con `j` come unità immaginaria: `(3+4j).real`, `abs(3+4j)`, `z.conjugate()`. Servono ai metodi delle variabili complesse del Modulo 2 e all'analisi di von Neumann.

**`bool`** `True` e `False`, risultato dei confronti (==, !=, <, <=, ...) e degli operatori logici `and`, `or`, `not`. Sono anche gli interi 1 e 0.

Le funzioni `int()`, `float()`, `str()` convertono fra tipi; `type(x)` dice il tipo di `x`. Il modulo `math` fornisce `sqrt`, `sin`, `exp`, `pi`, ... per singoli numeri; per gli array si useranno le funzioni di NumPy.

## 1.3 La virgola mobile: che cosa un calcolatore sa fare con i numeri reali

Questa è la sezione più importante del capitolo per chi fa calcolo numerico. Un `float` non è un numero reale: è un numero della forma $\pm m \cdot 2^e$ con una mantissa $m$ di 53 bit e un esponente $e$ fra $-1022$ e 1023. Ne seguono tre conseguenze.

**Epsilon di macchina, arrotondamento, cancellazione**

*Epsilon di macchina.* Il più piccolo numero $\varepsilon$ tale che $1 + \varepsilon$ sia rappresentabile e diverso da 1 è $\varepsilon = 2^{-52} \approx 2{,}2 \times 10^{-16}$: ogni operazione è esatta a meno di un errore relativo dell'ordine di $\varepsilon/2$. Numeri come 0,1 non sono rappresentabili esattamente (in base 2 sono periodici), quindi `0.1 + 0.2 == 0.3` è `False`: i confronti fra float vanno fatti con una tolleranza, `abs(a - b) < tol` o `math.isclose(a, b)`.

*Non associatività.* Poiché ogni somma arrotonda, $(a+b)+c \neq a+(b+c)$ in generale, e l'ordine delle operazioni conta. Sommando $10^6$ termini $1/k^2$ dal più grande al più piccolo si perdono le cifre dei termini piccoli, "schiacciati" dalla somma parziale già grande; sommando dal più piccolo al più grande l'errore è minore. Il Programma 0.1 misura una differenza di $4 \times 10^{-14}$ fra i due ordini; l'Esercizio 3 mostra la somma compensata di Kahan, che recupera quasi tutta la precisione.
*Cancellazione.* Sottrarre due numeri quasi uguali distrugge le cifre significative: in $(1-\cos x)/x^2$ per $x = 10^{-8}$, $\cos x$ vale 1 alla precisione di macchina, il numeratore è esattamente 0 e il risultato è 0 invece di $1/2$. La cura è riscrivere la formula senza la sottrazione: $1-\cos x = 2\sin^2(x/2)$. È la stessa trappola delle differenze finite con passo troppo piccolo: l'errore di arrotondamento, $\sim \varepsilon/h$, cresce al decrescere di $h$ e oltre un certo punto vince sull'errore di troncamento (Cap. 4).

**I numeri speciali**

Un float può valere `inf` (overflow: `1e308 * 10`) o `nan` ("not a number": `0.0/0.0` con NumPy, `inf - inf`, $\sqrt{-1}$). Python puro si ferma con un errore sulla divisione per zero; NumPy invece prosegue silenziosamente producendo `inf` e `nan`, che si propagano in tutto il calcolo: un campo di velocità che "esplode" per instabilità numerica diventa un array di `nan` senza alcun messaggio. Per questo i programmi del corso controllano `np.isfinite(u).all()` e sospendono gli avvisi solo dove sanno che cosa stanno facendo (`with np.errstate(...)`, Cap. 9).

## 1.4 Stringhe e formattazione

Una **stringa** (`str`) è una sequenza di caratteri fra virgolette (doppie o semplici). Si concatena con `+`, si misura con `len`, si indicizza e si "affetta" come le liste (`nome[:12]`, `nome[-12:]`, Cap. 3); ha metodi come `.upper()`, `.split()`, `.replace()`, `.strip()`. I caratteri speciali si scrivono con la barra: `\n` (a capo), `\t` (tabulazione).

Lo strumento che i programmi del corso usano continuamente è la **f-string**: una stringa preceduta da `f` in cui le espressioni fra graffe sono valutate e inserite, con una *specifica di formato* dopo i due punti:

```
v, Mach = 251.3, 0.7382
print(f"v = {v:.1f} m/s, Mach = {Mach:.3f}, {v:.3e}, [{v:10.2f}] [{v:<10.2f}]")
# v = 251.3 m/s, Mach = 0.738, 2.513e+02, [    251.30] [251.30    ]
```

Le specifiche più usate: `.3f` (tre decimali), `.3e` (notazione esponenziale), `10.2f` (dieci caratteri, due decimali, allineato a destra), `<10.2f` (a sinistra), `>8` (larghezza 8, a destra, per testo), `:d` (intero), `:.1%` (percentuale), `:,` (separatore delle migliaia), `!r` (rappresentazione con virgolette). Con `.replace(".", ",")` si ottiene la virgola decimale italiana nelle tabelle. La funzione `print` accetta più argomenti separati da virgole e termina con un a capo (modificabile con `end=""`).

## 1.5 Il Programma 0.1: l'atmosfera standard

Il primo programma del modulo mette insieme numeri, virgola mobile e stringhe su un calcolo che ogni allievo di Costruzioni Aeronautiche conosce: l'**atmosfera standard internazionale**

(ISA). Nella troposfera ($z \le 11$ km) la temperatura decresce linearmente, $T = T_0 - Lz$ con $L = 6{,}5$ K/km, e dall'equilibrio idrostatico con la legge dei gas, $\mathrm{d}p/\mathrm{d}z = -\rho g = -pg/(RT)$, si ottiene

$$\frac{p}{p_0} = \left(\frac{T}{T_0}\right)^{g/(RL)}, \qquad \rho = \frac{p}{RT}, \qquad a = \sqrt{\gamma RT}; \tag{1.1}$$

nella stratosfera (11–20 km) $T$ è costante e $p = p_{11}\exp[-g(z - z_{11})/(RT_{11})]$. Il programma definisce la funzione `isa(z)` (le funzioni sono il tema del capitolo seguente; qui basta leggerla come una formula con un nome) e stampa la tabella.

**Dati del Programma 0.1**

$T_0 = 288{,}15$ K, $p_0 = 101\,325$ Pa, $L = 0{,}0065$ K/m, $R = 287{,}05$ J/(kg K), $\gamma = 1{,}4$, $g = 9{,}80665$ m/s$^2$; quote da 0 a 20 km. Verifica: alla tropopausa la tabella ISA dà $p = 22\,632$ Pa e $\rho = 0{,}3639$ kg/m$^3$.


prog0_1_primi_passi.py

```python
"""
prog0_1_primi_passi.py -- Programma 0.1: variabili, numeri, stringhe e stampa.

Esempio conduttore: l'atmosfera standard internazionale (ISA) fino a 20 km,
    troposfera (z <= 11 km):  T = T0 - L z,   p = p0 (T/T0)^(g/(R L)),
    stratosfera (11-20 km):   T = T11 = 216,65 K,  p = p11 exp[-g (z - z11)/(R T11)],
    rho = p/(R T),  a = sqrt(gamma R T).
Il programma mostra i tipi numerici (int, float, complex, bool), gli operatori,
la precisione finita dei numeri in virgola mobile (epsilon di macchina,
cancellazione, non associativita' della somma), le stringhe formattate (f-string)
e la stampa di una tabella; infine traccia T, p, rho in funzione della quota.
Verifica: a 11 000 m la tabella ISA da' p = 22 632 Pa e rho = 0,3639 kg/m^3.
"""
import math
import sys
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

# ---------------------------------------------------------------- costanti e funzione dell'atmosfera standard
G0 = 9.80665                        # accelerazione di gravita' standard [m/s^2]
T0, P0, L, R, GAMMA = 288.15, 101325.0, 0.0065, 287.05287, 1.4
Z11, T11 = 11000.0, T0 - 0.0065 * 11000.0
P11 = P0 * (T11 / T0) ** (G0 / (R * L))


def isa(z):
    """Temperatura [K], pressione [Pa], densita' [kg/m^3] e velocita' del suono [m/s] alla quota z [m]."""
    if z <= Z11:
        T = T0 - L * z
        p = P0 * (T / T0) ** (G0 / (R * L))
    else:
        T = T11
        p = P11 * math.exp(-G0 * (z - Z11) / (R * T11))
    rho = p / (R * T)
    return T, p, rho, math.sqrt(GAMMA * R * T)


if __name__ == "__main__":
```

```
    # ---------------------------------------------------------------- 1) numeri e tipi
    print("1) Tipi numerici")
    n_passi = 400                       # int: intero di lunghezza arbitraria
    g = 9.80665                         # float: virgola mobile a 64 bit (IEEE 754 "double")
    z = 3 + 4j                          # complex: parte reale e immaginaria (j = unita' immaginaria)
    aperto = True                       # bool: True/False (sono anche gli interi 1 e 0)
    for v in (n_passi, g, z, aperto):
        print(f"   {v!r:>12}  e' di tipo {type(v).__name__}")
    print(f"   7 / 2 = {7 / 2}   (divisione vera)      7 // 2 = {7 // 2}   (quoziente intero)"
          f"      7 % 2 = {7 % 2}   (resto)      2 ** 10 = {2 ** 10}   (potenza)")
    print(f"   2 ** 100 = {2 ** 100}   (gli interi non vanno in overflow)")
    print(f"   |3+4j| = {abs(z)},  parte reale {z.real},  coniugato {z.conjugate()}")
    print(f"   math.sqrt(2) = {math.sqrt(2)},  math.pi = {math.pi},  math.e = {math.e}")

    # ---------------------------------------------------------------- 2) virgola mobile
    print("\n2) Virgola mobile: precisione finita")
    eps = sys.float_info.epsilon
    print(f"   epsilon di macchina: {eps} = 2^-52  (1 + eps e' il primo numero > 1 rappresentabile)")
    print(f"   1 + eps/2 == 1 ? {1 + eps / 2 == 1}      0.1 + 0.2 == 0.3 ? {0.1 + 0.2 == 0.3}   (0.1 + 0.2 = {0.1 + 0.2:.17f})")
    print(f"   confronto corretto: math.isclose(0.1 + 0.2, 0.3) = {math.isclose(0.1 + 0.2, 0.3)}")
    print(f"   massimo float: {sys.float_info.max:.3e}; 1e308 * 10 = {1e308 * 10}; 1e-320 = {1e-320} (denormalizzato)")
    # cancellazione: (1 - cos x)/x^2 -> 1/2 per x -> 0
    for x in (1e-2, 1e-4, 1e-6, 1e-8):
        ingenuo = (1 - math.cos(x)) / x**2
        stabile = 2 * math.sin(x / 2)**2 / x**2      # identita' 1 - cos x = 2 sin^2(x/2): nessuna sottrazione
        print(f"   x = {x:.0e}: (1 - cos x)/x^2 = {ingenuo:.10f} (ingenuo)  {stabile:.10f} (stabile)   esatto -> 0,5")
    # non associativita': somma di 1/k^2 in avanti e all'indietro
    N = 10**6
    avanti = 0.0
    for k in range(1, N + 1):
        avanti += 1.0 / k**2
    indietro = 0.0
    for k in range(N, 0, -1):
        indietro += 1.0 / k**2
    esatto = math.pi**2 / 6 - 1.0 / (N + 0.5)          # coda stimata con l'integrale
    print(f"   somma di 1/k^2, k = 1..{N}: in avanti {avanti:.15f}, all'indietro {indietro:.15f}")
    print(f"   differenza {abs(avanti - indietro):.1e}; la somma all'indietro (dai termini piccoli) e' la piu' accurata"
          f" (riferimento {esatto:.15f})")

    # ---------------------------------------------------------------- 3) stringhe
    print("\n3) Stringhe e formattazione")
    nome = "Costruzioni Aeronautiche"
    print(f"   lunghezza {len(nome)}, maiuscolo {nome.upper()!r}, prime 12 lettere
```

```
    {nome[:12]!r}, ultime 12 {nome[-12:]!r}")
83  print(f"   {'a capo e tabulazioni si scrivono con \\n e \\t'}; concatenazione:
    " + "ITIS " + "Majorana")
84  v, Mach = 251.3, 0.7382
85  print(f"   f-string: v = {v:.1f} m/s, Mach = {Mach:.3f}, in notazione
    esponenziale {v:.3e}, in colonna [{v:10.2f}] [{v:<10.2f}]")
86  print(f"   percentuale {0.0523:.1%}, separatore delle migliaia {123456789:,},
    intero in binario {37:b} e in esadecimale {255:x}")
87  print(f"   virgola decimale italiana: {v:.1f}".replace(".", ","))
88
89  # ------------------------------------------------------------------ 4)
    atmosfera standard
90  print("\n4) Atmosfera standard (ISA)")
91  print(f"   {'z [m]':>8} {'T [K]':>8} {'p [Pa]':>10} {'rho [kg/m3]':>12} {'a
    [m/s]':>8} {'p/p0':>7}")
92  for z in range(0, 20001, 2000):
93      T, p, rho, a = isa(z)
94      print(f"   {z:8d} {T:8.2f} {p:10.1f} {rho:12.4f} {a:8.2f} {p / P0:7.4f}")
95  T, p, rho, a = isa(11000.0)
96  print(f"   verifica alla tropopausa: p = {p:.0f} Pa (tabella ISA: 22632), rho =
    {rho:.4f} (tabella: 0,3639), T = {T:.2f} K")
97  assert abs(p - 22632) < 1 and abs(rho - 0.3639) < 5e-4, "la tabella ISA non
    torna"
98
99  # ------------------------------------------------------------------ 5) figura
100 zz = np.linspace(0, 20000, 401)
101 dati = np.array([isa(z) for z in zz])           # colonne: T, p, rho, a
102 fig, ax = plt.subplots(1, 3, figsize=(8.4, 2.8))
103 ax[0].plot(dati[:, 0], zz / 1000, color=BLU); ax[0].set_xlabel("$T$ [K]");
    ax[0].set_ylabel("$z$ [km]")
104 ax[1].plot(dati[:, 1] / 1000, zz / 1000, color=ARANCIO); ax[1].set_xlabel("$p$
    [kPa]")
105 ax[2].semilogx(dati[:, 2], zz / 1000, color=VERDE); ax[2].set_xlabel("$\\rho$
    [kg/m$^3$]")
106 for a_ in ax:
107     a_.axhline(11, color=GRIGIO, lw=0.6, ls=":")
108 fig.tight_layout()
109 salva(fig, "prog0_1_isa")
```

Listato 1.1: Programma 0.1 – tipi numerici, virgola mobile, stringhe e l'atmosfera standard (`prog0_1_primi_passi.py`).

**Risultati del Programma 0.1**

- `7 / 2 = 3.5, 7 // 2 = 3, 2 ** 100` è un intero di 31 cifre; `1 + eps/2 == 1` è `True`: la metà dell'epsilon di macchina "scompare".
- $(1-\cos x)/x^2$ calcolato ingenuamente dà 0,50004 per $x = 10^{-6}$ e 0 per $x = 10^{-8}$; con $2\sin^2(x/2)/x^2$ dà 0,5000000000 sempre.
- La somma di $10^6$ termini $1/k^2$ differisce di $4{,}4 \times 10^{-14}$ fra i due ordini; quella all'indietro coincide con il riferimento $\pi^2/6 - 1/(N+\frac{1}{2})$ a tutte le cifre stampate.
- La tabella ISA (Fig. 1.1): a 11 000 m $p = 22\,632$ Pa e $\rho = 0{,}3639$ kg/m$^3$, come la tabella ufficiale; l'istruzione `assert` alla fine lo verifica automaticamente e fermerebbe il programma se non fosse vero.

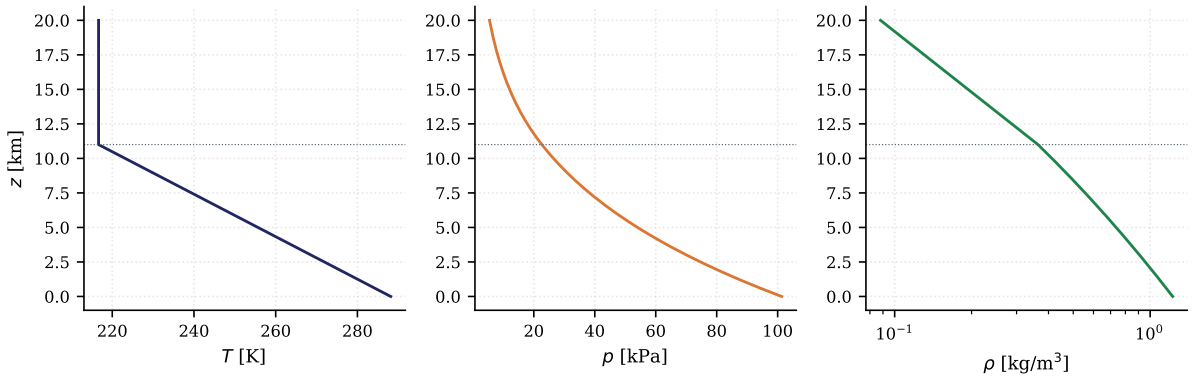



Figura 1.1: Programma 0.1: temperatura, pressione e densità dell'atmosfera standard fino a 20 km; la linea punteggiata è la tropopausa.

**Leggere il codice: tre righe**

`for z in range(0, 20001, 2000):` ripete il blocco indentato per $z = 0, 2000, \ldots, 20000$ (il valore finale è escluso, per questo 20001). `T, p, rho, a = isa(z)` chiama la funzione e distribuisce i quattro valori restituiti su quattro variabili. `print(f"{z:8d} {T:8.2f} ...")` stampa una riga della tabella con le colonne allineate. L'*indentazione* (i quattro spazi all'inizio delle righe dentro il `for`) non è ornamento: in Python delimita i blocchi, al posto delle parentesi graffe di altri linguaggi.

## 1.6 Approfondimenti

### 1.6.1 Lo standard IEEE 754 e l'arrotondamento

Un `float` occupa 64 bit: 1 di segno, 11 di esponente, 52 di mantissa (più un bit implicito, da cui le 53 cifre binarie, circa 15,9 decimali). I numeri rappresentabili non sono equispaziati: fra 1 e 2 la spaziatura è $\varepsilon = 2^{-52}$, fra $2^{10}$ e $2^{11}$ è $2^{10}\varepsilon$; l'errore *relativo* di arrotondamento è però sempre al più $\varepsilon/2$ (arrotondamento al più vicino). Sotto $2^{-1022} \approx 2{,}2 \times 10^{-308}$ i numeri "denormalizzati" perdono gradualmente precisione fino a $5 \times 10^{-324}$, e sopra $1{,}8 \times 10^{308}$ si ha `inf`. Le operazioni elementari $(+, -, \times, /, \sqrt{\ })$ sono *correttamente arrotondate*: il risultato è il float più vicino al risultato esatto. Questo permette di ragionare sugli errori: la somma di $n$ termini positivi in ordine qualsiasi ha errore relativo al più $(n-1)\varepsilon$, mentre la somma dal più piccolo al più grande, o la somma compensata di Kahan (Esercizio 3), lo riducono a $\sim \varepsilon$. Nei codici del corso tutto questo si traduce in poche regole: non confrontare float con `==`; non sottrarre quantità quasi uguali se esiste una formula equivalente; non scegliere passi di differenze finite troppo piccoli; e controllare `np.isfinite`.

### 1.6.2 Notebook, ambienti e versioni

Il *notebook* Jupyter alterna celle di codice, risultati e testo: è l'ambiente in cui è stato scritto il corso "CFD Python" [3] ed è ottimo per esplorare; i programmi del corso sono però file `.py`, perché un file si esegue dall'inizio alla fine in modo riproducibile, si versiona e si importa da altri programmi (Cap. 2), mentre un notebook eseguito in ordine sparso può dare risultati che dipendono dalla storia delle celle. Un *ambiente virtuale* (`python3 -m venv cfd`, poi `source cfd/bin/activate`) isola le librerie di un progetto; le versioni usate per le

monografie sono Python 3.11–3.13, NumPy $\geq$ 1.26 (con `np.trapezoid`, che ha sostituito `np.trapz`), SciPy $\geq$ 1.11, Matplotlib $\geq$ 3.8, SymPy $\geq$ 1.12. `python3 -c "import numpy; print(numpy.__version__)"` dice quale versione è installata.

# Capitolo 2

# Controllo del flusso e funzioni

## 2.1 Decisioni e ripetizioni

Un algoritmo numerico è fatto di decisioni (“se il residuo è sotto la tolleranza, fermati”) e di ripetizioni (“per ogni passo temporale”, “finché non converge”). Python le esprime con tre istruzioni, tutte seguite da due punti e da un blocco *indentato*.

**`if`, `for`, `while`**

```
if Re < 1.0:                  # ramo eseguito se la condizione e' vera
    cd = 24.0 / Re
elif Re < 400.0:              # altrimenti, se...
    cd = 24.0 / Re**0.646
else:                         # in tutti gli altri casi
    cd = 0.5

for k in range(1, n + 1):     # k = 1, 2, ..., n
    somma += 1.0 / k**2

while abs(x_nuovo - x) > tol:   # ripete finche' la condizione e' vera
    x = x_nuovo
    x_nuovo = x - f(x) / fp(x)
```

`range(a, b, passo)` genera gli interi da `a` (incluso) a `b` (escluso): `range(n)` è $0, \ldots, n-1$, `range(0, 10, 2)` è $0, 2, 4, 6, 8$, `range(n, 0, -1)` conta all’indietro. Il `for` itera su qualunque sequenza: una lista, una stringa, un array, le righe di un file. Dentro un ciclo, `break` esce subito, `continue` salta all’iterazione successiva. Due funzioni accompagnano quasi sempre il `for` nei programmi del corso: `enumerate(seq)` fornisce coppie (indice, elemento), `zip(a, b)` fornisce coppie di elementi corrispondenti di due sequenze:

```
for i, (v, s) in enumerate(zip(velocita, segni)):
    if i > 0 and segni[i - 1] * s < 0:
        break                 # trovato il cambio di segno
```

**Il `while` e il tempo che si accumula**

Il modo più naturale di scrivere una marcia nel tempo, `while t < t_fin: t += h`, nasconde una trappola: `t` accumula un errore di arrotondamento a ogni somma e, dopo molti passi, può valere $t_{\text{fin}} - 10^{-15}$ invece di $t_{\text{fin}}$: il ciclo esegue un passo in più (o in meno)

e il risultato finale è sbagliato di $h \cdot \mathrm{d}u/\mathrm{d}t$. Nel Programma 0.2 questo errore, con $h = 10^{-4}$, produceva uno scarto di $8 \times 10^{-5}$ che rovinava la misura dell'ordine di convergenza. La forma robusta fissa il numero di passi in anticipo e calcola il tempo per moltiplicazione: `n = round(t_fin / h); for k in range(1, n + 1): ... t = k * h`. I programmi delle monografie che usano `while` aggiungono una tolleranza (`while t < t_fin - 1e-12`) e scelgono passi che dividono esattamente l'intervallo.

## 2.2 Funzioni

Una **funzione** racchiude un calcolo sotto un nome, con argomenti in ingresso e un valore restituito da `return`:

```
def forza_netta(v, d, m, cd=cd_sfera):
    """f(v) = peso - resistenza, per unita' di massa; cd e' la legge di
    resistenza."""
    Re = max(v * d / NU, 1e-9)
    return G - 0.5 * RHO_A * v * abs(v) * math.pi * d**2 / 4 * cd(Re) / m
```

La stringa fra triple virgolette subito dopo `def` è la **docstring**: documenta che cosa fa la funzione, e `help(forza_netta)` la mostra. L'argomento `cd=cd_sfera` ha un *valore predefinito*: si può omettere nella chiamata (`forza_netta(v, d, m)`) o specificare per nome (`forza_netta(v, d, m, cd=cd_liscio)`). Una funzione può restituire più valori, in realtà una tupla, che la chiamata spacchetta: `T, p, rho, a = isa(z)`.

### Le funzioni sono valori: argomenti e `lambda`

In Python una funzione è un oggetto come un numero: si può assegnare a una variabile, mettere in un dizionario, passare come argomento. È ciò che rende generici i metodi numerici: la bisezione e il metodo di Newton del Programma 0.2 ricevono `f` come argomento e funzionano per qualunque equazione; l'integratore `integra(F, v0, h, t_fin)` riceve il secondo membro dell'equazione differenziale; `ode_solvers.py` del Modulo 1 tiene in un dizionario `METODI = {"eulero": passo_eulero, "rk4": passo_rk4, ...}` i passi elementari e li sceglie per nome. Una **lambda** è una funzione anonima di una riga, comoda per adattare gli argomenti: `f = lambda v: forza_netta(v, d, m)` fissa `d` e `m` e produce la funzione di una sola variabile che la bisezione vuole. Nei programmi del corso le lambda compaiono ovunque come "adattatori": `lambda t, y: [y[1], -np.sin(y[0])]` è il secondo membro del pendolo per `solve_ivp`, `key=lambda s: profili[s]` è la chiave di ordinamento di `sorted`.

### Ambito delle variabili

Le variabili assegnate dentro una funzione sono *locali*: nascono alla chiamata e muoiono al `return`, e non interferiscono con l'esterno. Una funzione può *leggere* le variabili globali (le costanti `G`, `NU`, `RHO_A` del programma) senza dichiarazioni; per *modificarne* una deve dichiararla con `global`, altrimenti l'assegnazione crea una variabile locale omonima. Nei programmi del corso le funzioni leggono le costanti globali (in maiuscolo) e ricevono tutto il resto come argomento: è la forma che rende una funzione riutilizzabile e verificabile. Le funzioni annidate (una `def` dentro un'altra, come `F(v)` dentro `caduta` nel Programma 1.1 del Modulo 1) vedono le variabili della funzione che le contiene:

servono a costruire “al volo” il secondo membro di un’equazione con i parametri correnti.

### 2.2.1 Ricorsione

Una funzione può chiamare se stessa: `fattoriale(n)` restituisce `n * fattoriale(n - 1)` con il caso base `fattoriale(0) = 1`. La ricorsione è elegante per le definizioni ricorsive (fattoriale, Fibonacci, polinomi di Legendre, suddivisioni di griglia) ma costosa se ricalcola gli stessi valori: `fibonacci(n)` ingenuo costa $\sim 1{,}6^n$ chiamate; con la *memorizzazione* (un dizionario dei valori già calcolati, come nel Programma 0.2) diventa lineare. Nei codici numerici la ricorsione è rara: i cicli sono più semplici e più veloci, e Python limita la profondità delle chiamate a circa mille.

## 2.3 Moduli e il blocco principale

Ogni file `.py` è un **modulo**: `import math` lo carica e `math.sqrt` ne usa una funzione; `from stile_grafici import salva, BLU` importa singoli nomi; `import numpy as np` assegna un’abbreviazione (quelle del corso: `np`, `plt`, `sp` per SymPy, `sp/spla` per le matrici sparse). Quando un modulo viene importato, il suo codice viene *eseguito* dall’inizio alla fine: se il file contiene anche i calcoli e le stampe del programma, importarlo per riusarne una funzione eseguirebbe tutto. La soluzione, adottata da tutti i programmi del corso che altri programmi importano, è il blocco

```
if __name__ == "__main__":
    ...   # calcoli, stampe e figure: eseguiti solo quando il file e' lanciato
    direttamente
```

La variabile speciale `__name__` vale `"__main__"` se il file è eseguito come programma e il nome del modulo se è importato. Così `prog0_2_controllo_funzioni.py` definisce `bisezione`, `newton`, `integra` importabili da altri file (gli esercizi li usano), e produce le sue tabelle solo con `python3 prog0_2_controllo_funzioni.py`.

## 2.4 Il Programma 0.2: velocità limite e caduta di una sfera

L’esempio conduttore ripropone il primo problema del Modulo 1. La *velocità limite* $v_t$ di una sfera in caduta è la radice di

$$f(v) = g - \frac{\frac{1}{2}\rho_a v^2 \frac{\pi d^2}{4} c_d(Re)}{m} = 0, \qquad Re = \frac{vd}{\nu}, \tag{2.1}$$

con $c_d(Re)$ a tratti (una catena di `if/elif`); si trova prima un intervallo con cambio di segno (ciclo `for` con `break`), poi la radice con la **bisezione** (dimezza l’intervallo: un bit per iterazione) e con il **metodo di Newton** (convergenza quadratica, derivata numerica). Il moto $\mathrm{d}v/\mathrm{d}t = f(v)$ è poi integrato con Eulero e con Runge–Kutta 4 e se ne misura l’ordine di convergenza.

**Dati del Programma 0.2**

Sfera d’acciaio $d = 1$ cm ($m = 4{,}08$ g), aria $\rho_a = 1{,}225$ kg/m$^3$, $\nu = 1{,}46 \times 10^{-5}$ m$^2$/s; tolleranze $10^{-10}$ (bisezione) e $10^{-12}$ (Newton); integrazione fino a $t = 8$ s con $h = 0{,}4$, 0,2, 0,1 e riferimento $h = 10^{-4}$; due leggi di resistenza, quella a tratti e la correlazione

regolare di Clift–Gauvin.


prog0_2_controllo_funzioni.py

```python
"""
prog0_2_controllo_funzioni.py -- Programma 0.2: controllo del flusso e funzioni.

Esempio conduttore: la velocita' limite di una sfera in caduta in aria,
radice dell'equazione  f(v) = m g - (1/2) rho_a v^2 (pi d^2/4) c_d(Re) = 0,
con c_d(Re) a tratti (Re = v d/nu); e il moto della stessa sfera integrato
con il metodo di Eulero e con Runge-Kutta 4 fino alla velocita' limite.
Il programma mostra: if/elif/else, for e while, break/continue, range,
enumerate e zip, definizione di funzioni con argomenti predefiniti e
docstring, funzioni come argomenti (bisezione e Newton ricevono f), lambda,
ricorsione (fattoriale e Fibonacci), variabili locali e globali, moduli e
il blocco  if __name__ == "__main__".
Verifica: Newton e bisezione convergono alla stessa radice; il moto tende
a v_t; RK4 e' del quarto ordine (l'errore si divide per 16 dimezzando h).
"""
import math
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

RHO_A, NU, G = 1.225, 1.46e-5, 9.80665          # aria al livello del mare


# ---------------------------------------------------------------- funzioni con if/elif/else
def cd_sfera(Re):
    """Coefficiente di resistenza di una sfera liscia: spezzata a tratti in Re."""
    if Re < 1e-9:
        return 1e9                          # evita la divisione per zero: resistenza di Stokes enorme
    elif Re < 1.0:
        return 24.0 / Re
    elif Re < 400.0:
        return 24.0 / Re**0.646
    elif Re < 3e5:
        return 0.5
    elif Re < 2e6:
        return 3.66e-4 * Re**0.4275
    else:
        return 0.18


def cd_liscio(Re):
    """Correlazione di Clift-Gauvin, regolare per ogni Re > 0 (valida fino a Re ~ 3e5)."""
    return 24.0 / Re * (1 + 0.15 * Re**0.687) + 0.42 / (1 + 42500.0 * Re**-1.16)


def forza_netta(v, d, m, cd=cd_sfera):
    """f(v) = peso - resistenza, per unita' di massa; cd e' la legge di resistenza (funzione di Re)."""
    Re = max(v * d / NU, 1e-9)
    return G - 0.5 * RHO_A * v * abs(v) * math.pi * d**2 / 4 * cd(Re) / m


# ---------------------------------------------------------------- funzioni che ricevono funzioni
```

```
def bisezione(f, a, b, tol=1e-10, it_max=200):
    """Radice di f in [a, b] (f(a) f(b) < 0) con il metodo di bisezione.
    Restituisce (x, iterazioni)."""
    fa, fb = f(a), f(b)
    if fa * fb > 0:
        raise ValueError("f(a) e f(b) hanno lo stesso segno")
    for k in range(1, it_max + 1):
        c = 0.5 * (a + b)
        fc = f(c)
        if fc == 0.0 or 0.5 * (b - a) < tol:
            return c, k
        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc
    return 0.5 * (a + b), it_max


def newton(f, x0, tol=1e-12, it_max=50, h=1e-6):
    """Radice di f con il metodo di Newton e derivata numerica centrata.
    Restituisce (x, iterazioni, storia)."""
    x = x0
    storia = [x]
    for k in range(1, it_max + 1):
        fp = (f(x + h) - f(x - h)) / (2 * h)
        x_nuovo = x - f(x) / fp
        storia.append(x_nuovo)
        if abs(x_nuovo - x) < tol:
            return x_nuovo, k, storia
        x = x_nuovo
    return x, it_max, storia


# ---------------------------------------------------------------- integrazione
    nel tempo
def integra(F, v0, h, t_fin, metodo="rk4"):
    """Integra dv/dt = F(v) da 0 a t_fin; restituisce le liste t e v.
    Il numero di passi e' fissato in anticipo (n = t_fin/h) e il tempo e' calcolato
    come k h:
    accumulare t += h in un ciclo while produce errori di arrotondamento che possono
    aggiungere o togliere un passo (vedi Programma 0.9)."""
    n = round(t_fin / h)
    v = v0
    T, V = [0.0], [v]
    for k in range(1, n + 1):
        if metodo == "eulero":
            v = v + h * F(v)
        elif metodo == "rk4":
            k1 = h * F(v)
            k2 = h * F(v + k1 / 2)
            k3 = h * F(v + k2 / 2)
            k4 = h * F(v + k3)
            v = v + (k1 + 2 * k2 + 2 * k3 + k4) / 6
        else:
            raise ValueError(f"metodo sconosciuto: {metodo}")
        T.append(k * h); V.append(v)
    return T, V


# ---------------------------------------------------------------- ricorsione
```

```python
def fattoriale(n):
    """n! per ricorsione: n! = n (n-1)!, 0! = 1."""
    return 1 if n == 0 else n * fattoriale(n - 1)


def fibonacci(n, memo={}):
    """Numero di Fibonacci con memorizzazione (senza, il costo cresce come 1,6^n)."""
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fibonacci(n - 1) + fibonacci(n - 2)
    return memo[n]


contatore = 0                                       # variabile globale


def incrementa():
    global contatore                                # senza 'global' l'assegnazione creerebbe una variabile locale
    contatore += 1


if __name__ == "__main__":
    d, rho_s = 0.01, 7800.0                         # sfera d'acciaio da 1 cm
    m = rho_s * math.pi * d**3 / 6
    f = lambda v: forza_netta(v, d, m)              # funzione anonima di una sola variabile
    print(f"Sfera d'acciaio, d = {d} m, m = {m*1000:.2f} g")

    # --- cicli for con range, enumerate, zip, break e continue
    print("\n1) Ricerca di un intervallo con cambio di segno (for con break):")
    velocita = [10, 20, 30, 40, 50, 60]
    segni = [f(v) for v in velocita]
    for i, (v, s) in enumerate(zip(velocita, segni)):
        print(f"   v = {v:3d} m/s: f = {s:+.3f}", end="")
        if i > 0 and segni[i - 1] * s < 0:
            print("   <- cambio di segno")
            a, b = velocita[i - 1], v
            break
        print()
    print("\n2) Bisezione e Newton:")
    vb, kb = bisezione(f, a, b)
    vn, kn, storia = newton(f, 10.0)
    print(f"   bisezione: v_t = {vb:.8f} m/s in {kb} iterazioni (ogni iterazione guadagna un bit)")
    print(f"   Newton:    v_t = {vn:.8f} m/s in {kn} iterazioni; errori successivi: "
          + ", ".join(f"{abs(x - vn):.1e}" for x in storia[:-1]))
    Re_t = vn * d / NU
    print(f"   Re = {Re_t:.0f}, c_d = {cd_sfera(Re_t)}; verifica dell'equilibrio: f(v_t) = {f(vn):.1e}")

    # --- while e integrazione nel tempo; ordine di convergenza
    print("\n3) Moto della sfera: v(t) con Eulero e RK4, confronto con v_t")
    t_fin = 8.0
    for metodo in ("eulero", "rk4"):
        T, V = integra(f, 0.0, 0.5, t_fin, metodo)
        print(f"   {metodo:6s} h = 0.5: " + "  ".join(f"v({t:.0f})={v:6.3f}" for t,
```

```
    v in zip(T[::4], V[::4])))
163 T, V = integra(f, 0.0, 0.01, 30.0, "rk4")
164 k = 0
165 while V[k] < 0.99 * vn:                    # ciclo while: avanza finche' la
    condizione e' vera
166     k += 1
167 print(f"   il 99% della velocita' limite e' raggiunto a t = {T[k]:.2f} s (ciclo
    while sulla soluzione RK4)")
168 print("   ordine di convergenza (errore a t = 8 rispetto a h = 1e-4), con la
    legge c_d a tratti e con quella regolare:")
169 for nome, cd in (("c_d a tratti", cd_sfera), ("c_d regolare", cd_liscio)):
170     fl = lambda v: forza_netta(v, d, m, cd)
171     _, Vrif = integra(fl, 0.0, 1e-4, t_fin, "rk4")
172     for metodo in ("eulero", "rk4"):
173         errori = []
174         for h in (0.4, 0.2, 0.1):
175             _, V = integra(fl, 0.0, h, t_fin, metodo)
176             errori.append(abs(V[-1] - Vrif[-1]))
177         rapporti = [errori[i] / errori[i + 1] for i in range(len(errori) - 1)]
178         print(f"   {nome}, {metodo:6s}: errori {', '.join(f'{e:.2e}' for e in
    errori)}; rapporti "
179               f"{', '.join(f'{r:.1f}' for r in rapporti)} (attesi {2 if metodo
    == 'eulero' else 16})")
180 print("   con la legge a tratti (derivata discontinua in Re = 400) RK4 perde
    l'ordine: il quarto ordine")
181 print("   richiede una funzione F regolare; con la legge regolare il rapporto
    16 e' rispettato.")
182
183 # --- ricorsione e variabili globali
184 print("\n4) Ricorsione e ambito delle variabili")
185 print(f"   10! = {fattoriale(10)}, F(50) = {fibonacci(50)}, F(90) =
    {fibonacci(90)}")
186 for _ in range(3):
187     incrementa()
188 print(f"   contatore globale dopo tre chiamate: {contatore}")
189 print(f"   il modulo si chiama {__name__!r}: questo blocco gira solo se il file
    e' eseguito direttamente")
190
191 # --- figura
192 T, V = integra(f, 0.0, 0.05, t_fin, "rk4")
193 Te, Ve = integra(f, 0.0, 0.5, t_fin, "eulero")
194 fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
195 ax[0].plot(T, V, color=BLU, label="RK4, $h=0{,}05$")
196 ax[0].plot(Te, Ve, "o--", color=ARANCIO, ms=3, label="Eulero, $h=0{,}5$")
197 ax[0].axhline(vn, color=GRIGIO, ls=":", label="$v_t$")
198 ax[0].set_xlabel("$t$ [s]"); ax[0].set_ylabel("$v$ [m/s]");
    ax[0].legend(fontsize=7)
199 err = [abs(x - vn) for x in storia[:-1]]
200 ax[1].semilogy(range(len(err)), err, "o-", color=BLU, label="Newton")
201 ax[1].semilogy(range(kb), [abs(b - a) / 2**(k + 1) for k in range(kb)], "-",
    color=ARANCIO, label="bisezione (ampiezza)")
202 ax[1].set_xlabel("iterazione"); ax[1].set_ylabel("errore [m/s]");
    ax[1].legend(fontsize=7)
203 fig.tight_layout()
204 salva(fig, "prog0_2_funzioni")
```

Listato 2.1: Programma 0.2 – cicli, funzioni, bisezione e Newton, Eulero e Runge–Kutta sulla caduta di una sfera (`prog0_2_controllo_funzioni.py`).

**Risultati del Programma 0.2**

- Il cambio di segno di $f$ è fra 40 e 50 m/s. La bisezione converge a $v_t = 40{,}80599396$ m/s in 37 iterazioni; Newton allo stesso valore in 7, con errori 31, 47, 13, 1,5, 0,027, $9 \times 10^{-6}$, $10^{-12}$: una volta vicino, le cifre esatte raddoppiano a ogni passo (Fig. 2.1). $Re_t = 27\,949$, $c_d = 0{,}5$.
- Il moto: Eulero con $h = 0{,}5$ sovrastima $v$ (39,6 contro 39,1 m/s a $t = 8$); il 99% di $v_t$ è raggiunto a $t = 11{,}0$ s (ciclo `while` sulla soluzione).
- Ordine di convergenza. Eulero: errori che si dimezzano con $h$ (rapporti 2,0; ordine 1) con entrambe le leggi. RK4: con la legge regolare i rapporti sono 15,3 e 13,1 (ordine 4, atteso 16); con la legge a tratti 26,9 e 8,4: la discontinuità della derivata di $c_d$ in $Re = 400$ fa perdere l'ordine. Un metodo di ordine $p$ richiede $p$ derivate continue del secondo membro.
- $10! = 3\,628\,800$, $F(90) = 2\,880\,067\,194\,370\,816\,120$ (intero esatto); `__name__` vale `'__main__'`.

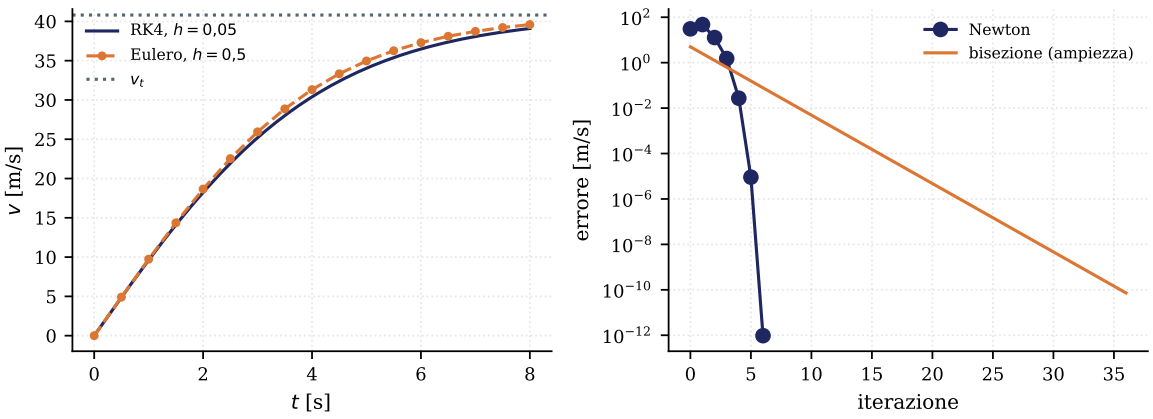



Figura 2.1: Programma 0.2: velocità della sfera con RK4 ed Eulero rispetto alla velocità limite (a sinistra); errore del metodo di Newton e ampiezza dell'intervallo di bisezione in funzione dell'iterazione (a destra).

**Dove ritrovare questi costrutti nel corso**

La catena `if/elif` di `cd_sfera` è la stessa di `ode_solvers.cd_sfera` nel Modulo 1; il ciclo `while t < tmax - 1e-12` con i quattro stadi di Runge–Kutta è il cuore di `prog1_1_caduta_sfera.py` (Cap. 8); il metodo di Newton a una e a due variabili (`newtn1`, `newtn2`) risolve il sistema di Galerkin della sfera nel Modulo 4; la bisezione sulla curvatura della velocità trova il numero di Reynolds di separazione nello stesso programma; il *tiro* del Modulo 3 è un ciclo `while` che corregge $f''(0)$ finché la condizione all'infinito è soddisfatta.

## 2.5 Approfondimenti

### 2.5.1 Quando usare `for` e quando `while`

Il `for` è adatto quando il numero di iterazioni è noto in anticipo (passi temporali a passo fisso, nodi di una griglia, righe di una tabella); il `while` quando dipende dal risultato (iterazioni fino a convergenza, ricerca di un evento). Un `while` senza un limite massimo di iterazioni è un rischio: se la convergenza non arriva, il programma non termina. I programmi del corso scrivono `for it in range(1, it_max + 1):` con un `return` o `break` alla convergenza e un errore (`raise RuntimeError`) se si esce dal ciclo senza averla raggiunta: è la forma di `sorlx` nel Modulo 4 e di `sor_poisson` in `metodi_numerici.py`.

### 2.5.2 L'ordine di un metodo e la regolarità del problema

Il risultato "RK4 perde l'ordine con la legge $c_d$ a tratti" merita una riflessione. L'errore locale di un metodo di Runge–Kutta di ordine $p$ è $\mathcal{O}(h^{p+1})$ perché gli sviluppi di Taylor della soluzione fino all'ordine $p$ coincidono con quelli del metodo: ciò richiede che $F$ abbia $p$ derivate continue. Se $F$ ha un salto nella derivata prima (il "gomito" di $c_d$ a $Re = 400$, cioè $v = 0{,}58$ m/s), nel passo che attraversa il gomito l'errore locale è $\mathcal{O}(h^2)$ qualunque sia $p$, e l'errore globale è dominato da quel passo: $\mathcal{O}(h^2)$ nel migliore dei casi, con rapporti irregolari (26,9 e 8,4) perché dipendono da dove cade il gomito rispetto ai nodi. La lezione è generale: le correlazioni empiriche a tratti, comodissime, vanno raccordate con continuità delle derivate (o si accetta il primo ordine), e la *misura* dell'ordine di convergenza è la verifica più semplice e più severa che un programma può fare di se stesso. Nel Programma 0.2 è stata proprio questa misura, che dava rapporti 1,5 invece di 16, a rivelare prima l'errore dell'accumulo del tempo e poi quello della legge a tratti.

### 2.5.3 Argomenti per nome, `*args` e `**kwargs`

Una chiamata può passare gli argomenti per posizione o per nome (`cavita(100, 64, 30.0, tol=1e-6, stampa=False)`): i nomi rendono leggibili le chiamate con molti parametri e permettono di ometterne alcuni. Una funzione può accettare un numero variabile di argomenti con `*args` (tupla) e `**kwargs` (dizionario); nei programmi del corso compare solo lo *spacchettamento* nella chiamata, `f(*p)` per passare gli elementi di un vettore `p` come argomenti separati (Esercizio 12), e `dict(...)` per restituire molti risultati con un nome ciascuno, come fa `cavita()` nel Modulo 4: `r = cavita(...)`, poi `r["psi"]`, `r["u"]`.

# Capitolo 3

# Liste, tuple, dizionari e stringhe

## 3.1 Le collezioni di Python

Un programma di calcolo maneggia raramente un numero alla volta: ha quote, nodi, coefficienti, parametri. Prima degli array di NumPy (Cap. 4), che sono lo strumento per i campi di moto, Python offre quattro *collezioni* che i programmi del corso usano di continuo per tutto ciò che non è un vettore numerico: la **lista**, la **tupla**, il **dizionario** e l'**insieme**; la **stringa** si comporta come una sequenza di caratteri.

**Liste e tuple**

Una **lista** è una sequenza ordinata e modificabile, scritta fra parentesi quadre: `quote = [-500, 0, 2000, 4000]`. Gli elementi si raggiungono con l'indice a partire da zero (`quote[0]` è $-500$), gli indici negativi contano dalla fine (`quote[-1]` è l'ultimo), e lo **slicing** `quote[a:b:passo]` estrae da `a` incluso a `b` escluso: `quote[1:3]`, `quote[::2]` (uno ogni due), `quote[::-1]` (rovesciata), `quote[1:-1]` (tutti tranne il primo e l'ultimo: i *nodi interni* di una griglia, la forma che ritroveremo in ogni schema alle differenze). Operazioni: `len`, `append`, `insert`, `remove`, `pop`, `in`, `index`, `sort`/`sorted`, `sum`, `min`, `max`, concatenazione con `+` e ripetizione con `*`. Una lista può contenere qualunque cosa, anche altre liste.
Una **tupla** è una sequenza *immutabile*, scritta fra parentesi tonde: `punto = (0.25, 0.05)`. Serve per i dati che non devono cambiare e per *raggruppare* valori: una funzione che restituisce `T, p, rho, a` restituisce una tupla, e la chiamata la spacchetta in quattro variabili. Attenzione: `(3,)` è una tupla di un elemento, `(3)` è il numero 3.

**Copie e riferimenti: la trappola degli alias**

L'assegnazione `b = a` *non copia* la lista: `a` e `b` sono due nomi dello stesso oggetto, e `b[0] = 99` modifica anche `a`. Per avere una copia indipendente si scrive `c = a.copy()` (o `a[:]`, o `list(a)`). La variante più insidiosa è `m = [[0]*3]*2`: sembra una matrice $2 \times 3$ di zeri, ma le due righe sono *lo stesso oggetto*, e `m[0][0] = 7` cambia entrambe. La forma corretta è `[[0]*3 for _ in range(2)]`, e per le matrici numeriche si usa comunque `np.zeros((2, 3))`. Lo stesso meccanismo vale per gli array di NumPy (`u_vecchio = u` non salva il livello temporale precedente: serve `u.copy()`, Cap. 9) ed è il primo errore silenzioso che chi scrive una marcia nel tempo incontra.

## 3.2 Comprehension

La **list comprehension** costruisce una lista con un'espressione e un ciclo in una riga sola:

```
pari = [k for k in range(20) if k % 2 == 0]
spessori = [parametri_naca(s)[2] for s in sigle]
quadrati = {k: k**2 for k in range(5)}              # dict comprehension
errs = [errore(n, R) for n in ns]                    # Programma 0.8
dati = np.array([isa(z) for z in zz])                # Programma 0.1: tabella da
    una funzione scalare
```

È più leggibile e più veloce dell'equivalente ciclo con `append`, ed è onnipresente nei programmi del corso per costruire tabelle di risultati ("l'errore per ogni griglia", "la soluzione per ogni passo"). Con `zip` si combinano due sequenze elemento per elemento e con `sorted(seq, key=f)` si ordina secondo una chiave: `sorted(sigle, key=lambda s: parametri_naca(s)[2])` ordina i profili per spessore.

## 3.3 Dizionari e insiemi

**Dizionari**

Un **dizionario** associa chiavi a valori: `fluidi = {"aria": (1.225, 1.46e-5), "acqua": (998.0, 1.0e-6)}`; `fluidi["aria"]` restituisce la tupla, `fluidi["elio"]` solleva `KeyError`, `fluidi.get("elio", "assente")` restituisce un valore predefinito; `"elio" in fluidi` controlla la presenza; `for nome, (rho, nu) in fluidi.items()`: scorre le coppie. I dizionari mantengono l'ordine di inserimento. Nei programmi del corso compaiono con tre ruoli: *tabelle di parametri* (`GHIA = {100: (...), 400: (...)}` nel Programma 4.5; `PROFILI`, `FLUIDI`); *selezione per nome* di una funzione (`METODI["rk4"]` in `ode_solvers.py`); *risultati con un nome ciascuno* (`return dict(Re=Re, psi=psi, u=u, ...)` in `cavita()`, poi `r["psi"]`). Il formato JSON dei file di parametri (Cap. 9) è un dizionario scritto su disco.

Un **insieme** (`set`) è una collezione senza ordine e senza ripetizioni: `set(spessori)` elimina i duplicati, `&` e `|` fanno intersezione e unione. Serve raramente nei codici numerici, ma `sorted(set(...))` è il modo idiomatico per ottenere i valori distinti di una lista.

## 3.4 Stringhe come sequenze

Una stringa è una sequenza immutabile di caratteri: `s[0]`, `s[-4:]`, `len(s)`, `for c in s` funzionano come per le liste. I metodi più usati: `upper`, `lower`, `strip` (toglie gli spazi), `split` (spezza in una lista di parole: `"1.0 2.0 3.0".split()` dà tre stringhe), `join` (l'inverso: `" ".join(f"{v:.4f}" for v in u)` è la riga della tabella nel Programma 3.4), `replace`, `startswith`, `int(s)` e `float(s)` per convertire. La sigla NACA a quattro cifre è l'esempio perfetto: `"2412"` contiene tre numeri, $m$ = `int(sigla[0])/100`, $p$ = `int(sigla[1])/10`, $t$ = `int(sigla[2:])/100`, e una sigla non valida va intercettata con un `raise ValueError`.

## 3.5 Il Programma 0.3: profili NACA a quattro cifre

Il profilo NACA *mptt* ha distribuzione di spessore

$$\frac{y_t}{c} = \frac{t}{0{,}2}\left(0{,}2969\sqrt{\xi} - 0{,}1260\,\xi - 0{,}3516\,\xi^2 + 0{,}2843\,\xi^3 - 0{,}1015\,\xi^4\right), \qquad \xi = x/c, \tag{3.1}$$

e linea media parabolica a due tratti con curvatura massima $m$ in $\xi = p$. Il programma decodifica la sigla, costruisce dorso e ventre con `profilo(sigla, n)`, calcola lo spessore massimo, l'area della sezione con la regola dei trapezi scritta come comprehension e ordina un elenco di profili per spessore; tutto con liste, tuple, dizionari e stringhe, senza NumPy (che compare solo per la figura).

**Dati del Programma 0.3**

Sigle `0009`, `0012`, `2412`, `4415`, `6409`; $n = 100$ punti per lato con spaziatura a coseno; fluidi aria, acqua, olio in un dizionario; valori di confronto: spessore massimo del NACA 0012 pari a $0{,}12\,c$ in $x = 0{,}30\,c$, area $0{,}0822\,c^2$ con il bordo d'uscita aperto (coefficiente $-0{,}1015$) e $0{,}0817\,c^2$ con il bordo chiuso ($-0{,}1036$).

prog0_3_strutture_dati.py

```python
"""
prog0_3_strutture_dati.py -- Programma 0.3: liste, tuple, dizionari, insiemi e stringhe.

Esempio conduttore: la geometria dei profili NACA a quattro cifre. La sigla
"2412" e' una stringa da cui si estraggono curvatura massima m = 2 %, posizione
p = 40 % e spessore t = 12 %; la linea media e la distribuzione di spessore sono
calcolate con liste (senza NumPy, che arriva nel Programma 0.4), i punti del
profilo sono tuple (x, y), un dizionario raccoglie i profili per sigla.
Il programma mostra: indici e slicing, metodi delle liste, list comprehension,
spacchettamento delle tuple, dizionari e insiemi, ordinamento con chiave, la
differenza fra copia e riferimento (la trappola degli alias), le stringhe.
Verifica: lo spessore massimo del NACA 0012 e' 0,12 c in x = 0,30 c (tabulato:
0,3 c); l'area della sezione del NACA 0012 e' 0,0817 c^2 (0,0822 con il bordo
d'uscita aperto).
"""
import math
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

def parametri_naca(sigla):
    """Da '2412' restituisce (m, p, t) = (0,02, 0,4, 0,12)."""
    if len(sigla) != 4 or not sigla.isdigit():
        raise ValueError(f"sigla non valida: {sigla!r}")
    return int(sigla[0]) / 100, int(sigla[1]) / 10, int(sigla[2:]) / 100


def spessore(x, t):
    """Semispessore NACA a quattro cifre (bordo d'uscita chiuso con l'ultimo coefficiente -0,1036)."""
    return 5 * t * (0.2969 * math.sqrt(x) - 0.1260 * x - 0.3516 * x**2 + 0.2843 * x**3 - 0.1036 * x**4)


def linea_media(x, m, p):
    """Ordinata e pendenza della linea media NACA a quattro cifre."""
    if m == 0:
        return 0.0, 0.0
```

```
    if x < p:
        return m / p**2 * (2 * p * x - x**2), 2 * m / p**2 * (p - x)
    return m / (1 - p)**2 * (1 - 2 * p + 2 * p * x - x**2), 2 * m / (1 - p)**2 * (p - x)


def profilo(sigla, n=100):
    """Punti (x, y) del dorso e del ventre, come liste di tuple; ascisse addensate al bordo d'attacco."""
    m, p, t = parametri_naca(sigla)
    xs = [0.5 * (1 - math.cos(math.pi * i / n)) for i in range(n + 1)]     # spaziatura coseno
    dorso, ventre = [], []
    for x in xs:
        yc, dyc = linea_media(x, m, p)
        yt = spessore(x, t)
        th = math.atan(dyc)
        dorso.append((x - yt * math.sin(th), yc + yt * math.cos(th)))
        ventre.append((x + yt * math.sin(th), yc - yt * math.cos(th)))
    return dorso, ventre



if __name__ == "__main__":
    # ------------------------------------------------------------------ 1) liste
    print("1) Liste")
    quote = [0, 2000, 4000, 6000, 8000]              # lista di interi
    quote.append(10000)                              # aggiunge in coda
    quote.insert(0, -500)                            # inserisce in testa
    print(f"   {quote}, lunghezza {len(quote)}, primo {quote[0]}, ultimo {quote[-1]}, secondo-terzo {quote[1:3]}")
    print(f"   ogni due {quote[::2]}, rovesciata {quote[::-1]}, 4000 in lista? {4000 in quote}, indice di 4000: {quote.index(4000)}")
    quote.remove(-500)
    ultimo = quote.pop()                             # toglie e restituisce l'ultimo
    print(f"   dopo remove e pop: {quote} (tolto {ultimo}); somma {sum(quote)}, massimo {max(quote)}, ordinata decrescente {sorted(quote, reverse=True)}")
    mista = [1, "due", 3.0, [4, 5]]                  # una lista puo' contenere tipi diversi (ma e' raro che convenga)
    print(f"   lista eterogenea: {mista}, elemento annidato {mista[3][1]}")

    # ------------------------------------------------------------------ 2) copie e riferimenti
    print("\n2) Copie e riferimenti: la trappola degli alias")
    a = [1, 2, 3]
    b = a                                            # b e' un altro NOME dello stesso oggetto
    b[0] = 99
    print(f"   a = {a} dopo b[0] = 99 con b = a      (stesso oggetto: id(a) == id(b) e' {a is b})")
    c = a.copy()                                     # copia (anche a[:] o list(a))
    c[0] = -1
    print(f"   a = {a}, c = {c} dopo c = a.copy(); c[0] = -1")
    m = [[0] * 3] * 2                                # TRAPPOLA: due riferimenti alla stessa riga
    m[0][0] = 7
    print(f"   [[0]*3]*2 dopo m[0][0] = 7: {m}   (le righe sono lo stesso oggetto!)")
    m = [[0] * 3 for _ in range(2)]                  # corretto: una riga nuova per ogni iterazione
```

```
    m[0][0] = 7
    print(f"   [[0]*3 for _ in range(2)] dopo m[0][0] = 7: {m}")

    # ------------------------------------------------------------------ 3) tuple e stringhe: la sigla NACA
    print("\n3) Tuple, stringhe e profili NACA a quattro cifre")
    punto = (0.25, 0.05)                                # tupla: immutabile, si spacchetta
    x, y = punto
    print(f"   punto = {punto}, x = {x}, y = {y}; (3,) e' una tupla di un elemento, (3) e' il numero 3")


    for sigla in ("0012", "2412", "4415"):
        m, p, t = parametri_naca(sigla)
        print(f"   NACA {sigla}: curvatura {m:.0%} in x = {p:.0%} c, spessore {t:.0%} c")
    try:
        parametri_naca("24a2")
    except ValueError as errore:
        print(f"   errore intercettato: {errore}")

    # ------------------------------------------------------------------ 4) list comprehension e funzioni su liste
    print("\n4) Comprehension, zip, ordinamento con chiave")
    dorso, ventre = profilo("0012", n=400)
    ys = [y for x, y in dorso]                          # list comprehension con spacchettamento
    k = ys.index(max(ys))
    print(f"   NACA 0012: spessore massimo {2 * max(ys):.4f} c in x = {dorso[k][0]:.3f} c (tabulato: 0,12 c in 0,30 c)")
    area = 0.0
    for (x1, y1), (x2, y2) in zip(dorso[:-1], dorso[1:]):          # regola dei trapezi sul dorso, x2 per simmetria
        area += (y1 + y2) * (x2 - x1)
    print(f"   area della sezione = {area:.4f} c^2 (valore noto: 0,0822 c^2 con il bordo d'uscita aperto, coefficiente -0,1015;")
    print("   con il bordo chiuso, -0,1036, l'area e' 0,0817 c^2)")
    assert abs(area - 0.0817) < 2e-4
    pari = [x for x in range(20) if x % 2 == 0]
    quadrati = {x: x**2 for x in range(5)}              # dict comprehension
    print(f"   pari < 20: {pari}; quadrati: {quadrati}")
    profili = {"0012": 0.12, "2412": 0.12, "4415": 0.15, "0009": 0.09, "6409": 0.09}
    per_spessore = sorted(profili, key=lambda s: (profili[s], s))  # chiave di ordinamento: (spessore, sigla)
    print(f"   sigle ordinate per spessore: {per_spessore}")

    # ------------------------------------------------------------------ 5) dizionari e insiemi
    print("\n5) Dizionari e insiemi")
    fluidi = {"aria": {"rho": 1.225, "nu": 1.46e-5}, "acqua": {"rho": 998.0, "nu": 1.0e-6}}
    fluidi["olio"] = {"rho": 870.0, "nu": 1.0e-4}
    for nome, prop in fluidi.items():
        print(f"   {nome:6s}: rho = {prop['rho']:7.1f} kg/m^3, nu = {prop['nu']:.2e} m^2/s, mu = {prop['rho'] * prop['nu']:.2e} Pa s")
    print(f"   chiavi: {list(fluidi)}; 'elio' presente? {'elio' in fluidi}; valore con default: {fluidi.get('elio', 'assente')}")
    spessori = {t for t in profili.values()}            # insieme: senza duplicati, senza ordine
```

```
127     print(f"   spessori distinti: {sorted(spessori)}; intersezione {{9, 12, 15}} &
        {{12, 18}} = {set([9, 12, 15]) & set([12, 18])}")
128     parole = "la portanza nasce dalla differenza di pressione fra ventre e
        dorso".split()
129     print(f"   {len(parole)} parole, la piu' lunga {max(parole, key=len)!r}, unite
        con '-': {'-'.join(parole[:4])!r}")
130
131     # ------------------------------------------------------------------ figura
132     fig, ax = plt.subplots(figsize=(6.0, 2.6))
133     for sigla, col in zip(("0012", "2412", "4415"), (BLU, ARANCIO, VERDE)):
134         dorso, ventre = profilo(sigla)
135         xs = [x for x, y in dorso] + [x for x, y in ventre[::-1]]
136         ys = [y for x, y in dorso] + [y for x, y in ventre[::-1]]
137         ax.plot(xs, ys, color=col, label=f"NACA {sigla}")
138     ax.set_aspect("equal"); ax.set_xlabel("$x/c$"); ax.set_ylabel("$y/c$");
        ax.legend(fontsize=7)
139     fig.tight_layout()
140     salva(fig, "prog0_3_naca")
```

Listato 3.1: Programma 0.3 – le collezioni di Python sull'esempio dei profili NACA a quattro cifre (`prog0_3_strutture_dati.py`).

**Risultati del Programma 0.3**

- Le operazioni sulle liste: slicing, rovesciamento, `in`, `index`, `remove`, `pop`, ordinamento. `b = a` è un alias (`id(a) == id(b)`); `[[0]*3]*2` ha righe identiche, la comprehension no.
- Sigle decodificate correttamente; `'24a2'` intercettata con `ValueError`.
- NACA 0012: spessore massimo 0,1200 $c$ in $x = 0{,}298\,c$ (tabulato 0,30 $c$; la differenza è la risoluzione dei 100 punti); area 0,0817 $c^2$, che è il valore con il bordo d'uscita chiuso, come il programma fa.
- Ordinamento per spessore con `key`: `['0009', '6409', '0012', '2412', '4415']`; valori distinti con `set`; dizionario dei fluidi con `get` e `items`.

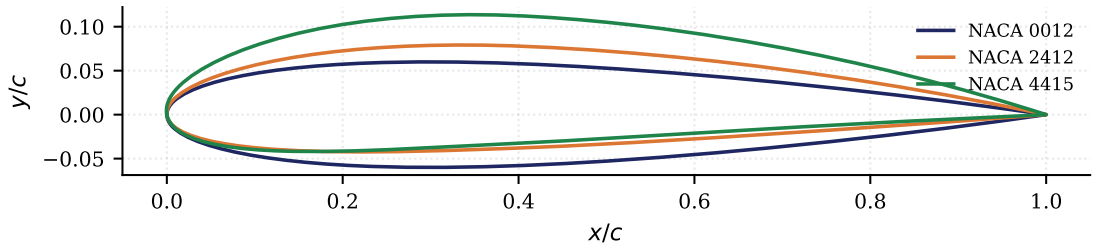



Figura 3.1: Programma 0.3: i profili NACA costruiti dal programma (a sinistra) e la distribuzione di spessore del NACA 0012 con il massimo indicato (a destra).

**Liste o array?**

I programmi del corso usano le liste per *accumulare* risultati di lunghezza non nota in anticipo (`T.append(t)` nella caduta della sfera, `storia.append((t, res))` nella cavità, `profili.append((t, u.copy()))` nel problema di Rayleigh) e per *elencare* casi da esaminare (`casi = [(1, 64, 5.0), (100, 64, 30.0), ...]`); alla fine convertono in array con `np.array(T)` o `map(np.array, (T, Z, V))` per fare arit-

metica e grafici. I campi di moto, le griglie e tutto ciò su cui si fa aritmetica nascono invece direttamente come array: una lista di float sommata elemento per elemento richiede un ciclo Python ed è cento volte più lenta.

## 3.6 Approfondimenti

### 3.6.1 Mutabile e immutabile

La distinzione fra oggetti mutabili (liste, dizionari, insiemi, array) e immutabili (numeri, stringhe, tuple) spiega i comportamenti che sembrano strani. `x = 3; y = x; y += 1` lascia `x` a 3 perché `y += 1` *crea* un nuovo intero e lo assegna a `y`; `a = [3]; b = a; b += [4]` modifica *anche* `a` perché += su una lista la estende sul posto. Per lo stesso motivo una funzione che riceve una lista o un array può modificarli (`passo_ftcs(u, R)` nel Programma 0.8 aggiorna `u` sul posto, e il chiamante lo vede), mentre una che riceve un numero non può. La regola pratica dei programmi del corso: le funzioni che modificano un argomento sul posto lo dichiarano nella docstring; le altre restituiscono un nuovo oggetto.

### 3.6.2 Complessità delle operazioni

L'accesso `lista[i]` e `dizionario[chiave]` costa un tempo costante; `x in lista` scorre tutta la lista (costo proporzionale alla lunghezza), `x in insieme` e `chiave in dizionario` sono in tempo costante grazie alla tabella hash; `append` è costante, `insert(0, x)` sposta tutti gli elementi; `sorted` costa $n \log n$. Per un programma di fluidodinamica queste differenze contano poco rispetto a quella fra cicli Python e operazioni vettoriali di NumPy (fattore 40–100, Cap. 4), ma spiegano una scelta ricorrente: le tabelle di confronto (i dati di Ghia et al., le polari) sono dizionari con chiave il numero di Reynolds o l'angolo, non liste da scorrere.

### 3.6.3 Spacchettamento e assegnazioni multiple

`a, b = b, a` scambia due variabili senza appoggio; `T, Z, V = map(np.array, (T, Z, V))` converte tre liste in una riga; `for i, (v, s) in enumerate(zip(...))` spacchetta tuple annidate; `primo, *resto = lista` raccoglie il resto in una lista. Queste forme sono comuni nei programmi del corso e, una volta riconosciute, rendono il codice più breve da leggere, non più difficile.

# Capitolo 4

# NumPy: gli array e le differenze finite

## 4.1 Perché gli array

Un campo di moto discreto è una tabella di numeri: $u_i$ su una griglia monodimensionale, $\psi_{i,j}$ su una bidimensionale. NumPy fornisce l'oggetto `ndarray`, un blocco di memoria contigua di numeri dello stesso tipo con una *forma* (`shape`), e le operazioni aritmetiche agiscono su tutto l'array in una volta, in codice compilato: `u + v`, `np.sin(x)`, `u[2:] - u[:-2]`. È ciò che permette di scrivere uno schema alle differenze finite in una riga e di eseguirlo quaranta volte più velocemente dei cicli equivalenti. Ogni programma delle monografie inizia con `import numpy as np`; questo capitolo spiega le poche idee (forma, tipo, slicing come vista, broadcasting) da cui discende tutto il resto.

**Creare un array**

```
a = np.array([1.0, 2.0, 3.0])          # da una lista; a.shape = (3,),
    a.dtype = float64
M = np.array([[1, 2, 3], [4, 5, 6]])   # da una lista di liste: forma (2,
    3), dtype int64
x = np.linspace(0.0, 1.0, n + 1)       # n+1 nodi equispaziati, estremi
    inclusi: la griglia
k = np.arange(0, 10, 2)                # come range: 0, 2, 4, 6, 8
u = np.zeros((N + 1, N + 1))           # campo nullo; np.ones, np.full,
    np.zeros_like(u), np.empty
X, Y = np.meshgrid(x, y, indexing="ij")   # X[i, j] = x_i, Y[i, j] = y_j
```

Il **tipo** (`dtype`) è unico per tutto l'array e viene dedotto dai dati: `np.array([1, 2])` è di interi, e assegnarvi 2,7 *tronca* a 2 senza avviso. I campi di moto nascono sempre come `float`: `np.zeros(n)` lo è, `np.array([0, 0])` no. Gli attributi `shape`, `ndim`, `size`, `dtype` e la trasposta `M.T` descrivono l'array; `reshape`, `ravel` (appiattisce) e `np.unravel_index` passano da forma a forma: `rhs = (div / dt).ravel()` e `p = LU.solve(rhs).reshape(N, N)` nel Programma 4.5 trasformano il campo di pressione in vettore per il sistema lineare e viceversa.

## 4.2 Indici, slicing e viste

Lo slicing degli array ha la stessa sintassi delle liste, `v[a:b:passo]`, con una differenza che è la chiave di tutto il calcolo alle differenze finite:

**Lo slicing è una vista**

`w = v[2:5]` *non copia* i tre elementi: `w` è una **vista** sulla stessa memoria, e `w[:] = -1` modifica `v`. Lo stesso vale per righe, colonne e blocchi di una matrice: `A[1, :]`, `A[:, 2]`, `A[1:-1, 1:-1]`. È ciò che rende gratuito scrivere uno schema: `u[1:-1] = u[1:-1] + R * (u[2:] - 2*u[1:-1] + u[:-2])` crea tre viste sfalsate, fa l'aritmetica e scrive il risultato nei nodi interni senza ciclo e senza memoria aggiuntiva. Quando si vuole *conservare* un livello temporale serve una copia esplicita: `u_vecchio = u.copy()`, `profili.append((t, u.copy()))`. L'assegnazione `u_vecchio = u`, come per le liste, crea solo un secondo nome.

Le tre forme di indicizzazione che i programmi del corso usano:

- **slicing** `u[1:-1]`, `u[2:]`, `u[:-2]`, `A[1:-1, 1:-1]`, `A[:, 0]`: viste su nodi interni, nodi a destra, nodi a sinistra, blocchi, contorni;
- **indicizzazione booleana** `v[v > 6]`, `np.where(cond, a, b)`: selezione per condizione, che sostituisce un `if` dentro un ciclo (lo schema upwind sceglie la differenza all'indietro o in avanti con `np.where(u > 0, ..., ...)`);
- **scorrimento ciclico** `np.roll(v, 1)`: l'elemento $i-1$ con $v_{-1} = v_{n-1}$, cioè una *condizione periodica* senza nodi fittizi (il trasporto della gaussiana nel Programma 0.4; il problema di Burgers periodico).

`np.argmax`, `np.argmin` e `np.unravel_index` trovano *dove* è un massimo (il centro del vortice: `k = np.unravel_index(np.argmin(psi), psi.shape)`).

## 4.3 Broadcasting

**Regola del broadcasting**

Due array di forme diverse si combinano se, confrontando le forme da destra, ogni dimensione è uguale oppure vale 1: la dimensione 1 viene "stirata". Così `r` di forma `(3,)` più `c` di forma `(2, 1)` dà `(2, 3)`; uno scalare si combina con tutto; `x[:, None] + 10 * y[None, :]` costruisce la tabella $f(x_i, y_j)$ senza `meshgrid`; `T / T.max(axis=0)` normalizza ogni colonna per il proprio massimo. Il broadcasting evita cicli e copie: nei programmi del corso compare quando si moltiplica una matrice per un vettore di coefficienti riga per riga (`A * coeff[:, None]`), si costruiscono condizioni iniziali su una griglia (`np.sin(np.pi * X) * np.sin(2 * np.pi * Y)`) o si confrontano soluzioni su griglie diverse.

**La convenzione degli indici nel corso**

Con `np.meshgrid(x, y, indexing="ij")` gli array hanno `X[i, j] = x[i]` e `Y[i, j] = y[j]`: il *primo* indice corre lungo $x$, il secondo lungo $y$, come nella notazione $\psi_{i,j}$ delle monografie. È la convenzione di tutti i programmi del corso (`u[i, j]`, `psi[:, j]` è la riga $y = y_j$). L'altra convenzione, `indexing="xy"` (predefinita in `meshgrid` e richiesta da `streamplot`), scambia i due indici: `X[j, i]`. Il Cap. 5 mostra come passare dall'una all'altra con la trasposta.

## 4.4 Differenze finite scritte con lo slicing

Su una griglia di $n+1$ nodi $x_i = ih$, le tre approssimazioni elementari della derivata si scrivono senza cicli:

$$f_i' \approx \frac{f_{i+1} - f_i}{h} : \quad \texttt{(f[1:] - f[:-1]) / h} \quad \text{(nodi } 0 \dots n-1\text{)}, \tag{4.1}$$

$$f_i' \approx \frac{f_{i+1} - f_{i-1}}{2h} : \quad \texttt{(f[2:] - f[:-2]) / (2*h)} \quad \text{(nodi } 1 \dots n-1\text{)}, \tag{4.2}$$

$$f_i'' \approx \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2} : \quad \texttt{(f[2:] - 2*f[1:-1] + f[:-2]) / h**2.} \tag{4.3}$$

La regola per leggerle: `f[2:]` è "$f$ nel nodo a destra", `f[:-2]` "$f$ nel nodo a sinistra", `f[1:-1]` "$f$ nel nodo", tutte e tre di lunghezza $n-1$, allineate sui nodi interni. Il laplaciano a cinque punti in due dimensioni è la stessa idea con due indici:

```
lap[1:-1, 1:-1] = (u[2:, 1:-1] + u[:-2, 1:-1] + u[1:-1, 2:] + u[1:-1, :-2] - 4 *
    u[1:-1, 1:-1]) / h**2
```

È la riga che compare, con nomi diversi, in ogni programma dei Moduli 2, 3 e 4: l'operatore di Liebmann, la diffusione della vorticità, il laplaciano della quantità di moto nella cavità MAC (`lapu` nel Programma 4.5, lettura guidata del Cap. 8).

## 4.5 Il Programma 0.4

**Dati del Programma 0.4**

Derivate di $f = \sin x$ su $[0, \pi]$ con $n = 10, \dots, 320$ intervalli e confronto con $\cos x$ e $-\sin x$; laplaciano di $u = \sin \pi x \, \sin 2\pi y$ su griglia $201 \times 201$ (esatto $-5\pi^2 u$), con doppio ciclo e con slicing; trasporto upwind $\zeta_t + \zeta_x = 0$ di una gaussiana centrata in $x = 4$ su griglia periodica di 400 nodi, $C = 0{,}5$, fino a $t = 8$; sistema $3 \times 3$ tridiagonale, autovalori, riduzioni.

```python
"""
prog0_4_numpy.py -- Programma 0.4: NumPy, gli array e le differenze finite con lo
    slicing.

1) Creazione di array (array, zeros, ones, linspace, arange, meshgrid), forma, tipo,
   dimensioni; operazioni elemento per elemento e funzioni universali.
2) Indici e slicing: viste e copie (un sottovettore e' una finestra sullo stesso
   array), indicizzazione booleana, np.where, np.roll, argmax/argmin.
3) Broadcasting: regole e usi (griglie 2D, tabelle, normalizzazione).
4) Differenze finite con lo slicing: derivate prima e seconda, errore in funzione
   di h e ordine di accuratezza misurato (1 per la differenza in avanti, 2 per
   quella centrata e per la derivata seconda).
5) Vettorizzazione: laplaciano discreto 2D con doppio ciclo e con lo slicing,
   tempi a confronto; trasporto upwind di una gaussiana (lo schema del Modulo 4).
6) Riduzioni e algebra lineare: somme, norme, prodotto matrice-vettore, soluzione
   di un sistema, autovalori.
"""
import time
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

np.set_printoptions(precision=4, suppress=True, linewidth=110)
```

prog0_4_numpy.py

```
23
24 # ------------------------------------------------------------------ 1) creazione
25 print("1) Creazione di array")
26 a = np.array([1.0, 2.0, 3.0])
27 M = np.array([[1, 2, 3], [4, 5, 6]])
28 print(f"   a = {a}, forma {a.shape}, tipo {a.dtype}, dimensioni {a.ndim};  M
       =\n{M}\n   forma {M.shape}, tipo {M.dtype}, M.T =\n{M.T}")
29 x = np.linspace(0.0, 1.0, 5)                        # 5 punti equispaziati, estremi
       inclusi
30 k = np.arange(0, 10, 2)                             # come range, ma restituisce un
       array
31 print(f"   linspace(0, 1, 5) = {x};  arange(0, 10, 2) = {k};  zeros((2,
       3)):\n{np.zeros((2, 3))}")
32 print(f"   a + 1 = {a + 1},  a * a = {a * a},  a @ a = {a @ a} (prodotto scalare),
       np.exp(a) = {np.exp(a)}")
33 print(f"   attenzione ai tipi: np.array([1, 2]) ha dtype {np.array([1, 2]).dtype};
       np.zeros(3, dtype=int) = {np.zeros(3, dtype=int)}")
34 u = np.zeros(5, dtype=int)
35 u[0] = 2.7
36 print(f"   assegnare 2,7 in un array di interi lo tronca: {u}  -> usare float per i
       campi di moto")
37
38 # ------------------------------------------------------------------ 2) slicing,
       viste e copie
39 print("\n2) Indici, slicing, viste")
40 v = np.arange(10.0)
41 print(f"   v = {v};  v[2:5] = {v[2:5]};  v[-3:] = {v[-3:]};  v[::3] = {v[::3]};
       v[1:-1] = {v[1:-1]}")
42 w = v[2:5]                                           # VISTA: condivide la memoria con v
43 w[:] = -1
44 print(f"   dopo w = v[2:5]; w[:] = -1  ->  v = {v}   (lo slicing e' una vista, non
       una copia)")
45 w = v[2:5].copy(); w[:] = 7
46 print(f"   con w = v[2:5].copy(); w[:] = 7  ->  v = {v}   (v non cambia)")
47 v = np.arange(10.0)
48 maschera = v > 6
49 print(f"   v > 6 = {maschera};  v[v > 6] = {v[v > 6]};  np.where(v > 6, 1, 0) =
       {np.where(v > 6, 1, 0)}")
50 print(f"   np.roll(v, 1) = {np.roll(v, 1)}  (scorrimento ciclico: utile per le
       condizioni periodiche)")
51 print(f"   argmax di sin(v) = {np.argmax(np.sin(v))} (indice), valore
       {np.max(np.sin(v)):.4f}")
52 A = np.arange(12.0).reshape(3, 4)
53 print(f"   A =\n{A}\n   riga 1: {A[1]};  colonna 2: {A[:, 2]};  blocco A[1:, 1:3]
       =\n{A[1:, 1:3]}\n   A[1:-1, 1:-1] (nodi interni) = {A[1:-1, 1:-1]}")
54
55 # ------------------------------------------------------------------ 3) broadcasting
56 print("\n3) Broadcasting")
57 r = np.array([1.0, 2.0, 3.0])                       # forma (3,)
58 c = np.array([[10.0], [20.0]])                      # forma (2, 1)
59 print(f"   r (3,) + c (2,1) ->\n{r + c}   (forma {(r + c).shape}: ogni dimensione 1
       si 'stira')")
60 xg = np.linspace(0, 1, 4)[:, None]                  # (4, 1)
61 yg = np.linspace(0, 1, 3)[None, :]                  # (1, 3)
62 print(f"   tabella f(x, y) = x + 10 y senza meshgrid:\n{xg + 10 * yg}")
63 X, Y = np.meshgrid(np.linspace(0, 1, 4), np.linspace(0, 1, 3), indexing="ij")
64 print(f"   con meshgrid(indexing='ij'): X[i, j] = x_i, Y[i, j] = y_j; X + 10 Y
       =\n{X + 10 * Y}")
65 T = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
66 print(f"   normalizzare le colonne: T / T.max(axis=0) =\n{T / T.max(axis=0)}
```

```
    (media per riga: {T.mean(axis=1)})")

# ------------------------------------------------------------------ 4) differenze finite
print("\n4) Differenze finite con lo slicing: f = sin(x) su [0, pi], errore massimo nei nodi interni")
print(f"   {'h':>8} {'avanti':>10} {'centrata':>10} {'seconda':>10}")
hs, e1, e2, e3 = [], [], [], []
for n in (10, 20, 40, 80, 160, 320):
    x = np.linspace(0, np.pi, n + 1); h = x[1] - x[0]
    f = np.sin(x)
    d_avanti = (f[1:] - f[:-1]) / h                       # nei nodi 0..n-1
    d_centr = (f[2:] - f[:-2]) / (2 * h)                  # nei nodi 1..n-1
    d2 = (f[2:] - 2 * f[1:-1] + f[:-2]) / h**2
    ea = np.max(np.abs(d_avanti[:-1] - np.cos(x[:-2])))   # stessi nodi interni (0..n-2 -> confrontati in x[:-2])
    ec = np.max(np.abs(d_centr - np.cos(x[1:-1])))
    es = np.max(np.abs(d2 + np.sin(x[1:-1])))
    hs.append(h); e1.append(ea); e2.append(ec); e3.append(es)
    print(f"   {h:8.4f} {ea:10.2e} {ec:10.2e} {es:10.2e}")
ordini = [np.polyfit(np.log(hs), np.log(e), 1)[0] for e in (e1, e2, e3)]
print(f"   ordini misurati (pendenza nel piano log-log): {ordini[0]:.2f}, {ordini[1]:.2f}, {ordini[2]:.2f}")

# ------------------------------------------------------------------ 5) vettorizzazione
print("\n5) Vettorizzazione: laplaciano discreto su griglia 201 x 201")
N = 200
h = 1.0 / N
xx = np.linspace(0, 1, N + 1)
X, Y = np.meshgrid(xx, xx, indexing="ij")
u = np.sin(np.pi * X) * np.sin(2 * np.pi * Y)
esatto = -5 * np.pi**2 * u
t0 = time.perf_counter()
lap1 = np.zeros_like(u)
for i in range(1, N):
    for j in range(1, N):
        lap1[i, j] = (u[i + 1, j] + u[i - 1, j] + u[i, j + 1] + u[i, j - 1] - 4 * u[i, j]) / h**2
t_cicli = time.perf_counter() - t0
t0 = time.perf_counter()
lap2 = np.zeros_like(u)
lap2[1:-1, 1:-1] = (u[2:, 1:-1] + u[:-2, 1:-1] + u[1:-1, 2:] + u[1:-1, :-2] - 4 * u[1:-1, 1:-1]) / h**2
t_slicing = time.perf_counter() - t0
print(f"   doppio ciclo for: {t_cicli*1000:7.1f} ms;  slicing: {t_slicing*1000:6.2f} ms;  rapporto {t_cicli / t_slicing:.0f}")
print(f"   stessi risultati? {np.allclose(lap1, lap2)};  errore rispetto al laplaciano esatto {np.max(np.abs(lap2 - esatto)[1:-1, 1:-1]):.2e} (O(h^2))")

print("\n   Trasporto upwind di una gaussiana (zeta_t + u zeta_x = 0, u = 1, C = 0,5, griglia periodica):")
L, nx = 20.0, 400
x = np.linspace(0, L, nx, endpoint=False); dx = x[1] - x[0]
z = np.exp(-(x - 4) ** 2 / (2 * 0.5**2))
C = 0.5
n_passi = int(8.0 / (C * dx))
for _ in range(n_passi):
    z = (1 - C) * z + C * np.roll(z, 1)          # z[i-1] con np.roll: periodicita' automatica
```

```
print(f"   dopo {n_passi} passi il massimo ({z.max():.3f}) e' in x = {x[np.argmax(z)]:.2f} (atteso 12,0): il profilo e' traslato"
      f" e attenuato dalla viscosita' artificiale")

# ------------------------------------------------------------------ 6) riduzioni e algebra lineare
print("\n6) Riduzioni e algebra lineare")
A = np.array([[4.0, -1.0, 0.0], [-1.0, 4.0, -1.0], [0.0, -1.0, 4.0]])
b = np.array([1.0, 2.0, 3.0])
sol = np.linalg.solve(A, b)
print(f"   A x = b: x = {sol}, residuo {np.linalg.norm(A @ sol - b):.1e}, cond(A) = {np.linalg.cond(A):.2f}")
lam, V = np.linalg.eigh(A)
print(f"   autovalori di A: {lam} (esatti 4 - 2cos(k pi/4): {4 - 2*np.cos(np.arange(1, 4)*np.pi/4)})")
print(f"   somma {u.sum():.3f}, media {u.mean():.3e}, massimo {u.max():.3f} nel nodo {tuple(int(i) for i in np.unravel_index(np.argmax(u), u.shape))},"
      f" norma {np.linalg.norm(u):.3f}, trapezi {np.trapezoid(np.sin(xx * np.pi), xx):.6f} (esatto 2/pi = {2/np.pi:.6f})")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
ax[0].loglog(hs, e1, "o-", color=BLU, label="in avanti (ordine 1)")
ax[0].loglog(hs, e2, "s-", color=ARANCIO, label="centrata (ordine 2)")
ax[0].loglog(hs, e3, "^-", color=VERDE, label="derivata seconda (ordine 2)")
ax[0].set_xlabel("$h$"); ax[0].set_ylabel("errore massimo"); ax[0].legend(fontsize=7)
ax[1].plot(x, np.exp(-(x - 4) ** 2 / (2 * 0.5**2)), color=GRIGIO, label="$t=0$")
ax[1].plot(x, np.exp(-(x - 12) ** 2 / (2 * 0.5**2)), ":", color=GRIGIO, label="esatta, $t=8$")
ax[1].plot(x, z, color=BLU, label="upwind, $C=0{,}5$")
ax[1].set_xlabel("$x$"); ax[1].set_ylabel("$\\zeta$"); ax[1].legend(fontsize=7)
fig.tight_layout()
salva(fig, "prog0_4_numpy")
```

Listato 4.1: Programma 0.4 – array, slicing, broadcasting, differenze finite, vettorializzazione e algebra lineare (`prog0_4_numpy.py`).

**Risultati del Programma 0.4**

- Viste: dopo `w = v[2:5]; w[:] = -1` l'array `v` è cambiato; con `.copy()` no. `np.roll(v, 1)` porta l'ultimo elemento in testa.
- Differenze finite: gli errori massimi scalano come $h$, $h^2$, $h^2$; pendenze misurate nel piano log-log 1,00, 1,99, 2,00 (Fig. 4.1, a sinistra).
- Laplaciano $201 \times 201$: doppio ciclo $\sim 50$ ms, slicing $\sim 1$ ms, rapporto fra 40 e 50 (dipende dal calcolatore); risultati identici (`np.allclose`); errore rispetto al laplaciano esatto $3{,}5 \times 10^{-3} = \mathcal{O}(h^2)$.
- Upwind con `np.roll`: dopo 320 passi il massimo (0,745, attenuato dalla viscosità artificiale dello schema) è in $x = 12{,}00$, come atteso $(4 + 8)$: la condizione periodica non ha richiesto alcun trattamento del contorno.
- Algebra lineare: `np.linalg.solve` con residuo nullo e $\mathrm{cond}(A) = 2{,}09$; autovalori di `eigh` coincidenti con $4 - 2\cos(k\pi/4)$; `np.trapezoid` di $\sin \pi x$ dà 0,636607 contro $2/\pi = 0{,}636620$.

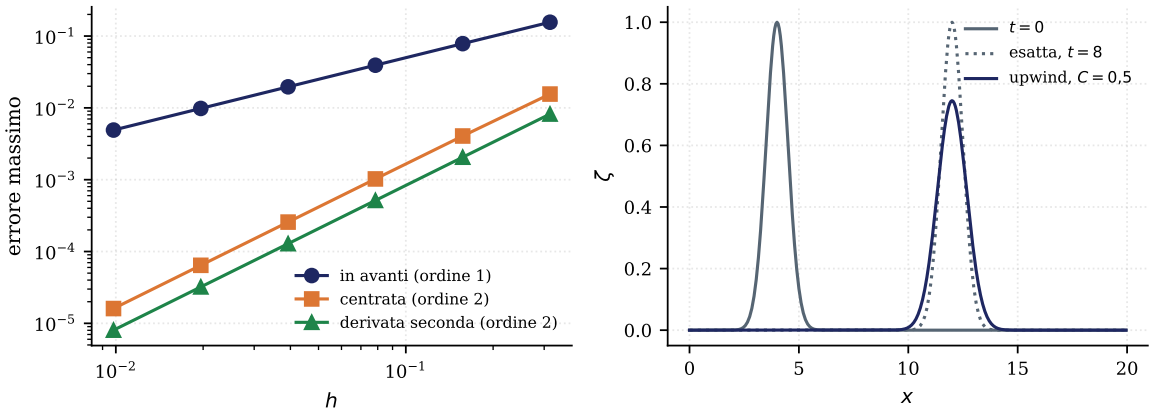



Figura 4.1: Programma 0.4: errori delle tre formule alle differenze in funzione di $h$ con le rette di pendenza 1 e 2 (a sinistra); trasporto upwind della gaussiana su griglia periodica, profilo iniziale e finale (a destra).

**Tre errori con gli array**

(1) `N/2` è un float e non è un indice valido: il nodo centrale è `u[N//2]`. (2) Un array creato di interi tronca i float che vi si assegnano. (3) Gli operatori di NumPy sono *elemento per elemento*: `A * B` non è il prodotto matriciale, che si scrive `A @ B`; `v**2` eleva ogni componente; i confronti `u > 0` danno array booleani, e in un `if` vanno ridotti con `.any()` o `.all()` (`if np.isfinite(v).all():`).

## 4.6 Approfondimenti

### 4.6.1 Perché la vettorializzazione è veloce

Un ciclo `for` in Python esegue, per ogni iterazione, decine di operazioni interpretate: cercare i nomi, controllare i tipi, creare oggetti per i risultati intermedi. L'espressione `u[2:] - 2*u[1:-1] + u[:-2]` fa invece un numero fisso di chiamate a funzioni compilate in C che scorrono la memoria contigua con istruzioni vettoriali del processore; il costo per elemento scende da microsecondi a nanosecondi. Il rapporto misurato (40–50 su $4 \times 10^4$ nodi) cresce con la dimensione della griglia. Il prezzo è la memoria: ogni sotto-espressione crea un array temporaneo (tre, nel laplaciano), il che per griglie $1000 \times 1000$ in doppia precisione sono 8 MB ciascuno; raramente è un problema nei casi del corso. Quando un calcolo *non* si vettorializza (una ricorrenza in cui $u_i$ dipende da $u_{i-1}$ appena calcolato, come l'algoritmo di Thomas o Gauss–Seidel) il ciclo è inevitabile, e la riga *(g)* del Programma 0.9 mostra che vettorializzare una ricorrenza cambia l'algoritmo (da Gauss–Seidel a Jacobi), non solo la velocità.

### 4.6.2 Memoria e ordine degli indici

Un array bidimensionale è conservato riga per riga (ordine "C"): `u[i, j]` e `u[i, j+1]` sono contigui, `u[i+1, j]` dista una riga intera. Scorrere lungo l'ultimo indice è più veloce; con la convenzione `ij` del corso l'ultimo indice è $y$, e le operazioni di slicing sono comunque compilate, quindi la differenza è trascurabile per le griglie del corso. Conta invece per la trasposta: `u.T` è una vista con i passi scambiati, gratuita, e `np.ascontiguousarray` ne fa una copia contigua quando una libreria la richiede.

### 4.6.3 Indicizzazione avanzata e funzioni universali

Un array di interi come indice, `v[[0, 3, 5]]`, e un array booleano, `v[v > 0]`, producono *copie*, non viste; si può assegnare attraverso di essi (`u[maschera] = 0` azzera i nodi interni al cilindro, Programma 0.5). Le *funzioni universali* (`np.sin`, `np.exp`, `np.maximum`, `np.abs`, ...) agiscono elemento per elemento e accettano l'argomento `out=` per scrivere sul posto; le *riduzioni* (`sum`, `max`, `mean`, `np.linalg.norm`) accettano `axis=` per ridurre lungo una sola direzione. Le funzioni del modulo `math` non accettano array: `math.sin(x)` con `x` array dà `TypeError`, e i programmi usano `np.sin`. Per i numeri casuali (griglie perturbate, test) si usa `rng = np.random.default_rng(seme)` e `rng.normal(size=n)`: il seme rende riproducibile il risultato.

# Capitolo 5

# Le figure con Matplotlib

## 5.1 Figura, assi, grafico

Un risultato numerico si capisce guardandolo: il profilo di velocità, le linee di corrente, l'errore in funzione di $h$. Matplotlib è la libreria con cui tutte le figure delle monografie sono state prodotte, e la sua grammatica si riduce a tre oggetti: la **figura** (`fig`, il foglio), gli **assi** (`ax`, un sistema di assi cartesiani sul foglio; una figura può averne molti) e gli **artisti** che si disegnano sugli assi (linee, marcatori, contorni, testi).

**Lo schema di ogni figura del corso**

```
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, GRIGIO

fig, ax = plt.subplots(figsize=(5.0, 3.0))          # un foglio 5x3 pollici
    con un sistema di assi
ax.plot(x, u, "-", color=BLU, lw=1.4, label="numerica")
ax.plot(x, u_es, "o", color=ARANCIO, ms=3, mfc="none", label="esatta")
ax.axhline(0, color=GRIGIO, lw=0.6)                  # linea di riferimento
ax.set_xlabel("$x/c$"); ax.set_ylabel("$C_p$")       # etichette, anche in
    LaTeX
ax.set_xlim(0, 1); ax.legend(fontsize=8)
fig.tight_layout()                                    # sistema i margini
salva(fig, "nome_figura")                             # PDF nella cartella
    figure/
```

`plt.subplots(righe, colonne)` restituisce la figura e un array di assi: `fig, ax = plt.subplots(1, 3)` dà `ax[0]`, `ax[1]`, `ax[2]`; con due righe, `axs[1, 0]` o `axs.ravel()` per scorrerli tutti. La stringa di formato `"o-"` combina marcatore e linea; gli argomenti `color`, `lw` (spessore), `ls` (tratteggio: `"-"`, `":"`), `ms` (dimensione del marcatore), `mfc="none"` (marcatore vuoto), `label` (voce della legenda), `alpha` (trasparenza) sono quelli che i programmi del corso usano. Le etichette accettano formule fra `$...$`: `"$psi/(Ua)$"` (il backslash va raddoppiato nelle stringhe normali, oppure si usa una stringa grezza `r"$psi$"`).

## 5.2 Scale, legende, annotazioni

- `ax.loglog(h, err)` per l'ordine di convergenza: una retta di pendenza $p$ nel piano $\log h$–$\log e$ è l'errore $\mathcal{O}(h^p)$, e la si disegna come riferimento con `err[0]*(h/h[0])**p`; `ax.semilogy(it, res)` per i residui delle iterazioni (una retta è una convergenza geometrica); `ax.invert_xaxis()` quando $h$ decresce.
- `ax.set_xlim`, `set_ylim`, `set_xticks`, `set_aspect("equal")` (indispensabile per i campi di moto: un cerchio deve sembrare un cerchio), `set_title`, `fig.suptitle`.
- `ax.legend(fontsize=8, loc="lower right")` raccoglie le `label`; `ax.annotate` (con `xy`, `xytext` e `arrowprops`) mette una freccia da un testo a un punto; `ax.text`, `ax.axvline`, `ax.axhline`, `ax.grid`.
- `sharex=True` in `subplots` condivide l'asse $x$ fra i pannelli (una sola scala e una sola etichetta in basso), `sharey` analogo.

## 5.3 Campi bidimensionali

**contour, contourf, quiver, streamplot**

Dati `X, Y` da `meshgrid` e un campo `psi` della stessa forma:

```
ax.contour(X, Y, psi, levels=np.linspace(-3, 3, 25), colors=BLU,
    linewidths=0.6, linestyles="solid")
ax.contour(X, Y, psi, levels=[0.0], colors=ROSSO)                    # la
    linea psi = 0 in evidenza
cf = ax.contourf(X, Y, cp, levels=np.linspace(-3, 1, 17), cmap="RdBu_r")   #
    riempito, con mappa divergente
fig.colorbar(cf, ax=ax, label="$C_p$")                               # barra
    dei colori
ax.quiver(X[::12, ::12], Y[::12, ::12], u[::12, ::12], v[::12, ::12],
    scale=25)   # frecce, una ogni 12 nodi
ax.streamplot(x, y, uT, vT, color=vel, cmap="viridis", density=1.2)  # linee
    di corrente integrate
ax.add_patch(plt.Circle((0, 0), a, color=GRIGIO))                    # un
    corpo solido
```

`contour` traccia le isolinee (le linee di corrente sono le isolinee di $\psi$: non serve integrare nulla); `contourf` riempie le fasce fra i livelli; `quiver` disegna i vettori, da *diradare* con lo slicing `[::passo, ::passo]` perché una freccia per nodo è illeggibile; `streamplot` integra le linee di corrente dal campo di velocità (utile quando $\psi$ non c'è, come nelle variabili primitive del Modulo 4) e richiede `x`, `y` monodimensionali e `u`, `v` con la convenzione `xy`: se il campo è nella convenzione `ij` del corso si passa `u.T`, `v.T`. Per non disegnare nulla dentro un corpo si pongono i valori a `np.nan` con `np.where(dentro, np.nan, psi)`: Matplotlib salta i `nan`. Per i livelli negativi `contour` usa il tratteggio per impostazione predefinita; `linestyles="solid"` lo disattiva.

## 5.4 Lo stile della casa: `stile_grafici.py`

Tutte le monografie importano `stile_grafici`, che fa tre cose: fissa i parametri globali di Matplotlib (`plt.rcParams`: carattere serif a 9 pt, griglia punteggiata leggera, niente cornice sopra e a destra, ciclo di colori della palette), definisce i colori con nome (`BLU`,

`ARANCIO`, `VERDE`, `ROSSO`, `VIOLA`, `GIALLO`, `GRIGIO`, e la lista `PALETTE` per i cicli `for k, ... in enumerate(...): color=PALETTE[k]`) coerenti con i colori del documento LaTeX, e fornisce `salva(fig, nome)`, che scrive `figure/nome.pdf` nella cartella del progetto, toglie le etichette secondarie dagli assi logaritmici e chiude la figura per liberare memoria. Il PDF vettoriale si include nel documento senza perdita di qualità; `matplotlib.use("Agg")` all'inizio del modulo seleziona un *backend* senza finestre, perché i programmi girano da terminale e salvano su file. Chi lavora in un notebook può omettere quella riga e usare `plt.show()`.

## 5.5 Il Programma 0.5: la corrente intorno al cilindro

Il programma è un catalogo dei grafici delle monografie, costruito sulla corrente potenziale intorno a un cilindro con circolazione (Modulo 2):

$$\psi = U\Big(r - \frac{a^2}{r}\Big)\sin\theta - \frac{\Gamma}{2\pi}\ln\frac{r}{a}, \qquad V_\theta\big|_{r=a} = 2U\sin\theta + \frac{\Gamma}{2\pi a}, \qquad C_p = 1 - \Big(\frac{V}{U}\Big)^2. \tag{5.1}$$

La funzione `campo(X, Y, gamma)` restituisce $u$, $v$ e $\psi$ sulla griglia; il resto del programma è disegno.

**Dati del Programma 0.5**

$U = a = 1$; $\Gamma = 0$, $\pi Ua$, $2\pi Ua$; griglia $201 \times 201$ su $[-3,3] \times [-2{,}2,2{,}2]$ (convenzione `ij` per `contour`, `xy` per `streamplot`); 25 livelli di $\psi$ fra $-3$ e 3; 17 fasce di $C_p$ fra $-3$ e 1; vettori ogni 12 nodi; profili lungo $y/a = 0$, 0,5, 1,5.

prog0_5_matplotlib.py

```python
"""
prog0_5_matplotlib.py -- Programma 0.5: le figure con Matplotlib.

Catalogo dei grafici usati nelle monografie, costruiti sulla corrente potenziale
intorno a un cilindro con circolazione (Modulo 2):
    psi = U (r - a^2/r) sin(theta) - Gamma/(2 pi) ln(r/a),   C_p = 1 - (V/U)^2.
1) figura e assi (plt.subplots), plot con stili, marcatori, etichette in LaTeX,
   legenda, griglia, limiti; 2) scale logaritmiche (loglog/semilogy) e rette di
   pendenza; 3) piu' pannelli e condivisione degli assi; 4) campi 2D: contour,
   contourf con barra dei colori, streamplot, quiver, aspect; 5) annotazioni,
   linee di riferimento, testo; 6) salvataggio in PDF con la funzione salva() del
   modulo stile_grafici (e che cosa fa quel modulo).
Verifica: il C_p minimo sul cilindro senza circolazione e' -3; con circolazione
Gamma = 2 pi U a i punti di ristagno sono in theta = -pi/6 e -5pi/6 (sin theta =
    -1/2).
"""
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO, PALETTE

U, A = 1.0, 1.0


def campo(X, Y, gamma=0.0):
    """Componenti u, v e funzione di corrente della corrente intorno al cilindro
    con circolazione gamma."""
    r2 = np.maximum(X**2 + Y**2, 1e-12)          # evita la divisione per zero
    nell'origine (dentro il cilindro)
    u = U * (1 - A**2 * (X**2 - Y**2) / r2**2) + gamma / (2 * np.pi) * Y / r2
```

```
    v = -U * 2 * A**2 * X * Y / r2**2 - gamma / (2 * np.pi) * X / r2
    psi = U * Y * (1 - A**2 / r2) - gamma / (2 * np.pi) * np.log(np.sqrt(r2) / A)
    return u, v, psi


# ---------------------------------------------------------------- 1) grafico di base: C_p sul cilindro
theta = np.linspace(0, 2 * np.pi, 361)
fig, ax = plt.subplots(figsize=(5.0, 3.0))                # una figura con un solo sistema di assi
for gamma, col, stile in ((0.0, BLU, "-"), (np.pi, ARANCIO, "--"), (2 * np.pi, VERDE, ":")):
    Vt = 2 * U * np.sin(theta) + gamma / (2 * np.pi * A)   # velocita' tangenziale sulla superficie
    cp = 1 - (Vt / U)**2
    ax.plot(np.degrees(theta), cp, stile, color=col, lw=1.4, label=f"$\\Gamma = {gamma / np.pi:g}\\pi Ua$".replace("0\\pi", "0"))
ax.axhline(1, color=GRIGIO, lw=0.6)
ax.set_xlabel("$\\theta$ [gradi]"); ax.set_ylabel("$C_p$")
ax.set_xlim(0, 360); ax.set_xticks(range(0, 361, 90))
ax.legend(fontsize=8, loc="lower right")
ax.set_title("coefficiente di pressione sul cilindro", fontsize=9)
fig.tight_layout()
salva(fig, "prog0_5_cp")
print(f"C_p minimo senza circolazione: {np.min(1 - (2 * np.sin(theta))**2):.1f} (esatto -3)")
Vt = 2 * U * np.sin(theta) + 1.0
print(f"punti di ristagno per Gamma = 2 pi U a: theta = {np.degrees(theta[np.argsort(np.abs(Vt))[:2]])} gradi (esatti 210 e 330)")

# ---------------------------------------------------------------- 2) scale logaritmiche: ordine di convergenza
h = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625])
err1, err2 = 0.5 * h, 0.3 * h**2
fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
ax[0].loglog(h, err1, "o-", color=BLU, label="ordine 1")
ax[0].loglog(h, err2, "s-", color=ARANCIO, label="ordine 2")
ax[0].loglog(h, err2[0] * (h / h[0])**2, ":", color=GRIGIO, label="pendenza 2")
ax[0].set_xlabel("$h$"); ax[0].set_ylabel("errore"); ax[0].legend(fontsize=7)
ax[0].invert_xaxis()                                      # h decresce verso destra
t = np.linspace(0, 10, 200)
ax[1].semilogy(t, np.exp(-t), color=BLU, label="$e^{-t}$")
ax[1].semilogy(t, np.exp(-t / 3), color=ARANCIO, label="$e^{-t/3}$")
ax[1].set_xlabel("$t$"); ax[1].set_ylabel("residuo"); ax[1].legend(fontsize=7); ax[1].grid(True, which="both", alpha=0.3)
fig.tight_layout()
salva(fig, "prog0_5_log")

# ---------------------------------------------------------------- 3) e 4) campi 2D
n = 201
x = np.linspace(-3, 3, n); y = np.linspace(-2.2, 2.2, n)
X, Y = np.meshgrid(x, y, indexing="ij")
fig, axs = plt.subplots(1, 3, figsize=(9.0, 3.0))
for ax, gamma, titolo in zip(axs, (0.0, 2 * np.pi), ("$\\Gamma=0$", "$\\Gamma=2\\pi Ua$")):
    u, v, psi = campo(X, Y, gamma)
    dentro = X**2 + Y**2 < A**2
    psi = np.where(dentro, np.nan, psi)                   # niente linee dentro il cilindro
```

```
    livelli = np.linspace(-3, 3, 25)
    ax.contour(X, Y, psi, levels=livelli, colors=BLU, linewidths=0.6, linestyles="solid")
    ax.contour(X, Y, psi, levels=[0.0], colors=ROSSO, linewidths=1.0)
    ax.add_patch(plt.Circle((0, 0), A, color=GRIGIO))
    ax.set_aspect("equal"); ax.set_title(titolo, fontsize=9); ax.set_xlabel("$x/a$")
axs[0].set_ylabel("$y/a$")
u, v, psi = campo(X, Y, 0.0)
cp = np.where(X**2 + Y**2 < A**2, np.nan, 1 - (u**2 + v**2) / U**2)
cf = axs[2].contourf(X, Y, cp, levels=np.linspace(-3, 1, 17), cmap="RdBu_r")
fig.colorbar(cf, ax=axs[2], label="$C_p$", shrink=0.9)
passo = 12
axs[2].quiver(X[::passo, ::passo], Y[::passo, ::passo], u[::passo, ::passo], v[::passo, ::passo], color="k", scale=25, width=0.003)
axs[2].add_patch(plt.Circle((0, 0), A, color="w"))
axs[2].set_aspect("equal"); axs[2].set_title("$C_p$ e vettori velocità", fontsize=9); axs[2].set_xlabel("$x/a$")
axs[0].annotate("punto di ristagno", xy=(-1, 0), xytext=(-2.9, 1.6), fontsize=7,
                arrowprops=dict(arrowstyle="->", lw=0.6))
fig.tight_layout()
salva(fig, "prog0_5_campi")

# streamplot (richiede griglia con indexing='xy': assi scambiati rispetto a 'ij')
fig, ax = plt.subplots(figsize=(4.6, 3.2))
Xs, Ys = np.meshgrid(x, y)                                  # indexing='xy': Xs[j, i] = x_i
us, vs, _ = campo(Xs, Ys, np.pi)
dentro = Xs**2 + Ys**2 < A**2
us = np.where(dentro, np.nan, us); vs = np.where(dentro, np.nan, vs)   # niente linee dentro il cilindro
vel = np.hypot(us, vs)
sp = ax.streamplot(x, y, us, vs, color=vel, cmap="viridis", density=1.2, linewidth=0.8, arrowsize=0.7)
fig.colorbar(sp.lines, ax=ax, label="$|V|/U$")
ax.add_patch(plt.Circle((0, 0), A, color=GRIGIO)); ax.set_aspect("equal")
ax.set_xlabel("$x/a$"); ax.set_ylabel("$y/a$"); ax.set_title("streamplot, $\\Gamma=\\pi Ua$", fontsize=9)
fig.tight_layout()
salva(fig, "prog0_5_streamplot")

# ---------------------------------------------------------------------- 5) piu' pannelli con assi condivisi
fig, axs = plt.subplots(2, 2, figsize=(7.0, 4.4), sharex=True)
u, v, psi = campo(X, Y, 0.0)
campi = ((u, "$u/U$"), (v, "$v/U$"), (psi, "$\\psi/(Ua)$"), (1 - (u**2 + v**2), "$C_p$"))
for ax, (f, nome), col in zip(axs.ravel(), campi, PALETTE):
    for off, ls in ((0.0, "-"), (0.5, "--"), (1.5, ":")):
        j = np.argmin(np.abs(y - off))
        ax.plot(x, np.where(np.abs(x) < np.sqrt(max(A**2 - off**2, 0)), np.nan, f[:, j]), ls, color=col, label=f"$y/a={off:g}$")
    ax.set_ylabel(nome); ax.axvline(-1, color=GRIGIO, lw=0.5); ax.axvline(1, color=GRIGIO, lw=0.5)
axs[0, 0].legend(fontsize=6)
for ax in axs[1]:
    ax.set_xlabel("$x/a$")
fig.suptitle("profili lungo rette orizzontali ($\\Gamma=0$)", fontsize=9)
fig.tight_layout()
salva(fig, "prog0_5_pannelli")
print("figure salvate: prog0_5_cp, prog0_5_log, prog0_5_campi, prog0_5_streamplot,
```

```
    prog0_5_pannelli")
```

Listato 5.1: Programma 0.5 – il catalogo dei grafici del corso sulla corrente intorno al cilindro (`prog0_5_matplotlib.py`).

**Risultati del Programma 0.5**

Il $C_p$ minimo senza circolazione è $-3$ (in $\theta = 90°$, dove $V = 2U$); con $\Gamma = 2\pi U a$ i punti di ristagno trovati con `np.argsort(np.abs(Vt))` sono in $\theta = 210°$ e $330°$, cioè $\sin\theta = -1/2$, come prevede la (5.1). Le cinque figure prodotte (Figg. 5.1–5.3) sono i modelli di tutte le figure del corso: grafico di linee con legenda, assi logaritmici con retta di pendenza, isolinee con il corpo in evidenza, campo riempito con barra dei colori e vettori, linee di corrente colorate dal modulo della velocità, pannelli con asse condiviso.

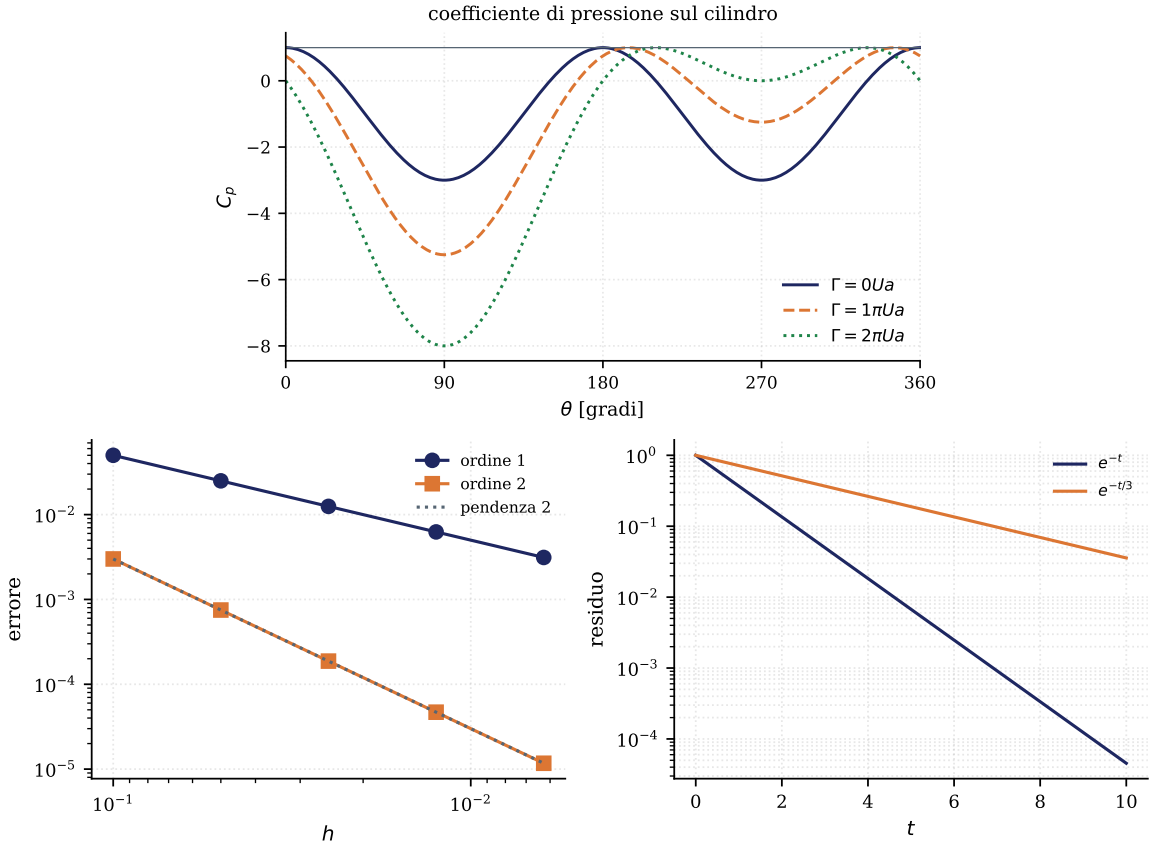



Figura 5.1: Programma 0.5: grafico di base con tre curve, stili e legenda (sopra); assi logaritmici: errori di ordine 1 e 2 con la retta di riferimento, e residui in scala semilogaritmica (sotto).

**Leggere una figura del corso**

Nel Programma 4.5 (cavità) la figura dei risultati è costruita esattamente così: `ax.contour(X, Y, psi, levels=...)` per le linee di corrente nei vertici delle celle, `ax.plot(GHIA_U[Re], GHIA_U["y"], "o")` per i punti di confronto, `ax.semilogy` per la storia del residuo. Riconosciuti i tre oggetti (figura, assi, artisti) e le poche funzioni di questo capitolo, nessuna figura delle monografie contiene altro.

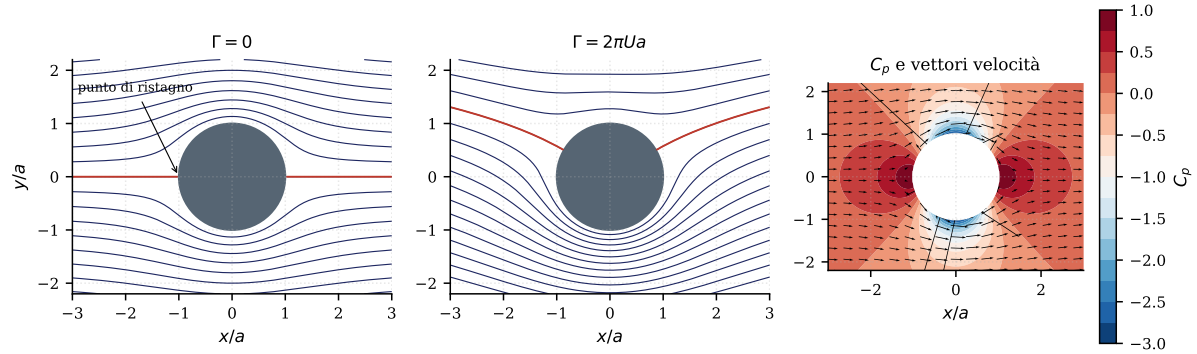



Figura 5.2: Programma 0.5: isolinee di $\psi$ senza e con circolazione (linea $\psi = 0$ in rosso, annotazione con freccia) e campo di $C_p$ riempito con barra dei colori e vettori velocità diradati.

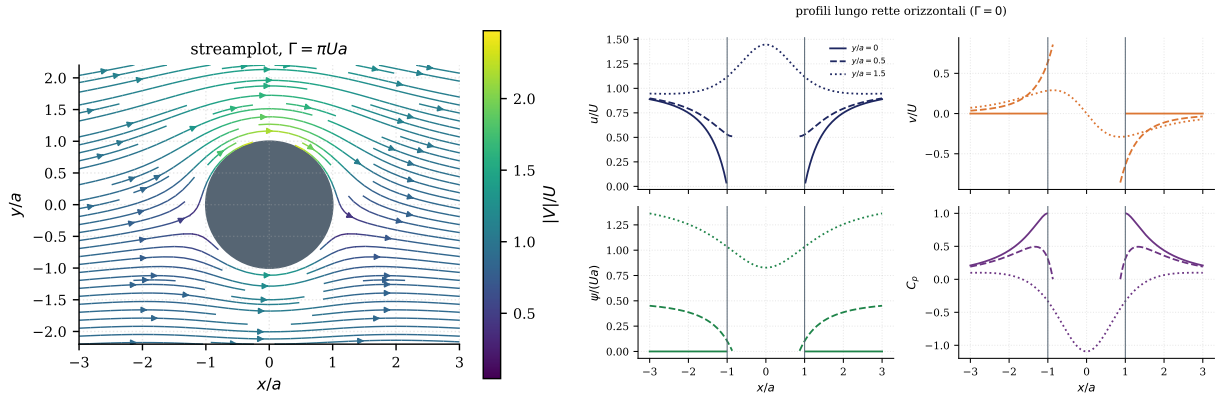



Figura 5.3: Programma 0.5: linee di corrente con `streamplot` colorate dal modulo della velocità (a sinistra); quattro pannelli con asse $x$ condiviso: profili di $u$, $v$, $\psi$ e $C_p$ lungo tre rette orizzontali (a destra).

## 5.6 Approfondimenti

### 5.6.1 Colori e leggibilità

Le mappe di colore *sequenziali* (`viridis`, `magma`) vanno usate per grandezze con un verso (modulo della velocità, temperatura), quelle *divergenti* (`RdBu_r`, `coolwarm`) per grandezze con uno zero significativo (vorticità, $C_p$, perturbazioni), centrando i livelli sullo zero; le mappe "arcobaleno" (`jet`) vanno evitate perché introducono bande che non esistono nei dati e non sono leggibili in bianco e nero. La palette di `stile_grafici` è stata scelta per essere distinguibile anche in stampa monocromatica e dai lettori con deficit della visione dei colori; quando le curve sono più di quattro, si combinano colore e tratteggio o marcatore, come nel Programma 0.5.

### 5.6.2 Formati di uscita e dimensioni

Il PDF è vettoriale: le linee restano nitide a ogni ingrandimento e il file è piccolo, salvo per `contourf` e `streamplot` con molte celle, dove il PNG a 200–300 dpi può essere preferibile (`fig.savefig("nome.png", dpi=250)`). `figsize` è in pollici: le figure del corso sono larghe 5–9 pollici per riempire la colonna di testo (larga circa 6,3 pollici) senza essere ridotte troppo, e i caratteri a 9 pt restano leggibili. `fig.tight_layout()` evita sovrapposizioni fra etichette e pannelli; `bbox_inches="tight"` (impostato in `rcParams`) ritaglia i margini

bianchi.

### 5.6.3 Grafici interattivi e animazioni

Il corso salva figure statiche, ma lo stesso codice funziona in modo interattivo in un notebook (`%matplotlib widget`) e `matplotlib.animation.FuncAnimation` produce animazioni di una marcia nel tempo aggiornando i dati di una linea (`linea.set_ydata(u)`) a ogni fotogramma; è un esercizio utile per vedere l'instabilità di uno schema nascere e crescere.

# Capitolo 6

# La cassetta degli attrezzi di SciPy

## 6.1 Quando usare SciPy

Le monografie scrivono *a mano* gli algoritmi che sono l'oggetto del corso: Runge–Kutta, l'algoritmo di Thomas, SOR, lo schema MAC. È una scelta didattica: si impara un metodo scrivendolo. Ma intorno a quegli algoritmi servono strumenti ausiliari (trovare una radice, risolvere un sistema lineare grande, integrare un problema ai limiti di riferimento, interpolare una tabella), e per questi il corso usa SciPy, la libreria di metodi numerici costruita su NumPy, perché i suoi risolutori sono verificati, robusti e veloci. Questo capitolo presenta i sottomoduli che compaiono nei programmi del corso, ognuno su un problema di fluidodinamica con soluzione nota, in modo che il lettore sappia che cosa aspettarsi da ciascuno e come controllarne il risultato.

**Convenzione**

`from scipy import optimize, integrate, linalg, sparse, interpolate, special` e `import scipy.sparse.linalg as spla`. Ogni funzione di SciPy riceve *funzioni* come argomenti (il secondo membro, la funzione di cui cercare la radice) e restituisce o il risultato direttamente o un oggetto con più campi (`s.t`, `s.y`, `s.sol`, `s.success`, `s.message`): la docstring, `help(integrate.solve_ivp)`, dice quali.

## 6.2 Radici e minimi: `scipy.optimize`

**`brentq`, `fsolve`, `minimize_scalar`**

```
v_t = optimize.brentq(f, 1.0, 100.0, xtol=1e-12)          # radice di f(v) =
    0 in [1, 100]: f cambia segno
sol = optimize.fsolve(F, x0=[-1.0, -0.5])                  # sistema F(x) = 0
    (Newton con jacobiano numerico)
r = optimize.minimize_scalar(lambda a: -gittata(a), bounds=(0, 90),
    method="bounded")
```

`brentq` (metodo di Brent: bisezione più interpolazione) è lo strumento per un'equazione scalare quando si dispone di un intervallo con cambio di segno: converge sempre, al contrario di Newton, con la velocità di Newton. `fsolve` risolve sistemi non lineari a partire da un punto iniziale, e il punto iniziale decide *quale* soluzione si trova e *se* si trova:

va scelto con un ragionamento fisico. `minimize_scalar` con `method="bounded"` minimizza una funzione di una variabile in un intervallo; per massimizzare si cambia segno. I risolutori di Newton scritti a mano nelle monografie (`newtn1`, `newtn2` del Modulo 4) fanno ciò che fa `fsolve`, in forma leggibile.

## 6.3 Integrali ed equazioni differenziali: `scipy.integrate`

**`quad`, `solve_ivp`, `solve_bvp`**

```
Q, err = integrate.quad(lambda r: 2*np.pi*r*u(r), 0, R)        # integrale
    definito, con stima dell'errore
s = integrate.solve_ivp(F, (0, 60), y0, method="RK45", rtol=1e-10,
    atol=1e-12,
                        events=suolo, dense_output=True)     # dy/dt = F(t,
    y), passo adattivo
s.t, s.y[0], s.t_events[0][0], s.sol(2.0)                     # istanti,
    soluzione, evento, interpolazione
sb = integrate.solve_bvp(f_bl, cc, eta, y0, tol=1e-10)        # problema ai
    limiti con collocazione
```

`solve_ivp` è l'integratore generale per $\mathrm{d}\mathbf{y}/\mathrm{d}t = \mathbf{F}(t, \mathbf{y})$: sceglie il passo in base alle tolleranze (`rtol`, `atol`), può fermarsi a un *evento* (una funzione che si annulla: la sfera tocca il suolo), e con `dense_output=True` restituisce una funzione `s.sol(t)` valutabile ovunque. Il metodo predefinito `RK45` è un Runge–Kutta di ordine 4(5) con stima dell'errore; `Radau` e `BDF` sono per i problemi *stiff* (Cap. 9). Il vettore di stato va scritto come lista o array: `y = [z, v]`, e `F` restituisce `[v, f(v)]`. `solve_bvp` risolve un problema ai limiti su un intervallo con condizioni ai due estremi date da una funzione `cc(ya, yb)` che restituisce i residui: è il modo di ottenere il valore di riferimento $f''(0) = 0{,}33205734$ di Blasius, che il Modulo 3 calcola con il metodo del tiro.

## 6.4 Sistemi lineari: `scipy.linalg` e `scipy.sparse`

**Denso, a banda, sparso**

```
x = linalg.solve(A, b)                                  # denso: O(n^3),
    per n fino a qualche migliaio
x = linalg.solve_banded((1, 1), ab, b)                  # tridiagonale:
    O(n), con ab = [sopra; diag; sotto]
lu, piv = linalg.lu_factor(A); x = linalg.lu_solve((lu, piv), b)   #
    fattorizza una volta, risolvi molte volte
lam, V = linalg.eigh(A)                                 #
    autovalori/autovettori di una matrice simmetrica
A = sparse.lil_matrix((n, n)); A[p, q] = ...; A = A.tocsc()        # sparsa:
    riempimento comodo, poi formato compresso
x = spla.spsolve(A, b);  LU = spla.splu(A); x = LU.solve(b)        #
    risolutori sparsi diretti
```

La scelta dipende dalla struttura. Il sistema tridiagonale dei metodi impliciti monodimensionali (Crank–Nicolson, Modulo 3) si risolve con `solve_banded` in tempo lineare, cinquanta volte più in fretta di `solve` già per $n = 1000$; è ciò che fa, a mano, l'algoritmo

di Thomas di `metodi_numerici.py`. Il laplaciano bidimensionale su $N \times N$ nodi è una matrice $N^2 \times N^2$ con cinque elementi non nulli per riga: per $N = 60$ è lo 0,14% della matrice, e il formato sparso la risolve in millisecondi dove la versione densa impiega un secondo e, per $N = 200$, non entrerebbe in memoria ($N^4 \cdot 8$ byte $= 12{,}8$ GB). Quando lo stesso sistema va risolto a ogni passo temporale con secondo membro diverso (l'equazione di Poisson per la pressione nella cavità MAC) si fattorizza *una volta* con `splu` e si chiama `LU.solve(rhs)` a ogni passo: nel Programma 4.5 la fattorizzazione è calcolata in `matrice_pressione` fuori dal ciclo temporale, e nel Programma 0.6 `LU.solve` costa venti volte meno di `spsolve`.

## 6.5 Interpolazione e funzioni speciali

`interpolate.interp1d(alpha, cl)` e `interpolate.CubicSpline(alpha, cl)` costruiscono funzioni interpolanti da una tabella (la polare di un profilo); la spline cubica ha derivata continua e `spl(x, 1)` ne dà la pendenza, utile per $\mathrm{d}C_L/\mathrm{d}\alpha$. `np.interp` è l'equivalente NumPy per l'interpolazione lineare. Il modulo `special` fornisce le funzioni che compaiono nelle soluzioni esatte: `erf`, `erfc` (il problema di Rayleigh, Modulo 3), le funzioni di Bessel `j0`, `j1` (il tubo in moto impulsivo), `ellipk` (il periodo del pendolo, Esercizio 13), `gamma`.

## 6.6 Il Programma 0.6

**Dati del Programma 0.6**

Velocità limite della sfera d'acciaio di 1 cm con la correlazione regolare di Clift–Gauvin; punto di ristagno della corrente intorno al cilindro con $\Gamma = 1{,}6\pi U a$ (esatto $\sin\theta = -0{,}4$); gittata nel vuoto; portata di Poiseuille con $R = 1$ cm, $\mu = 10^{-3}$ Pa s, $\mathrm{d}p/\mathrm{d}x = -100$ Pa/m; caduta della sfera fino a $z = 100$ m; Blasius su $0 \le \eta \le 10$; sistema tridiagonale $n = 1000$; Poisson su griglia $60 \times 60$ con $u = \sin\pi x \sin\pi y$; polare $C_L(\alpha)$ tabulata ogni $2^\circ$.

prog0_6_scipy.py

```
"""
prog0_6_scipy.py -- Programma 0.6: la cassetta degli attrezzi di SciPy.

Le funzioni di SciPy usate nelle monografie, ciascuna su un problema della
fluidodinamica con soluzione nota:
1) scipy.optimize: brentq (radice scalare: velocita' limite), fsolve (sistema non
   lineare: punto di ristagno di una corrente), minimize_scalar (gittata massima).
2) scipy.integrate: quad (integrale: portata di Poiseuille), solve_ivp (problema ai
   valori iniziali con passo adattivo ed evento: caduta della sfera fino al suolo),
   solve_bvp (problema ai limiti: Blasius).
3) scipy.linalg: solve, solve_banded (sistema tridiagonale di Crank-Nicolson), eigh
   (autovalori dell'operatore delle differenze seconde), lu_factor/lu_solve.
4) scipy.sparse e sparse.linalg: matrice del laplaciano 2D in formato lil/csr,
   spsolve e splu (fattorizza una volta, risolve molte volte); confronto dei tempi
   con la matrice piena.
5) scipy.interpolate: interp1d e CubicSpline (polare di un profilo), scipy.special:
   erf, funzioni di Bessel.
"""
import time
import numpy as np
```

```python
from scipy import optimize, integrate, linalg, sparse, interpolate, special
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

# ------------------------------------------------------------------ 1) optimize
print("1) scipy.optimize")
RHO_A, NU, G = 1.225, 1.46e-5, 9.80665
d, m = 0.01, 7800.0 * np.pi * 0.01**3 / 6
cd = lambda Re: 24 / Re * (1 + 0.15 * Re**0.687) + 0.42 / (1 + 42500 * Re**-1.16)
f = lambda v: G - 0.5 * RHO_A * v**2 * np.pi * d**2 / 4 * cd(max(v * d / NU, 1e-9)) / m
vt = optimize.brentq(f, 1.0, 100.0, xtol=1e-12)
print(f"   brentq: velocita' limite v_t = {vt:.6f} m/s (f(v_t) = {f(vt):.1e})")
# punto di ristagno della corrente intorno al cilindro con circolazione: u = v = 0 (sistema 2x2)
gamma = 2 * np.pi * 0.8
def velocita(p):
    x, y = p; r2 = x * x + y * y
    return [1 - (x * x - y * y) / r2**2 + gamma / (2 * np.pi) * y / r2, -2 * x * y / r2**2 - gamma / (2 * np.pi) * x / r2]
sol, info, ier, msg = optimize.fsolve(velocita, [-0.9, -0.4], full_output=True)
print(f"   fsolve: punto di ristagno in ({sol[0]:.6f}, {sol[1]:.6f}); esatto: sin(theta) = -Gamma/(4 pi U a) = -0,4 ->"
      f" ({-np.sqrt(1 - 0.16):.6f}, {-0.4:.6f}); residuo {np.max(np.abs(velocita(sol))):.1e}, {info['nfev']} valutazioni")
gittata = lambda alfa: -(20.0**2 / G) * np.sin(2 * alfa)          # da minimizzare (segno meno)
ris = optimize.minimize_scalar(gittata, bounds=(0, np.pi / 2), method="bounded")
print(f"   minimize_scalar: angolo di gittata massima {np.degrees(ris.x):.4f} gradi (esatto 45), gittata {-ris.fun:.4f} m")

# ------------------------------------------------------------------ 2) integrate
print("\n2) scipy.integrate")
R, mu, dpdx = 0.01, 1e-3, -100.0
portata, err = integrate.quad(lambda r: 2 * np.pi * r * (-dpdx / (4 * mu)) * (R**2 - r**2), 0, R)
print(f"   quad: portata di Poiseuille {portata:.6e} m^3/s (esatta pi R^4 (-dp/dx)/(8 mu) = {np.pi * R**4 * (-dpdx) / (8 * mu):.6e}), errore stimato {err:.1e}")


def caduta(t, y):                                   # y = [z, v], z verso il basso
    z, v = y
    return [v, f(v)]


suolo = lambda t, y: 100.0 - y[0]                   # evento: z = 100 m
suolo.terminal = True
s = integrate.solve_ivp(caduta, (0, 60), [0.0, 0.0], method="RK45", rtol=1e-10, atol=1e-12, events=suolo, dense_output=True)
t_suolo = s.t_events[0][0]
print(f"   solve_ivp: la sfera tocca il suolo (100 m) a t = {t_suolo:.6f} s con v = {s.y_events[0][0][1]:.6f} m/s;"
      f" {len(s.t)} passi adattivi (passo medio {s.t[-1] / len(s.t):.3f} s)")
print(f"   soluzione densa: z(2 s) = {s.sol(2.0)[0]:.6f} m; nel vuoto sarebbe {0.5 * G * 4:.4f} m")


def blasius(eta, y):                                # y = [f, f', f'']
```

```
    return np.vstack([y[1], y[2], -0.5 * y[0] * y[2]])


def cc(ya, yb):
    return np.array([ya[0], ya[1], yb[1] - 1.0])


eta = np.linspace(0, 10, 50)
y0 = np.zeros((3, eta.size)); y0[1] = 1 - np.exp(-eta)
sb = integrate.solve_bvp(blasius, cc, eta, y0, tol=1e-10)
print(f"   solve_bvp: Blasius f''(0) = {sb.sol(0)[2]:.8f} (riferimento 0,33205734), {sb.x.size} nodi dopo il raffinamento")

# ------------------------------------------------------------------ 3) linalg
print("\n3) scipy.linalg")
n = 1000
dd = 0.8                                          # Crank-Nicolson: -d/2 u_(i-1) + (1+d) u_i - d/2 u_(i+1)
ab = np.zeros((3, n)); ab[0, 1:] = -dd / 2; ab[1] = 1 + dd; ab[2, :-1] = -dd / 2
rhs = np.sin(np.linspace(0, np.pi, n))
A = np.diag(np.full(n, 1 + dd)) + np.diag(np.full(n - 1, -dd / 2), 1) + np.diag(np.full(n - 1, -dd / 2), -1)
linalg.solve_banded((1, 1), ab, rhs); linalg.solve(A, rhs)        # prima chiamata "a freddo": non si cronometra
t0 = time.perf_counter(); x_b = linalg.solve_banded((1, 1), ab, rhs); tb = time.perf_counter() - t0
t0 = time.perf_counter(); x_f = linalg.solve(A, rhs); tf = time.perf_counter() - t0
print(f"   solve_banded contro solve (n = {n}): {tb*1e3:.2f} ms contro {tf*1e3:.2f} ms, differenza {np.max(np.abs(x_b - x_f)):.1e}")
lu, piv = linalg.lu_factor(A)
t0 = time.perf_counter()
for _ in range(100):
    linalg.lu_solve((lu, piv), rhs)
print(f"   lu_factor una volta + 100 lu_solve: {(time.perf_counter() - t0)*1e3:.1f} ms (100 solve: ~{100*tf*1e3:.0f} ms)")
m = 50; h = 1.0 / (m + 1)
D2 = (np.diag(np.full(m, -2.0)) + np.diag(np.ones(m - 1), 1) + np.diag(np.ones(m - 1), -1)) / h**2
lam = linalg.eigh(D2, eigvals_only=True)
print(f"   eigh: primi autovalori di d^2/dx^2 discreto {lam[-3:][::-1]}; esatti -(k pi)^2 = {-(np.arange(1, 4) * np.pi)**2}")

# ------------------------------------------------------------------ 4) sparse
print("\n4) scipy.sparse: equazione di Poisson lap u = f su griglia N x N con u = 0 al contorno")


def matrice_poisson(N, h):
    """Laplaciano a cinque punti sui nodi interni, formato lil (comodo da riempire) -> csc (comodo da fattorizzare)."""
    n = N * N
    idx = lambda i, j: i * N + j
    A = sparse.lil_matrix((n, n))
    for i in range(N):
        for j in range(N):
            p = idx(i, j)
            A[p, p] = -4.0 / h**2
            for a, b in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= a < N and 0 <= b < N:
                    A[p, idx(a, b)] = 1.0 / h**2
```

```
    return A.tocsc()


N = 60; h = 1.0 / (N + 1)
xg = np.linspace(h, 1 - h, N)
X, Y = np.meshgrid(xg, xg, indexing="ij")
u_es = np.sin(np.pi * X) * np.sin(np.pi * Y)
fsrc = (-2 * np.pi**2 * u_es).ravel()
t0 = time.perf_counter(); A = matrice_poisson(N, h); t_costr = time.perf_counter() - t0
print(f"   matrice {A.shape} con {A.nnz} elementi non nulli su {A.shape[0]**2} ({A.nnz / A.shape[0]**2:.2%}), costruita in {t_costr*1e3:.0f} ms")
t0 = time.perf_counter(); u1 = spla.spsolve(A, fsrc); t1 = time.perf_counter() - t0
t0 = time.perf_counter(); LU = spla.splu(A); t2 = time.perf_counter() - t0
t0 = time.perf_counter(); u2 = LU.solve(fsrc); t3 = time.perf_counter() - t0
t0 = time.perf_counter(); u3 = linalg.solve(A.toarray(), fsrc); t4 = time.perf_counter() - t0
print(f"   spsolve {t1*1e3:.1f} ms; splu {t2*1e3:.1f} ms + solve {t3*1e3:.2f} ms; matrice piena {t4*1e3:.0f} ms")
print(f"   errore rispetto alla soluzione esatta: {np.max(np.abs(u2 - u_es.ravel())):.2e} (O(h^2) = {h**2:.1e});"
      f" spsolve e splu coincidono: {np.allclose(u1, u2)}")

# ------------------------------------------------------------------ 5) interpolate e special
print("\n5) scipy.interpolate e scipy.special")
alfa = np.array([-4, -2, 0, 2, 4, 6, 8, 10, 12, 14])                # polare (dati tipici di un profilo)
cl = np.array([-0.2, 0.0, 0.25, 0.45, 0.67, 0.88, 1.08, 1.25, 1.35, 1.30])
lin = interpolate.interp1d(alfa, cl)
spl = interpolate.CubicSpline(alfa, cl)
print(f"   C_L(5 gradi): lineare {lin(5.0):.4f}, spline cubica {spl(5.0):.4f}; pendenza della spline in 0: {spl(0.0, 1):.4f} per grado "
      f"({np.degrees(spl(0.0, 1)):.3f} per radiante; teoria sottile: 2 pi = {2*np.pi:.3f})")
print(f"   np.interp e' l'equivalente NumPy per l'interpolazione lineare: {np.interp(5.0, alfa, cl):.4f}")
print(f"   erf(1) = {special.erf(1.0):.8f}, erfc(1) = {special.erfc(1.0):.8f} (soluzione di Rayleigh); J_0(2,4048) = {special.j0(2.4048):.2e} (zero di Bessel)")

# ------------------------------------------------------------------ figura
fig, ax = plt.subplots(1, 3, figsize=(9.0, 2.8))
tt = np.linspace(0, t_suolo, 200)
ax[0].plot(tt, s.sol(tt)[1], color=BLU, label="solve_ivp")
ax[0].plot(s.t, s.y[1], "o", color=ARANCIO, ms=2.5, label="passi adattivi")
ax[0].axhline(vt, color=GRIGIO, ls=":", label="$v_t$ (brentq)")
ax[0].set_xlabel("$t$ [s]"); ax[0].set_ylabel("$v$ [m/s]"); ax[0].legend(fontsize=7)
ax[1].plot(sb.x, sb.sol(sb.x)[1], color=BLU); ax[1].set_xlabel("$\\eta$"); ax[1].set_ylabel("$f'(\\eta)$"); ax[1].set_title("Blasius con solve\\_bvp", fontsize=8)
aa = np.linspace(-4, 14, 200)
ax[2].plot(aa, spl(aa), color=BLU, label="spline cubica"); ax[2].plot(aa, lin(aa), "--", color=ARANCIO, label="lineare")
ax[2].plot(alfa, cl, "o", color=GRIGIO, ms=3, label="dati")
ax[2].set_xlabel("$\\alpha$ [gradi]"); ax[2].set_ylabel("$C_L$"); ax[2].legend(fontsize=7)
fig.tight_layout()
salva(fig, "prog0_6_scipy")
```

Listato 6.1: Programma 0.6 – radici, integrali, problemi ai valori iniziali e ai limiti, sistemi densi, a banda e sparsi, interpolazione (`prog0_6_scipy.py`).

**Risultati del Programma 0.6**

- `brentq`: $v_t = 41{,}988$ m/s con $f(v_t) = 10^{-13}$ (il valore differisce dai 40,81 m/s del Programma 0.2 perché qui è usata la correlazione regolare di Clift–Gauvin al posto della legge a tratti: a $Re \approx 3 \times 10^4$ le due danno $c_d = 0{,}47$ e 0,50); `fsolve`: ristagno in $(-0{,}916515, -0{,}4)$, residuo $2 \times 10^{-16}$ in 11 valutazioni; `minimize_scalar`: 45,0000°.
- `quad`: $Q = 3{,}926991 \times 10^{-4}$ m$^3$/s, uguale a $\pi R^4(-\mathrm{d}p/\mathrm{d}x)/(8\mu)$, errore stimato $4 \times 10^{-18}$. `solve_ivp`: suolo a $t = 4{,}9167$ s con $v = 34{,}66$ m/s in 54 passi adattivi; $z(2\,\mathrm{s}) = 19{,}03$ m contro 19,61 nel vuoto. `solve_bvp`: $f''(0) = 0{,}33205734$ con 735 nodi dopo il raffinamento automatico.
- `solve_banded` 0,24 ms contro `solve` 12,3 ms per $n = 1000$, risultati identici; 100 `lu_solve` in 41 ms contro $\sim$ 1,2 s con 100 `solve`; i primi autovalori di $\mathrm{d}^2/\mathrm{d}x^2$ discreto, $-9{,}866$, $-39{,}43$, $-88{,}57$, approssimano $-(k\pi)^2$ con errore crescente con $k$, come l'analisi di Fourier del Modulo 2 prevede.
- Poisson $60 \times 60$: matrice $3600 \times 3600$ con 17 760 elementi non nulli; `spsolve` 8 ms, `splu` 8,5 ms più `solve` 0,5 ms, densa 983 ms; errore $2{,}2 \times 10^{-4} = \mathcal{O}(h^2)$.
- Spline: $C_L(5°) = 0{,}777$, pendenza in $\alpha = 0$ di 6,41 per radiante ($2\pi = 6{,}28$ per il profilo sottile); `erf(1)` $= 0{,}8427$; $J_0(2{,}4048) = 1{,}3 \times 10^{-5}$ (primo zero di Bessel).

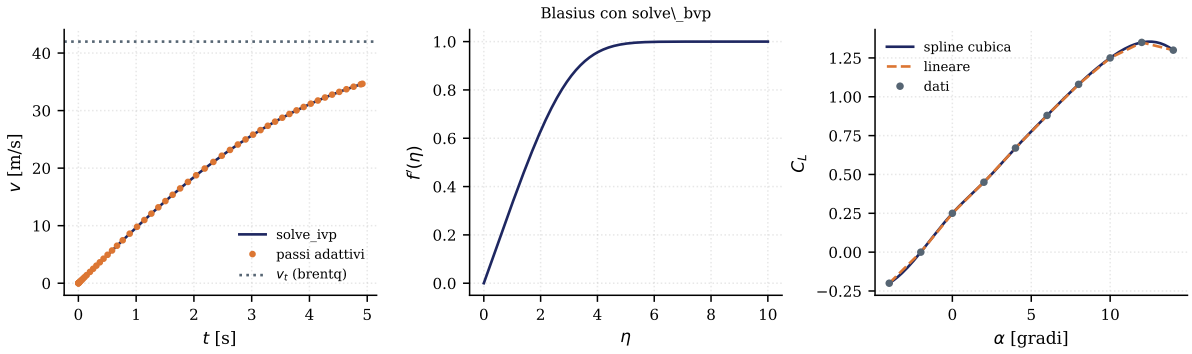



Figura 6.1: Programma 0.6: caduta della sfera con `solve_ivp` e istante dell'evento (a sinistra); profilo di Blasius $f'(\eta)$ da `solve_bvp` (al centro); polare interpolata con la spline cubica e i punti tabulati (a destra).

## 6.7 Approfondimenti

### 6.7.1 Tolleranze e messaggi

Ogni risolutore ha tolleranze con valori predefiniti pensati per la generalità, non per la precisione: `solve_ivp` usa `rtol=1e-3`, che per un confronto con una soluzione esatta è grossolano; il Programma 0.6 chiede `rtol=1e-10`. `quad` restituisce la stima dell'errore; `solve_ivp` e `solve_bvp` un campo `success` e un `message` da controllare (`assert s.success`,

`s.message`); `fsolve` con `full_output=True` restituisce lo stato della convergenza. Un risultato ottenuto senza guardare questi campi non è verificato.

### 6.7.2 Quando il risolutore "non converge"

Tre cause coprono quasi tutti i casi. (1) Il punto iniziale è lontano dalla soluzione o in un bacino diverso: `fsolve` e Newton sono locali; si parte da una stima fisica o da una soluzione a un parametro vicino (*continuazione*: il numero di Reynolds aumentato a gradini, come nel Modulo 4 per la sfera). (2) Il problema è *stiff*: scale temporali molto diverse (la diffusione su griglia fine ha $\Delta t \leq h^2/2\nu$), e un metodo esplicito riduce il passo fino a fermarsi; `method="Radau"` o `"BDF"` lo risolvono con passi grandi. (3) La funzione non è regolare o ha una singolarità nell'intervallo (`quad` segnala "integral is probably divergent" o raggiunge il limite di suddivisioni): si spezza l'intervallo nei punti critici (`points=[...]`). In tutti i casi il messaggio di SciPy dice quale.

### 6.7.3 Che cosa SciPy non sostituisce

I risolutori di SciPy sono scatole nere eccellenti, ma nel corso il metodo *è* l'argomento di studio: scrivere Thomas invece di chiamare `solve_banded` insegna la struttura del sistema, scrivere RK4 invece di chiamare `solve_ivp` insegna l'ordine e la stabilità. La pratica professionale ragionevole è l'opposto: usare SciPy per tutto ciò che non si sta studiando e verificare i propri algoritmi contro di essa, come fanno gli `assert` del Programma 0.9 (Thomas contro `solve`, RK4 contro la soluzione esatta). Gli stessi algoritmi delle monografie, scritti in Python puro, sono 10–100 volte più lenti delle versioni compilate: per problemi grandi si passa a `numba`, a `scipy.sparse` o a un codice compilato (Cap. 9).

# Capitolo 7

# Calcolo simbolico con SymPy

## 7.1 Simboli invece di numeri

Tutto ciò che precede calcola con *numeri*. SymPy calcola con *simboli*: deriva, integra, sviluppa in serie, risolve equazioni e sistemi in forma chiusa, e restituisce espressioni che si possono semplificare, stampare in LaTeX o trasformare in funzioni NumPy. Nelle monografie compare in tre ruoli: ricavare le formule alle differenze finite e i loro errori di troncamento (invece di farlo a mano con gli sviluppi di Taylor), verificare che una soluzione esatta soddisfi davvero l'equazione (la funzione di corrente di Stokes per la sfera, il potenziale del cilindro), e produrre lo jacobiano esatto per il metodo di Newton. Non è uno strumento di calcolo numerico: è il modo di *controllare* la matematica che sta dietro ai programmi.

**Simboli, espressioni, valutazione**

```
import sympy as sp
x, y = sp.symbols("x y")                          # simboli (senza ipotesi)
h = sp.symbols("h", positive=True)                # con ipotesi: semplifica
    sqrt(h**2) in h
e = (x**2 - y**2) / (x**2 + y**2)**2
e.subs({x: 1, y: 0})                              # sostituzione: 1
e.subs({x: 1.2, y: 0.3}).evalf()                  # valutazione numerica:
    0.576701
sp.simplify(e); sp.expand((x + y)**3); sp.factor(x**3 - y**3);
    sp.trigsimp(sp.sin(x)**2 + sp.cos(x)**2)
sp.Rational(1, 2); sp.pi.evalf(30); sp.sqrt(2)    # aritmetica esatta: 1/2
    resta 1/2, non 0.5
```

La trappola è scrivere `1/2` dove si intende la frazione: Python la calcola subito come 0,5 e l'espressione diventa numerica. Si usa `sp.Rational(1, 2)` o `sp.S(1)/2`. Le funzioni sono quelle di SymPy (`sp.sin`, `sp.exp`, `sp.sqrt`), non di `math` né di `numpy`.

## 7.2 Derivate, integrali, serie

```
sp.diff(psi, x, 2) + sp.diff(psi, y, 2)        # laplaciano: 0 se psi e' armonica
sp.integrate(2*sp.pi*r*u, (r, 0, R))           # integrale definito: la portata di
    Poiseuille
sp.integrate(sp.exp(-x**2), (x, -sp.oo, sp.oo))    # sqrt(pi)
sp.limit(sp.sin(x)/x, x, 0)                    # 1
```

```
sp.series(sp.exp(x), x, 0, 5)                    # 1 + x + x**2/2 + x**3/6 + x**4/24
    + O(x**5)
```

Con questi quattro strumenti si verifica ogni soluzione esatta delle monografie: il programma deriva $\psi$ del cilindro e trova $\nabla^2\psi = 0$, le componenti nulle nel punto di ristagno e $u = 2U$ in $(0, a)$; integra il profilo di Poiseuille e ottiene $Q = \pi G R^4/(8\mu)$ e $\tau_w = -GR/2$.

## 7.3 Le formule alle differenze finite con i coefficienti indeterminati

**Il metodo**

Si vuole la formula $f'(x) \approx [c_0 f(x) + c_1 f(x+h) + c_2 f(x+2h)]/h$ più accurata possibile. Si rappresentano i valori di $f$ e delle sue derivate nel nodo con simboli $f_0, f_1, f_2, \ldots$ e si scrive lo sviluppo di Taylor

$$f(x+kh) = f_0 + khf_1 + \frac{(kh)^2}{2} f_2 + \frac{(kh)^3}{6} f_3 + \frac{(kh)^4}{24} f_4 + \ldots \tag{7.1}$$

come `taylor = lambda k: f0 + k*h*f1 + (k*h)**2/2*f2 + ...`. La combinazione `(c0*taylor(0) + c1*taylor(1) + c2*taylor(2))/h`, espansa, è un polinomio in $f_0, f_1, f_2, \ldots$; si impone che il coefficiente di $f_0$ sia zero, quello di $f_1$ uno, quello di $f_2$ zero (tre equazioni lineari in $c_0, c_1, c_2$, `sp.solve`), e il primo coefficiente non annullato, quello di $f_3$, è l'errore di troncamento:

$$f'(x) = \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} - \frac{h^2}{3} f'''(x) + \ldots \tag{7.2}$$

Lo stesso codice, con cinque nodi, dà la formula centrata del quarto ordine (Esercizio 15); con `taylor(1) - taylor(-1)` dà la differenza centrata e il suo errore $h^2 f'''/6$; con `taylor(1) - 2*taylor(0) + taylor(-1)` la derivata seconda ed $h^2 f''''/12$. Questo è il modo in cui tutte le formule del Modulo 2 possono essere ricavate e controllate in pochi secondi.

## 7.4 Equazioni, sistemi, `lambdify`

La funzione `sp.solve(eq, var)` risolve in forma chiusa: nel programma, la velocità limite con $c_d$ costante, $v_t = \sqrt{2mg/(\rho A c_d)}$; le radici di un polinomio; i coefficienti dello spessore NACA imposti come vincoli. `sp.nsolve` risolve numericamente quando non c'è forma chiusa ($\cos x = x$). `sp.lambdify([x, y], espr, "numpy")` trasforma un'espressione in una funzione Python che accetta array NumPy: è il ponte fra i due mondi, e permette di derivare simbolicamente una funzione complicata (lo jacobiano di un sistema, `F.jacobian([x, y])`) e di usarla numericamente nel metodo di Newton senza errori di derivazione a mano e senza differenze finite. Infine `sp.latex(espr)` produce il codice LaTeX dell'espressione, pronto per il documento.

## 7.5 Il Programma 0.7

**Dati del Programma 0.7**

Espressione $\partial_x(x/r^2)$ del dipolo; $\psi$ del cilindro con $U = a = 1$; profilo di Poiseuille $u = G(R^2 - r^2)/(4\mu)$; sviluppi di Taylor al quarto ordine; vincoli NACA: $y_t(0{,}3) = 0{,}1$, $y_t'(0{,}3) = 0$, $y_t(1) = 0$ con $a_0 = 0{,}2969$ e $a_1 = 0{,}1260$ assegnati; sistema $x^2 + y^2 = 4$, $e^x + y = 1$ da $(-1{,}5; 1)$; funzione di corrente di Stokes $\psi = (r^2/2 - 3r/4 + 1/(4r))(1 - \zeta^2)$.

prog0_7_sympy.py

```
"""
prog0_7_sympy.py -- Programma 0.7: calcolo simbolico con SymPy.

1) Simboli, espressioni, semplificazione, sostituzione, valutazione numerica.
2) Derivate e integrali: la portata di Poiseuille, la funzione di corrente del
   dipolo, verifica che psi = U y (1 - a^2/r^2) e' armonica.
3) Serie di Taylor e derivazione delle formule alle differenze finite: la
   differenza centrata, la formula unilatera a tre punti e il suo errore di
   troncamento (il metodo dei coefficienti indeterminati, risolto con solve).
4) Equazioni: soluzione simbolica (velocita' limite con c_d costante), sistema
   lineare (coefficienti dello spessore NACA imposti come vincoli).
5) lambdify: da espressione simbolica a funzione NumPy vettorializzata; lo
   jacobiano esatto per il metodo di Newton.
6) Verifica di un risultato delle monografie: la funzione di corrente di Stokes
   per la sfera soddisfa D^4 psi = 0 e le condizioni di aderenza.
"""
import numpy as np
import sympy as sp

# ------------------------------------------------------------------ 1) simboli ed espressioni
print("1) Simboli ed espressioni")
x, y, r, theta, a, U = sp.symbols("x y r theta a U", positive=True)
espr = (x**2 - y**2) / (x**2 + y**2)**2
print(f"   espressione: {espr};  in (1, 0): {espr.subs({x: 1, y: 0})};  numerica in (1,2, 0,3): {espr.subs({x: 1.2, y: 0.3}).evalf(6)}")
print(f"   semplificazione: {sp.simplify((x**2 - 1) / (x - 1))};  espansione: {sp.expand((x + y)**3)};  fattorizzazione: {sp.factor(x**3 - y**3)}")
print(f"   trigonometria: {sp.trigsimp(sp.sin(theta)**2 + sp.cos(theta)**2)};  pi con 30 cifre: {sp.N(sp.pi, 30)};  razionali esatti: {sp.Rational(1, 3) + sp.Rational(1, 6)}")

# ------------------------------------------------------------------ 2) derivate e integrali
print("\n2) Derivate e integrali")
R, mu, G = sp.symbols("R mu G", positive=True)      # G = -dp/dx
u_pois = G / (4 * mu) * (R**2 - r**2)
Q = sp.integrate(2 * sp.pi * r * u_pois, (r, 0, R))
print(f"   portata di Poiseuille: Q = {sp.simplify(Q)};  sforzo alla parete tau = mu du/dr|_R = {sp.simplify(mu * sp.diff(u_pois, r).subs(r, R))}")
psi = U * y * (1 - a**2 / (x**2 + y**2))
lap = sp.simplify(sp.diff(psi, x, 2) + sp.diff(psi, y, 2))
u_c, v_c = sp.diff(psi, y), -sp.diff(psi, x)
print(f"   psi del cilindro: laplaciano = {lap};  velocita' in (a, 0): u = {sp.simplify(u_c.subs({x: a, y: 0}))}, v = {sp.simplify(v_c.subs({x: a, y: 0}))} (ristagno)")
print(f"   velocita' in (0, a): u = {sp.simplify(u_c.subs({x: 0, y: a}))} (il doppio di U: C_p = -3)")
print(f"   integrale indefinito: {sp.integrate(sp.exp(-x**2) * x, x)};  definito:
```

```
    {sp.integrate(sp.exp(-x**2), (x, -sp.oo, sp.oo))};  limite: {sp.limit(sp.sin(x)
    / x, x, 0)}")
40
41  # ------------------------------------------------------------------ 3) serie di
    Taylor e differenze finite
42  print("\n3) Serie di Taylor e formule alle differenze finite")
43  h = sp.symbols("h", positive=True)
44  print(f"   serie di exp(x) in x = 0: {sp.series(sp.exp(x), x, 0, 5)};  di sin(x)/x:
    {sp.series(sp.sin(x) / x, x, 0, 6)}")
45  # Per ricavare le formule alle differenze si rappresentano i valori di f, f', f'',
    ... nel nodo x con
46  # simboli f0, f1, f2, ... e si scrive lo sviluppo di Taylor di f(x + k h) "a mano"
47  f0, f1, f2, f3, f4 = sp.symbols("f0 f1 f2 f3 f4")
48  taylor = lambda k: f0 + k * h * f1 + (k * h)**2 / 2 * f2 + (k * h)**3 / 6 * f3 + (k
    * h)**4 / 24 * f4
49  centrata = sp.expand((taylor(1) - taylor(-1)) / (2 * h))
50  print(f"   differenza centrata: {centrata}  -> f1 + h^2 f3/6: errore di troncamento
    del secondo ordine")
51  seconda = sp.expand((taylor(1) - 2 * taylor(0) + taylor(-1)) / h**2)
52  print(f"   derivata seconda centrata: {seconda}  -> f2 + h^2 f4/12")
53  # metodo dei coefficienti indeterminati: f' ~ [c0 f(x) + c1 f(x+h) + c2 f(x+2h)]/h
    esatta per i polinomi di grado <= 2
54  c0, c1, c2 = sp.symbols("c0 c1 c2")
55  approx = sp.expand((c0 * taylor(0) + c1 * taylor(1) + c2 * taylor(2)) / h)
56  eqs = [approx.coeff(f0), approx.coeff(f1) - 1, approx.coeff(f2)]   # coefficiente
    di f0 nullo, di f1 unitario, di f2 nullo
57  sol = sp.solve(eqs, [c0, c1, c2])
58  print(f"   formula unilatera a tre punti: c0 = {sol[c0]}, c1 = {sol[c1]}, c2 =
    {sol[c2]}  ->  f' = (-3f0 + 4f1 - f2)/(2h)")
59  errore = sp.expand(approx.subs(sol) - f1)
60  print(f"   errore di troncamento: {errore.coeff(f3) * f3}  (secondo ordine; e' la
    formula della vorticita' di parete del Modulo 4)")
61  # la formula di parete di Thom: psi(h) = psi(0) + h psi'(0) + h^2/2 psi''(0) con
    psi(0) = psi'(0) = 0
62  psi1, om = sp.symbols("psi_1 omega")
63  print(f"   vorticita' di parete (Thom): da psi_1 = h^2/2 psi'' e omega = -psi''
    segue omega = {sp.solve(sp.Eq(psi1, h**2 / 2 * (-om)), om)[0]}")
64
65  # ------------------------------------------------------------------ 4) equazioni e
    sistemi
66  print("\n4) Equazioni e sistemi")
67  v, m, g, rho, A_, cd = sp.symbols("v m g rho A c_d", positive=True)
68  print(f"   velocita' limite (c_d costante): v_t = {sp.solve(sp.Eq(m * g,
    sp.Rational(1, 2) * rho * v**2 * A_ * cd), v)[0]}")
69  print(f"   radici di x^3 - 6x^2 + 11x - 6: {sp.solve(x**3 - 6 * x**2 + 11 * x - 6,
    x)};  numeriche di cos x = x: {sp.nsolve(sp.cos(x) - x, x, 0.7)}")
70  # spessore NACA: il profilo passa per (0,3; 0,1) con tangente nulla ed e' chiuso al
    bordo d'uscita (t = 0,2):
71  # si impongono tre vincoli sui coefficienti e si ritrovano (quasi) quelli classici
72  a0, a1, a2, a3, a4 = sp.symbols("a0:5")
73  yt = a0 * sp.sqrt(x) - a1 * x - a2 * x**2 + a3 * x**3 - a4 * x**4
74  vincoli = [sp.Eq(yt.subs(x, sp.Rational(3, 10)), sp.Rational(1, 10)),
    sp.Eq(sp.diff(yt, x).subs(x, sp.Rational(3, 10)), 0),
75             sp.Eq(yt.subs(x, 1), 0)]
76  sol_naca = sp.solve(vincoli, [a2, a3, a4])          # a0 = 0,2969 e a1 = 0,1260 dati
77  sol_num = {k: float(val.subs({a0: 0.2969, a1: 0.1260})) for k, val in
    sol_naca.items()}
78  print(f"   coefficienti NACA dai vincoli (a0, a1 dati): {sol_num}  (classici:
    0,3516, 0,2843, 0,1036)")
79
```

```python
# ---------------------------------------------------------------- 5) lambdify e jacobiano
print("\n5) lambdify e jacobiano esatto per Newton")
g1 = x**2 + y**2 - 4
g2 = sp.exp(x) + y - 1
F = sp.Matrix([g1, g2]); J = F.jacobian([x, y])
F_num = sp.lambdify([x, y], F, "numpy"); J_num = sp.lambdify([x, y], J, "numpy")
p = np.array([-1.5, 1.0])
for k in range(6):
    p = p - np.linalg.solve(np.array(J_num(*p), float), np.array(F_num(*p), float).ravel())
print(f"   J = {J.tolist()}; Newton da (-1,5, 1): {p} con residuo {np.max(np.abs(F_num(*p))):.1e}")
cp = sp.lambdify(theta, 1 - (2 * sp.sin(theta))**2, "numpy")
th = np.linspace(0, np.pi, 5)
print(f"   lambdify di C_p(theta) valutata su un array: {cp(th)}")

# ---------------------------------------------------------------- 6) verifica: funzione di corrente di Stokes
print("\n6) La funzione di corrente di Stokes per la sfera (Modulo 3 e 4)")
z = sp.symbols("zeta", real=True)
psi_s = (r**2 / 2 - sp.Rational(3, 4) * r + 1 / (4 * r)) * (1 - z**2)
D2 = lambda F: sp.diff(F, r, 2) + (1 - z**2) / r**2 * sp.diff(F, z, 2)
print(f"   D^4 psi = {sp.simplify(D2(D2(psi_s)))};  psi(1) = {sp.simplify(psi_s.subs(r, 1))};  psi_r(1) = {sp.simplify(sp.diff(psi_s, r).subs(r, 1))}")
print(f"   per r -> oo: psi/(r^2/2 (1 - zeta^2)) -> {sp.limit(psi_s / (r**2 / 2 * (1 - z**2)), r, sp.oo)} (corrente uniforme)")
print(f"   LaTeX pronto per il documento: {sp.latex(psi_s)}")
```

Listato 7.1: Programma 0.7 – simboli, derivate e integrali, formule alle differenze con i coefficienti indeterminati, equazioni, `lambdify`, verifica della soluzione di Stokes (`prog0_7_sympy.py`).

**Risultati del Programma 0.7**

- Semplificazione, espansione, fattorizzazione e trigonometria funzionano come atteso; $\pi$ con 30 cifre; `sp.Rational(1, 2)` resta $1/2$.
- $Q = \pi G R^4/(8\mu)$, $\tau_w = -GR/2$; $\nabla^2\psi = 0$ per il cilindro, velocità nulla in $(a, 0)$ e $u = 2U$ in $(0, a)$; $\int e^{-x^2} = \sqrt{\pi}$.
- Differenza centrata: $f_1 + h^2 f_3/6$; derivata seconda: $f_2 + h^2 f_4/12$; formula unilatera: $c_0 = -3/2$, $c_1 = 2$, $c_2 = -1/2$, errore $-h^2 f_3/3$; dalla formula di Thom $\omega_w = -2\psi_1/h^2$, la condizione di parete usata nel Modulo 4.
- $v_t = \sqrt{2gm/(Ac_d\rho)}$; radici $1, 2, 3$; $\cos x = x$ in 0,739085; coefficienti NACA dai vincoli 0,3538, 0,2931, 0,1102 (i classici 0,3516, 0,2843, 0,1036 furono scelti con un vincolo diverso sul bordo d'uscita: il programma mostra quanto i vincoli determinano i coefficienti).
- Newton con lo jacobiano esatto da `lambdify`: $(-1{,}8163;\ 0{,}8374)$ con residuo nullo in 6 iterazioni; $C_p(\theta)$ valutato su un array.
- Stokes: $D^4\psi = 0$, $\psi(1) = 0$, $\psi_r(1) = 0$, $\psi \to r^2(1-\zeta^2)/2$ per $r \to \infty$: la soluzione usata nei Moduli 3 e 4 è verificata; `sp.latex` produce $(1-\zeta^2)\left(\frac{r^2}{2} - \frac{3r}{4} + \frac{1}{4r}\right)$.

**Un uso tipico: controllare una formula prima di programmarla**

Nel Modulo 4 la vorticità alla parete è calcolata con la formula di Thom, $\omega_w = -2\psi_1/h^2$, e in alternativa con quella di Jensen, $\omega_w = (-8\psi_1 + \psi_2)/(2h^2)$. Con dieci righe di SymPy si ricavano entrambe dai vincoli $\psi(0) = \psi'(0) = 0$ e dagli sviluppi di $\psi(h)$ e $\psi(2h)$, e si scopre che la prima ha errore $\mathcal{O}(h)$ e la seconda $\mathcal{O}(h^2)$: lo stesso ragionamento del programma, applicato a $\psi$ invece che a $f$. Prima di scrivere una riga di NumPy conviene chiedere a SymPy se la formula è quella giusta e di che ordine è.

## 7.6 Approfondimenti

### 7.6.1 Limiti del calcolo simbolico

SymPy risolve in forma chiusa solo ciò che *ha* una forma chiusa: `solve` su un'equazione trascendente restituisce un errore o una risposta implicita, `integrate` può restituire l'integrale non valutato, `simplify` non trova sempre la forma più semplice (e "più semplice" non è univoco: `sp.trigsimp`, `sp.cancel`, `sp.together`, `sp.nsimplify` sono alternative). Le espressioni grandi crescono in fretta e le operazioni diventano lente; le ipotesi sui simboli (`positive=True`, `real=True`) aiutano molto perché permettono semplificazioni come $\sqrt{h^2} = h$ che altrimenti sono false. Per i calcoli numerici SymPy è lentissimo rispetto a NumPy: una funzione simbolica valutata in un ciclo va sempre convertita con `lambdify`.

### 7.6.2 Simbolico e numerico insieme

Lo schema di lavoro più produttivo nel corso è: ricavare (o verificare) con SymPy le formule, convertirle con `lambdify` o trascriverle, e calcolare con NumPy. Due esempi: le soluzioni manifatturate (*method of manufactured solutions*) scelgono una $u$ arbitraria, ne calcolano il termine sorgente $f = \mathcal{L}u$ simbolicamente e verificano che il codice ritrovi $u$ da $f$ con l'ordine atteso (l'Esercizio 10 usa questa idea con $f = -2\pi^2 \sin \pi x \sin \pi y$); l'analisi di stabilità di von Neumann di uno schema sostituisce $u_j^n = g^n e^{\mathrm{i}kjh}$ nello schema e risolve per il fattore di amplificazione $g$, cosa che SymPy fa in due righe e che nel Modulo 2 è fatta a mano.

# Capitolo 8

# Anatomia di un programma del corso

## 8.1 Il programma modello

Tutti i programmi delle monografie hanno la stessa struttura, e riconoscerla è la metà del leggerli. Il Programma 0.8 la realizza nella forma più pulita su un problema semplice, l'equazione della diffusione monodimensionale con lo schema esplicito FTCS, e serve da *modello* per i programmi che il lettore scriverà.

**Le sei parti di un programma del corso**

1. **Docstring del modulo** (righe 1–16): il problema, le equazioni, lo schema con le sue formule, le verifiche previste, l'uso da riga di comando. È la prima cosa da leggere, e la prima da scrivere: se non si riesce a scriverla, non si è pronti a programmare.
2. **Import** (17–21): la libreria standard (`sys`, `time`, `math`), poi NumPy, SciPy, Matplotlib, poi i moduli del corso (`stile_grafici`, `metodi_numerici`, `ode_solvers`).
3. **Costanti** in maiuscolo (23–25): i parametri fisici e numerici predefiniti, con unità nel commento.
4. **Funzioni** piccole, ognuna con un compito e una docstring (28–56): la soluzione esatta, *un* passo dello schema, l'integrazione completa, la misura dell'errore. Ricevono tutto come argomento e leggono al più le costanti; non stampano e non disegnano.
5. **Blocco principale** `if __name__ == "__main__":` (59–108), diviso in sezioni numerate con commenti: un calcolo e la sua verifica contro la soluzione esatta; l'ordine di convergenza; l'estrapolazione di Richardson; la stabilità; la figura.
6. **Uscita**: tabelle stampate con le f-string allineate, numeri di riferimento accanto a quelli calcolati, figure salvate con `salva`.

**Dal metodo all'algoritmo al programma**

Il percorso che le monografie seguono per ogni problema, e che si consiglia per gli esercizi, è: (a) scrivere le equazioni e le condizioni al contorno; (b) scegliere la discretizzazione e scrivere lo schema nodo per nodo, $u_i^{n+1} = u_i^n + R(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$; (c) tradurre lo schema in *una* funzione che fa *un* passo con lo slicing, `u[1:-1] += R*(u[2:] -`

`2*u[1:-1] + u[:-2])`; (d) avvolgerla in una funzione che fa la marcia nel tempo e restituisce i risultati; (e) prima di qualunque figura, verificare contro una soluzione nota e misurare l'ordine; (f) solo allora esplorare i parametri e disegnare. L'ordine delle fasi (c)–(e) è ciò che distingue un programma che si può credere da uno che sembra funzionare.

**Dati del Programma 0.8**

$u_t = \nu u_{xx}$ su $[0,1]$, $u = 0$ agli estremi, $u(x,0) = \sin \pi x$, soluzione esatta $\sin \pi x\, e^{-\nu \pi^2 t}$; $\nu = 1$, $t_{\text{fin}} = 0{,}1$; $R = \nu \Delta t / h^2 = 0{,}4$ (modificabile da riga di comando: `python3 prog0_8_anatomia.py 0.25`); griglie $n = 10, 20, 40, 80, 160$; stabilità con $R = 0{,}1$, $1/6$, $0{,}3$, $0{,}5$, $0{,}51$, $0{,}6$.

prog0_8_anatomia.py

```
"""
prog0_8_anatomia.py -- Programma 0.8: anatomia di un programma delle monografie.

Il "programma modello" del corso, nella forma che tutti i programmi delle monografie
seguono: (1) docstring con il problema, le equazioni e lo schema; (2) import;
(3) costanti in maiuscolo; (4) funzioni piccole e riutilizzabili con docstring;
(5) blocco  if __name__ == "__main__"  con le sezioni: calcolo, verifica contro una
soluzione esatta, ordine di convergenza, tabella stampata, figure salvate.

Problema: equazione della diffusione u_t = nu u_xx su 0 <= x <= 1, u = 0 agli
    estremi,
u(x, 0) = sin(pi x); soluzione esatta u = sin(pi x) exp(-nu pi^2 t). Schema
    esplicito
FTCS, u_i^(n+1) = u_i^n + R (u_(i+1)^n - 2 u_i^n + u_(i-1)^n), R = nu dt/h^2 <= 1/2.
Verifiche: errore O(h^2) a R fissato (l'ordine misurato e' 2), estrapolazione di
Richardson, instabilita' per R > 1/2, caso speciale R = 1/6 (ordine 4).
Uso:  python3 prog0_8_anatomia.py [R]      (R predefinito 0,4)
"""
import sys
import time
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO

NU = 1.0                        # diffusivita'
T_FIN = 0.1                     # istante finale
R_DEFAULT = 0.4                 # numero di diffusione nu dt/h^2


def esatta(x, t, nu=NU):
    """Soluzione esatta del problema modello."""
    return np.sin(np.pi * x) * np.exp(-nu * np.pi**2 * t)


def passo_ftcs(u, R):
    """Un passo dello schema esplicito; u e' modificato sul posto nei nodi
    interni."""
    u[1:-1] = u[1:-1] + R * (u[2:] - 2 * u[1:-1] + u[:-2])
    return u


def integra(n, R, t_fin=T_FIN, nu=NU):
    """Integra su n intervalli (n+1 nodi) con numero di diffusione R fino a t_fin.
```

```
    Restituisce x, u(x, t_fin), il tempo effettivo raggiunto e il numero di passi."""
    x = np.linspace(0.0, 1.0, n + 1)
    h = x[1] - x[0]
    dt = R * h**2 / nu
    n_passi = int(np.ceil(t_fin / dt))          # ultimo passo accorciato per arrivare esattamente a t_fin
    u = np.sin(np.pi * x)
    for k in range(n_passi):
        dt_k = min(dt, t_fin - k * dt)
        passo_ftcs(u, nu * dt_k / h**2)
    return x, u, t_fin, n_passi


def errore(n, R):
    """Errore massimo a t_fin rispetto alla soluzione esatta."""
    x, u, t, _ = integra(n, R)
    return np.max(np.abs(u - esatta(x, t)))


if __name__ == "__main__":
    R = float(sys.argv[1]) if len(sys.argv) > 1 else R_DEFAULT
    print(f"Diffusione 1D con lo schema FTCS, R = {R}, t_fin = {T_FIN}")

    # --- 1) un calcolo e la sua verifica
    n = 40
    t0 = time.perf_counter()
    x, u, t, n_passi = integra(n, R)
    durata = time.perf_counter() - t0
    err = np.max(np.abs(u - esatta(x, t)))
    print(f"   n = {n}: {n_passi} passi in {durata*1e3:.1f} ms; errore massimo {err:.3e}")
    print(f"   {'x':>6} {'numerica':>10} {'esatta':>10} {'errore':>10}")
    for i in range(0, n + 1, 8):
        print(f"   {x[i]:6.2f} {u[i]:10.6f} {esatta(x[i], t):10.6f} {u[i] - esatta(x[i], t):10.2e}")

    # --- 2) ordine di convergenza (R fissato: dt ~ h^2, l'errore totale e' O(h^2))
    print("\n   Ordine di convergenza (R fissato):")
    ns = [10, 20, 40, 80, 160]
    errs = [errore(n, R) for n in ns]
    for n, e, e_prec in zip(ns, errs, [None] + errs[:-1]):
        print(f"   n = {n:4d}: errore {e:.3e}" + (f", rapporto {e_prec / e:.2f}" if e_prec else ""))
    ordine = np.polyfit(np.log(ns), np.log(errs), 1)[0]
    print(f"   pendenza nel piano log-log: {-ordine:.2f} (atteso 2; con R = 1/6 si ottiene 4)")

    # --- 3) estrapolazione di Richardson con le soluzioni n = 80 e n = 160
    x80, u80, _, _ = integra(80, R)
    x160, u160, _, _ = integra(160, R)
    u_rich = (4 * u160[::2] - u80) / 3            # combinazione che elimina il termine O(h^2)
    print(f"\n   Richardson: errore n = 160: {np.max(np.abs(u160 - esatta(x160, T_FIN))):.3e}, "
          f"estrapolato: {np.max(np.abs(u_rich - esatta(x80, T_FIN))):.3e}")

    # --- 4) stabilita'
    print("\n   Stabilita' (n = 40): errore per vari R")
    for Rr in (0.1, 1 / 6, 0.3, 0.5, 0.51, 0.6):
```

```
93          with np.errstate(over="ignore", invalid="ignore"):
94              e = errore(40, Rr)
95          print(f"   R = {Rr:.3f}: " + ("instabile" if not np.isfinite(e) or e > 1
    else f"errore {e:.3e}"))
96
97      # --- 5) figura
98      fig, ax = plt.subplots(1, 2, figsize=(7.4, 2.8))
99      xe = np.linspace(0, 1, 200)
100     ax[0].plot(xe, esatta(xe, T_FIN), color=GRIGIO, label="esatta")
101     ax[0].plot(x, u, "o", color=BLU, ms=3, mfc="none", label=f"FTCS, $n={n}$")
102     ax[0].set_xlabel("$x$"); ax[0].set_ylabel("$u(x, t_{\\mathrm{fin}})$");
    ax[0].legend(fontsize=7)
103     ax[1].loglog(ns, errs, "o-", color=BLU, label=f"$R={R}$")
104     ax[1].loglog(ns, [errore(n, 1 / 6) for n in ns], "s-", color=ARANCIO,
    label="$R=1/6$")
105     ax[1].loglog(ns, errs[0] * (np.array(ns) / ns[0]) ** -2.0, ":", color=GRIGIO,
    label="pendenza $-2$")
106     ax[1].set_xlabel("$n$"); ax[1].set_ylabel("errore massimo");
    ax[1].legend(fontsize=7)
107     fig.tight_layout()
108     salva(fig, "prog0_8_anatomia")
```

Listato 8.1: Programma 0.8 – il programma modello del corso: diffusione monodimensionale con lo schema FTCS, verifica, ordine, Richardson, stabilità (`prog0_8_anatomia.py`).

**Risultati del Programma 0.8**

- $n = 40$, $R = 0{,}4$: 400 passi in 2 ms, errore massimo $2{,}65 \times 10^{-4}$, con la tabella numerica/esatta/errore.
- Ordine: errori $4{,}29 \times 10^{-3}$, $1{,}06 \times 10^{-3}$, $2{,}65 \times 10^{-4}$, $6{,}62 \times 10^{-5}$, $1{,}66 \times 10^{-5}$, rapporti 4,04, 4,01, 4,00, 4,00; pendenza 2,00 (Fig. 8.1). A $R$ fissato $\Delta t \propto h^2$, quindi l'errore totale $\mathcal{O}(\Delta t) + \mathcal{O}(h^2)$ è $\mathcal{O}(h^2)$.
- Richardson: $(4u_{160} - u_{80})/3$ porta l'errore da $1{,}66 \times 10^{-5}$ a $3{,}5 \times 10^{-9}$: la combinazione elimina il termine $h^2$ e lascia l'$h^4$.
- Stabilità: errore finito per $R \leq 0{,}51$ (a 0,51 cresce ma non esplode in 400 passi), `nan` a $R = 0{,}6$; il caso $R = 1/6$ dà errore $2{,}6 \times 10^{-8}$, quattro ordini di grandezza meglio, perché i termini di troncamento $\Delta t$ e $h^2$ si elidono (ordine 4, come la curva arancione in figura mostra).

## 8.2 Lettura guidata 1: la caduta della sfera (Modulo 1)

Il primo programma del corso integra con Runge–Kutta 4 il sistema $\mathrm{d}z/\mathrm{d}t = v$, $\mathrm{d}v/\mathrm{d}t = F(v)$ della sfera in caduta, con il coefficiente di resistenza a tratti importato da `ode_solvers.py`. Si legga la funzione `caduta` (righe 25–44) riconoscendo i costrutti dei capitoli precedenti.

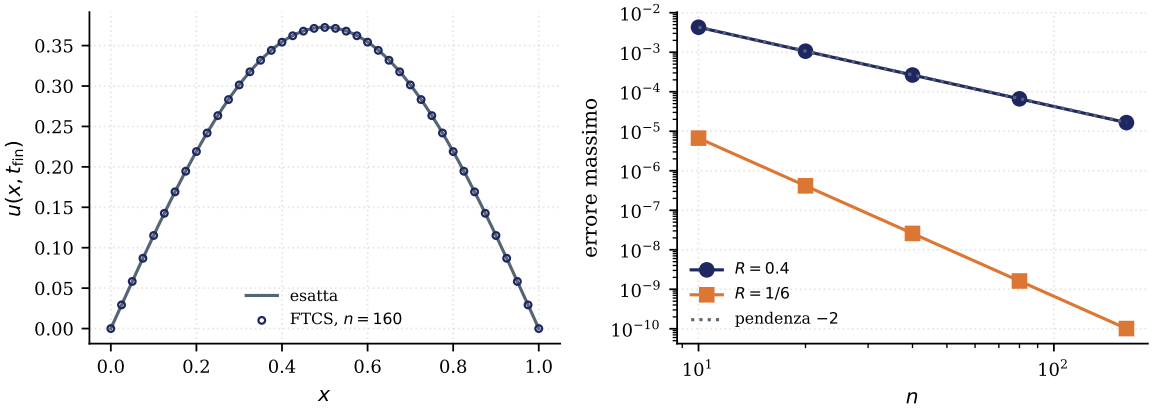



Figura 8.1: Programma 0.8: soluzione numerica ed esatta a $t_{\text{fin}}$ (a sinistra); errore in funzione di $n$ per $R = 0{,}4$ e $R = 1/6$ con la retta di pendenza $-2$ (a destra).

```
17 RHO, RHOF, NU, G = 8000.0, 1.22, 1.49e-5, 9.8   # acciaio in aria (livello del mare)
18
19
20 def costanti(d, rho=RHO, rhof=RHOF):
21     rb = rhof / rho
22     return 1.0 + 0.5 * rb, (1.0 - rb) * G, 3.0 * rb / (4.0 * d)
23
24
25 def caduta(d, h=0.1, tmax=10.0, z0=0.0, v0=0.0, rho=RHO, rhof=RHOF, nu=NU):
26     """Integra il moto con RK4 e restituisce gli array t, z, v, Re."""
27     A, B, C = costanti(d, rho, rhof)
28
29     def F(v):                                        # accelerazione dv/dt
30         return (B - C * v * abs(v) * cd_sfera(v * d / nu)) / A
31
32     t, z, v = 0.0, z0, v0
33     T, Z, V = [t], [z], [v]
34     while t < tmax - 1e-12:
35         d1z = h * v;                d1v = h * F(v)
36         d2z = h * (v + d1v / 2);    d2v = h * F(v + d1v / 2)
37         d3z = h * (v + d2v / 2);    d3v = h * F(v + d2v / 2)
38         d4z = h * (v + d3v);        d4v = h * F(v + d3v)
39         z += (d1z + 2 * d2z + 2 * d3z + d4z) / 6
40         v += (d1v + 2 * d2v + 2 * d3v + d4v) / 6
41         t += h
42         T.append(t); Z.append(z); V.append(v)
43     T, Z, V = map(np.array, (T, Z, V))
44     return T, Z, V, V * d / nu
```

monografia1/prog1_1_caduta_sfera.py

Listato 8.2: Lettura guidata 1 – la funzione `caduta` del Programma 1.1 (Modulo 1): Runge–Kutta 4 per il sistema $z$, $v$ con funzione annidata e accumulo in liste.

**Che cosa c'è in queste trenta righe**

*Costanti* in maiuscolo con un commento sulle unità (r. 17). *Spacchettamento* di una tupla restituita (27). Una *funzione annidata* `F(v)` (29–30) che vede `A, B, C, d, nu` della funzione che la contiene: è il secondo membro dell'equazione, costruito con i parametri correnti, e la sua definizione dentro `caduta` evita di passare cinque argomenti a ogni chiamata. *Assegnazione multipla* `t, z, v = 0.0, z0, v0` (32) e *liste* per accumulare

la storia (33, 42). Il ciclo `while t < tmax - 1e-12` (34) con la tolleranza che evita il passo in più dell'accumulo (Cap. 2). I quattro stadi di RK4 scritti *in parallelo* per le due equazioni, con due istruzioni per riga separate da `;` (35–38): è lo stesso `passo_rk4` di `ode_solvers.py`, scritto per esteso. `map(np.array, (T, Z, V))` (43) converte le tre liste in array, e la funzione restituisce quattro array, l'ultimo calcolato al volo (`V * d / nu`: aritmetica vettoriale). Il blocco principale (47 in poi) chiama `caduta` per varie `h` e vari diametri, stampa tabelle con le f-string e disegna con `zip(diametri, PALETTE)`.

Il modulo `ode_solvers.py` che fornisce `cd_sfera` merita di essere letto per intero, perché è il modello di una *libreria* del corso: funzioni di passo intercambiabili (tutte con la stessa firma `(f, t, y, h)`), un dizionario `METODI` che le seleziona per nome, una marcia nel tempo `integra` generica con una funzione di arresto opzionale, e la correlazione $c_d(Re)$. Lo stato `y` è un array, così le stesse formule valgono per un'equazione e per un sistema (`np.atleast_1d`).

```
14 def passo_eulero(f, t, y, h):
15     """Metodo di Eulero esplicito (ordine 1)."""
16     return y + h * f(t, y)
17
18
19 def passo_rk2(f, t, y, h):
20     """Runge-Kutta del secondo ordine (metodo del punto medio)."""
21     k1 = h * f(t, y)
22     k2 = h * f(t + 0.5 * h, y + 0.5 * k1)
23     return y + k2
24
25
26 def passo_rk4(f, t, y, h):
27     """Runge-Kutta classico del quarto ordine."""
28     k1 = h * f(t, y)
29     k2 = h * f(t + 0.5 * h, y + 0.5 * k1)
30     k3 = h * f(t + 0.5 * h, y + 0.5 * k2)
31     k4 = h * f(t + h, y + k3)
32     return y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
```

monografia1/ode_solvers.py

Listato 8.3: Lettura guidata 1 – i passi elementari di `ode_solvers.py` (Modulo 1): funzioni con la stessa firma, scelte per nome dal dizionario `METODI`.

```
49 METODI = {"eulero": passo_eulero, "rk2": passo_rk2, "rk4": passo_rk4,
50           "heun": passo_heun, "ralston": passo_ralston}
51
52
53 # ---------------------------------------------------------------------
54 # Marcia nel tempo a passo costante
55 # ---------------------------------------------------------------------
56 def integra(f, t0, y0, h, n_passi, metodo="rk4", stop=None):
57     """
58     Integra dy/dt = f(t, y) da t0 con passo h per n_passi passi.
59
60     f       : funzione f(t, y) -> dy/dt (stessa forma di y)
61     stop    : funzione opzionale stop(t, y) -> bool; se vera la marcia
62               si arresta (es. proiettile che tocca il suolo)
63     Restituisce gli array t e y (y ha forma (n+1, len(y0))).
64     """
65     passo = METODI[metodo]
66     y = np.atleast_1d(np.asarray(y0, dtype=float))
67     t = float(t0)
```

monografia1/ode_solvers.py

```
    T, Y = [t], [y.copy()]
    for _ in range(n_passi):
        y = passo(f, t, y, h)
        t = t + h
        T.append(t)
        Y.append(y.copy())
        if stop is not None and stop(t, y):
            break
    return np.array(T), np.array(Y)
```

Listato 8.4: Lettura guidata 1 – la marcia nel tempo generica `integra` di `ode_solvers.py`: il metodo è un argomento, l'arresto una funzione opzionale.

## 8.3 Lettura guidata 2: il problema di Rayleigh (Modulo 3)

Il Programma 3.4 integra la diffusione della quantità di moto fra due lastre, una delle quali messa in moto impulsivamente, con lo schema esplicito, e confronta con la serie esatta. È un programma *senza funzioni* oltre a `esatta`: tutto il calcolo è a livello di modulo, nello stile dei programmi brevi del corso; lo si legga come un unico flusso.

monografia3/prog3_4_rayleigh.py

```
M = 21
D = 1.0
H = D / (M - 1)
NU = 1.0e-6
R = 0.25
TAU = R * H**2 / NU
NPRINT, NPLOT = 20, 40
TMAX = 2.5e5
U0 = 1.0

y = np.linspace(0.0, D, M)


def esatta(y, t, nmax=200):
    u = y / D
    for n in range(1, nmax + 1):
        u = u + 2 * (-1) ** n / (n * np.pi) * np.sin(n * np.pi * y / D) *
    np.exp(-(n * np.pi) ** 2 * NU * t / D**2)
    return U0 * u


u = np.zeros(M)
u[-1] = U0
t, j = 0.0, 0
profili = []
print(f"tau = R h^2 / nu = {TAU:.0f} s")
print("   tempo (s)                velocita' (m/s) a y = 0; 0,1; ...; 1")
print(f"  {t:9.3e}  " + " ".join(f"{v:.4f}" for v in u[::2]))
while t < TMAX - 1e-9:
    j += 1
    t += TAU
    u[1:-1] = u[1:-1] + R * (u[:-2] - 2 * u[1:-1] + u[2:])     # tutto il vettore
    al livello j
    if j % NPRINT == 0:
        print(f"  {t:9.3e}  " + " ".join(f"{v:.4f}" for v in u[::2]))
    if j % NPLOT == 0 and len(profili) < 5:
        profili.append((t, u.copy()))

```

```
print("\nErrore massimo rispetto alla serie esatta:")
for tp, up in profili:
    print(f"   t = {tp:8.0f} s   max|u - u_es| = {np.max(np.abs(up - esatta(y, tp))):.2e} m/s")
print(f"   t = {t:8.0f} s   max|u - u_es| = {np.max(np.abs(u - esatta(y, t))):.2e} m/s;"
      f"  scarto dal profilo di Couette = {np.max(np.abs(u - y)):.3f} m/s")
```

Listato 8.5: Lettura guidata 2 – il Programma 3.4 (Modulo 3): costanti, soluzione esatta in serie, marcia nel tempo con lo slicing, campionamento dei profili e verifica.

**Che cosa c'è in queste quaranta righe**

Le costanti derivate l'una dall'altra (`H = D/(M-1), TAU = R*H**2/NU`: il passo temporale scelto dal numero di diffusione, non viceversa). La soluzione esatta come *somma di una serie* con un ciclo `for` che accumula su un array (32–35): `u` è un array, e ogni termine è vettoriale. La condizione iniziale e al contorno in due righe (38–39): `u[-1] = U0` è la lastra superiore in moto. Il ciclo `while` con tolleranza (45) e, dentro, *la* riga dello schema (48), identica a quella del Programma 0.8: `u[1:-1] = u[1:-1] + R*(u[:-2] - 2*u[1:-1] + u[2:])`. L'operatore *modulo* `j % NPRINT == 0` per stampare ogni tanto (49), e la stampa di un vettore con `" ".join(f"{v:.4f}" for v in u[::2])` (50): uno su due, formattati e uniti con uno spazio. La *copia* `u.copy()` quando si conserva un profilo (52): senza, tutti i profili salvati sarebbero lo stesso array, uguale al finale (Cap. 3). La verifica (54–58) con `np.max(np.abs(up - esatta(y, tp)))`, l'errore massimo che compare in ogni programma del corso.

## 8.4 Lettura guidata 3: la cavità MAC (Modulo 4)

Il Programma 4.5 è fra i più complessi del corso: risolve le equazioni di Navier–Stokes in variabili primitive su griglia sfalsata con il metodo a passi frazionari. Non si legga tutto in una volta: si identifichino prima le parti, poi si entri in una sola.

```
def matrice_pressione(N, h):
    """Laplaciano a cinque punti sulle celle di pressione con dp/dn = 0 (nodi fittizi
    uguali a quelli interni); la prima cella e' fissata a p = 0."""
    n = N * N
    idx = lambda i, j: i * N + j
    A = sp.lil_matrix((n, n))
    for i in range(N):
        for j in range(N):
            p = idx(i, j)
            diag = 0.0
            for (a, b) in ((i + 1, j), (i - 1, j), (i, j + 1), (i, j - 1)):
                if 0 <= a < N and 0 <= b < N:
                    A[p, idx(a, b)] = 1.0 / h**2
                    diag -= 1.0 / h**2
            A[p, p] = diag
    A[0, :] = 0.0; A[0, 0] = 1.0                    # livello di riferimento della pressione
    return spla.splu(A.tocsc())


def cavita(Re, N, t_max, tol=1e-6, u_cop=1.0, cfl=0.4, stampa=True, sforzo=None, u_max=1.0):
```

monografia4/prog4_5_cavita_mac.py

```
59      """Griglia MAC con N celle per lato. u: (N+1) x (N+2) (colonne = nodi u in x,
        righe
60      fittizie j = 0 e j = N+1); v: (N+2) x (N+1); p: N x N.
61      Se 'sforzo' non e' None, sulla superficie superiore si impone du/dy = sforzo
        (superficie
62      libera mossa dal vento) invece della velocita' u_cop; u_max serve per il passo
        temporale."""
63      h = 1.0 / N
64      u = np.zeros((N + 1, N + 2)); v = np.zeros((N + 2, N + 1)); p = np.zeros((N, N))
65      LU = matrice_pressione(N, h)
66      dt = min(cfl * 0.25 * Re * h * h, cfl * 0.5 * h / max(u_max, 1e-9))
67      t, passo, res = 0.0, 0, np.inf
68      storia = []
69      t0 = time.perf_counter()
70      while t < t_max and res > tol:
71          # nodi fittizi: aderenza sulle pareti orizzontali (per u) e verticali (per
        v)
72          u[:, 0] = -u[:, 1]                                  # fondo fermo
73          u[:, N + 1] = (2 * u_cop - u[:, N]) if sforzo is None else (u[:, N] + h *
        sforzo)    # coperchio mobile / sforzo imposto
74          v[0, :] = -v[1, :]; v[N + 1, :] = -v[N, :]    # pareti laterali ferme
```

Listato 8.6: Lettura guidata 3 – il Programma 4.5 (Modulo 4): la matrice sparsa della pressione fattorizzata una volta e l'inizio della funzione `cavita` con le condizioni al contorno sui nodi fittizi.

**Prima parte**

`matrice_pressione` (39–55) costruisce il laplaciano sulle celle di pressione: una *lambda* `idx` traduce la coppia $(i, j)$ nell'indice lineare $iN + j$ (Cap. 4, `ravel`/`reshape`); un doppio ciclo riempie una `lil_matrix` (comoda da scrivere elemento per elemento) con il ciclo `for (a, b) in ((i+1, j), ...)` sui quattro vicini e il controllo `0 <= a < N` che esclude quelli fuori dal dominio (la condizione di Neumann $\partial p/\partial n = 0$ nasce da qui: il vicino mancante non contribuisce né al fuori-diagonale né alla diagonale); la riga 54 fissa $p = 0$ nella prima cella per togliere la singolarità; la riga 55 restituisce *la fattorizzazione* `splu(A.tocsc())`, non la matrice: la funzione viene chiamata una volta (65) e `LU.solve` a ogni passo (97). La firma di `cavita` (58) ha molti argomenti con valori predefiniti e una docstring che dichiara le *forme* degli array (59–62): `u` è $(N+1) \times (N+2)$ perché ha una colonna di nodi fittizi sopra e sotto; chi legge deve tenere a mente queste forme per capire lo slicing che segue. Il passo temporale (66) è il minimo fra i due vincoli di stabilità (diffusivo e convettivo) ridotti con `cfl`. I nodi fittizi (72–74) realizzano l'aderenza per riflessione: `u[:, 0] = -u[:, 1]` significa $u_{-1/2} = -u_{1/2}$, media nulla sulla parete.

```
75          # --- predizione (Eulero esplicito, forma conservativa)
76          # u-momento nei nodi u interni i = 1..N-1 (x), j = 1..N (y)
77          ue = 0.5 * (u[1:, 1:-1] + u[:-1, 1:-1])           # u sui centri cella (N, N)
78          duudx = (ue[1:, :]**2 - ue[:-1, :]**2) / h        # (N-1, N)
79          uc = 0.5 * (u[1:-1, 1:] + u[1:-1, :-1])           # u sui vertici (N-1, N+1)
80          vc = 0.5 * (v[1:-1, :][1:, :] + v[1:-1, :][:-1, :])    # v sui vertici (N-1,
        N+1)
81          duvdy = (uc[:, 1:] * vc[:, 1:] - uc[:, :-1] * vc[:, :-1]) / h
82          lapu = (u[2:, 1:-1] - 2 * u[1:-1, 1:-1] + u[:-2, 1:-1] + u[1:-1, 2:] - 2 *
        u[1:-1, 1:-1] + u[1:-1, :-2]) / h**2
83          us = u.copy()
84          us[1:-1, 1:-1] = u[1:-1, 1:-1] + dt * (-duudx - duvdy + lapu / Re)
```

monografia4/prog4_5_cavita_mac.py

```
85          # v-momento nei nodi v interni i = 1..N (x), j = 1..N-1 (y)
86          ve = 0.5 * (v[1:-1, 1:] + v[1:-1, :-1])          # v sui centri cella (N, N)
87          dvvdy = (ve[:, 1:]**2 - ve[:, :-1]**2) / h       # (N, N-1)
88          uc2 = 0.5 * (u[:, 1:-1][:, 1:] + u[:, 1:-1][:, :-1])  # u sui vertici (N+1,
    N-1)
89          vc2 = 0.5 * (v[1:, 1:-1] + v[:-1, 1:-1])         # v sui vertici (N+1, N-1)
90          duvdx = (uc2[1:, :] * vc2[1:, :] - uc2[:-1, :] * vc2[:-1, :]) / h
91          lapv = (v[2:, 1:-1] - 2 * v[1:-1, 1:-1] + v[:-2, 1:-1] + v[1:-1, 2:] - 2 *
    v[1:-1, 1:-1] + v[1:-1, :-2]) / h**2
92          vs = v.copy()
93          vs[1:-1, 1:-1] = v[1:-1, 1:-1] + dt * (-duvdx - dvvdy + lapv / Re)
94          # --- pressione
95          div = (us[1:, 1:-1] - us[:-1, 1:-1]) / h + (vs[1:-1, 1:] - vs[1:-1, :-1]) /
    h
96          rhs = (div / dt).ravel(); rhs[0] = 0.0
97          p = LU.solve(rhs).reshape(N, N)
98          # --- correzione
99          un = us.copy(); vn = vs.copy()
100         un[1:-1, 1:-1] -= dt * (p[1:, :] - p[:-1, :]) / h
101         vn[1:-1, 1:-1] -= dt * (p[:, 1:] - p[:, :-1]) / h
102         res = max(np.max(np.abs(un - u)), np.max(np.abs(vn - v))) / dt
103         u, v = un, vn
104         t += dt; passo += 1
```

Listato 8.7: Lettura guidata 3 – il passo frazionario: predizione con lo slicing su griglia sfalsata, soluzione della pressione, correzione, residuo.

**Seconda parte**

Le righe 77–84 sono la traduzione, termine per termine, dell'equazione della quantità di moto in $x$ in forma conservativa. Ogni termine è calcolato *nel nodo giusto* della griglia sfalsata per media di due vicini: `ue = 0.5*(u[1:, 1:-1] + u[:-1, 1:-1])` porta $u$ sui centri delle celle, di forma $(N, N)$, come il commento dichiara; `duudx` è la differenza di `ue**2` fra centri adiacenti, di forma $(N-1, N)$, cioè i nodi $u$ interni. Il consiglio di lettura: *seguire le forme*, scritte nei commenti; se le forme delle sotto-espressioni tornano, l'operazione è quella giusta. Il laplaciano (82) è la riga a cinque punti del Cap. 4. `us = u.copy()` (83) crea il campo predetto conservando i nodi fittizi, e la predizione scrive solo nei nodi interni (84). Lo stesso schema per $v$ (86–93). La divergenza di `us` (95), il secondo membro appiattito con `ravel` e con il primo elemento azzerato in coerenza con la riga 54 (96), la soluzione con la fattorizzazione e il `reshape` (97). La correzione (99–101) sottrae il gradiente della pressione: `p[1:, :] - p[:-1, :]` è la differenza fra celle adiacenti in $x$, che vive proprio sui nodi $u$ interni. Il residuo (102) è la norma del massimo della variazione per unità di tempo, il criterio di stazionarietà; lo scambio `u, v = un, vn` (103) chiude il passo; `if passo % 200 == 0` campiona la storia. La seconda metà della funzione (107–124) calcola le quantità derivate ($\psi$ per integrazione di $u$ lungo $y$ con un ciclo sulle colonne, $\omega$ con lo slicing), stampa il confronto con i dati di riferimento con un dizionario indicizzato dal numero di Reynolds, e restituisce `dict(...)`: un dizionario di risultati con un nome ciascuno, che il blocco principale interroga con `r["psi"], r["u"][N//2, :]`.

## 8.5 Approfondimenti

### 8.5.1 Stile e leggibilità: PEP 8

La convenzione di stile di Python (PEP 8 [6]) è seguita dai programmi del corso con poche eccezioni consapevoli: quattro spazi di indentazione; nomi in `minuscolo_con_trattini` per variabili e funzioni, `MAIUSCOLO` per le costanti; spazi intorno agli operatori e dopo le virgole; righe non più lunghe di 79 caratteri (nel corso fino a 120, per tenere una formula su una riga); un'istruzione per riga (eccezione: i quattro stadi di RK4 scritti in parallelo, dove la disposizione a tabella aiuta a confrontare); commenti che dicono *perché*, non *che cosa* (`# evita la divisione per zero nell'origine`, non `# calcola r2`). I nomi delle variabili sono quelli delle formule (`psi`, `om`, `Re`, `dt`, `h`) perché il lettore ha le formule sotto gli occhi: la corrispondenza uno a uno fra equazione e riga di codice vale più di nomi descrittivi lunghi. Lo strumento `flake8` o `ruff` segnala le deviazioni; `black` riformatta automaticamente.

### 8.5.2 Moduli comuni e riuso

Quando una funzione serve a più programmi, va in un modulo. Nel corso i moduli comuni sono tre: `metodi_numerici.py` (Thomas, RK4, Liebmann, SOR, Chebyshev e numeri d'onda), `stile_grafici.py` e, nel Modulo 1, `ode_solvers.py`. Il modulo si importa se è nella stessa cartella del programma o in una cartella elencata in `sys.path` (gli esercizi del Modulo 0 aggiungono `codice/letture` con `sys.path.insert`). Un programma importabile deve avere il blocco `__main__`; un modulo deve avere una docstring che ne elenca le funzioni e, idealmente, un blocco `__main__` con i propri test (il Programma 0.9 li fa per `metodi_numerici`). Il passo successivo, non necessario nel corso, è un *pacchetto* (una cartella con `__init__.py`) e un file di test per `pytest`.

### 8.5.3 Argomenti da riga di comando

`sys.argv` è la lista degli argomenti con cui il programma è stato lanciato: `sys.argv[0]` è il nome del file, `sys.argv[1]` il primo argomento, come stringa (da convertire con `float`). Il Programma 0.8 lo usa per $R$, `esercizi_svolti.py` per scegliere quali esercizi eseguire. Per interfacce più ricche (opzioni con nome, aiuto automatico) la libreria standard offre `argparse`; per i programmi del corso `sys.argv` con un valore predefinito è sufficiente e leggibile.

# Capitolo 9

# Errori, verifiche, file e tempi

## 9.1 Gli errori che il calcolatore segnala

Quando Python incontra un'operazione impossibile solleva un'**eccezione**, stampa il **traceback** (la catena delle chiamate, dall'esterno verso l'interno, fino alla riga responsabile) e si ferma. Il traceback si legge *dal basso*: l'ultima riga dice il tipo di errore e il messaggio, quella sopra il file e la riga. Le eccezioni che chi scrive codice numerico incontra più spesso:

| Eccezione | Causa tipica |
|---|---|
| `IndexError` | indice fuori intervallo: `u[n]` con `len(u) == n`, un ciclo `range(n+1)` su un array di `n` |
| `ValueError` | forme incompatibili nel broadcasting; `float("dodici")`; argomento fuori dominio |
| `ZeroDivisionError` | divisione per zero fra float o interi di Python (NumPy invece dà `inf`/`nan`) |
| `NameError` | variabile mai assegnata, spesso un errore di battitura o una variabile locale usata fuori |
| `TypeError` | tipi incompatibili (`"s" + 1`; `math.sin(array)`; funzione chiamata con argomenti sbagliati) |
| `KeyError` | chiave assente in un dizionario |
| `AttributeError` | metodo inesistente: `lista.shape`, `array.append` |
| `RecursionError` | ricorsione senza caso base o troppo profonda |
| `AssertionError` | un `assert` falso: la verifica è fallita |

**`try`/`except` e `raise`**

```
try:
    valore = float(testo)
except ValueError:
    print(f"'{testo}' non e' un numero")

def velocita_limite(m, rho, A, cd):
    if min(m, rho, A, cd) <= 0:
        raise ValueError(f"tutti i parametri devono essere positivi: m={m},
    rho={rho}, A={A}, cd={cd}")
    return math.sqrt(2 * m * G / (rho * A * cd))
```

`try`/`except` intercetta un errore previsto (un file che manca, un dato non numerico) e permette di gestirlo; `raise` solleva un errore nelle proprie funzioni quando gli argomenti

sono insensati, con un messaggio che dica *quali*. Nei programmi del corso la regola è: *non* intercettare gli errori imprevisti (un `except:` generico nasconde i bug), ma verificare gli argomenti all'ingresso e fallire presto con un messaggio chiaro; `sor_poisson` e `sorlx` sollevano `RuntimeError` se non convergono entro il numero massimo di iterazioni.

**NumPy non si ferma**

`1/0` fra float di Python solleva `ZeroDivisionError`; `np.float64(1)/0` restituisce `inf` con un avviso, `0/0` dà `nan`, `np.exp(1000)` dà `inf`, e il calcolo *prosegue* con quei valori, che si propagano in tutto l'array (ogni operazione con `nan` dà `nan`). Uno schema instabile non produce un errore: produce `inf` e poi `nan` silenziosamente. Il controllo `np.isfinite(u).all()` dentro il ciclo temporale (o alla fine) è l'unico modo di accorgersene, e `np.errstate(over="raise")` trasforma l'avviso in un'eccezione quando si vuole essere severi.

## 9.2 Gli errori che il calcolatore non segnala

Sono i più pericolosi: il programma termina, stampa numeri plausibili, e sono sbagliati. Il Programma 0.9 ne raccoglie otto, tutti incontrati scrivendo le monografie.

**Otto errori silenziosi**

(a) **Scostamento di uno**: `h = 1/len(x)` con `x = np.linspace(0, 1, 11)` dà 1/11 invece di 1/10: gli $n+1$ nodi definiscono $n$ intervalli. Sempre `h = x[1] - x[0]`.

(b) **Accumulo del tempo**: `while t < 1.0: t += 0.1` esegue 11 passi, perché dopo dieci $t = 0{,}9999999999999999$. Numero di passi fissato, o tolleranza.

(c) **Uguaglianza fra float**: `0.1*3 == 0.3` è falso. `abs(a - b) < tol`, `math.isclose`, `np.isclose`, `np.allclose`.

(d) **Vista scambiata per copia**: `u_vecchio = u` e poi `u[0] = 1` modifica anche `u_vecchio`. `u.copy()`.

(e) **Argomento predefinito mutabile**: `def f(x, lista=[])` crea la lista *una volta*, alla definizione, e le chiamate successive la ritrovano piena. `lista=None` e `lista = [] if lista is None else lista`.

(f) **Divisione fra interi**: `N/2` è un float e non indicizza; `N//2`.

(g) **Aggiornamento sul posto non voluto**: la stessa formula scritta vettorializzata, `u[1:-1] = 0.5*(u[2:] + u[:-2])`, usa i valori *vecchi* (Jacobi); scritta in un ciclo, `u[i] = 0.5*(u[i+1] + u[i-1])`, usa quelli *già aggiornati* (Gauss–Seidel). Sono due algoritmi diversi, con velocità di convergenza diverse (Modulo 2); vettorializzare un ciclo può cambiare il metodo.

(h) **Overflow silenzioso**: lo schema FTCS con $R = 0{,}6$ dopo 3000 passi ha `max|v| = nan` senza alcun errore. `np.isfinite`.

## 9.3 Verifiche automatiche con `assert`

`assert condizione, "messaggio"` non fa nulla se la condizione è vera e interrompe il programma con `AssertionError` e il messaggio se è falsa. È il test più semplice che esista, e i programmi del corso lo usano dopo ogni verifica contro un valore noto: `assert abs(p - 22632) < 1` nell'atmosfera standard, `assert all(12 < r < 20 for r in rapporti)` per l'ordine di RK4. Il Programma 0.9 mette alla prova il modulo `metodi_numerici.py` in tre righe ciascuno: l'algoritmo di Thomas contro `np.linalg.solve` (differenza $4 \times 10^{-16}$), RK4 contro $e^{-2t}$ (rapporti degli errori 17,4 e 16,7: ordine 4), SOR contro la soluzione esatta di Poisson (129 iterazioni con $\omega$ ottimo, errore $8 \times 10^{-4} = \mathcal{O}(h^2)$). Quando si modifica un modulo, rieseguire i suoi test dice subito se si è rotto qualcosa: è l'idea del *test di regressione*, che `pytest` automatizza raccogliendo tutte le funzioni `test_*` di un file.

**La verifica di un programma, in ordine di forza crescente**

1. Il programma gira senza eccezioni e i risultati sono finiti (`np.isfinite`).
2. I risultati hanno il segno, l'ordine di grandezza e le simmetrie attese (la cavità ha $\psi < 0$, $\max|\psi|$ sulla parete superiore è $\sim 10^{-16}$, la divergenza è nulla).
3. Il confronto con una **soluzione esatta** (diffusione, Poiseuille, Rayleigh, Blasius, Stokes) o con un **valore di riferimento** pubblicato (Ghia et al.) dà un errore piccolo.
4. L'errore **decresce con l'ordine atteso** al raffinare la griglia: è la verifica più severa, perché un bug di consistenza (un coefficiente sbagliato, un nodo sbagliato) abbassa l'ordine anche quando l'errore sembra piccolo. Il Programma 0.2 ne è la dimostrazione.
5. Gli `assert` che codificano i punti 1–4 restano nel programma e vengono rieseguiti a ogni modifica.

## 9.4 File

**Leggere e scrivere**

```
np.savetxt("profilo.txt", np.column_stack([x, u]), fmt="%.6f", header="x
    u")   # testo, leggibile
tab = np.loadtxt("profilo.txt")                        # salta le righe che
    iniziano con #
np.save("campo.npy", u); u = np.load("campo.npy")     # binario, esatto,
    veloce: per i campi 2D
with open("parametri.json", "w", encoding="utf-8") as fp:
    json.dump({"Re": 100, "griglia": [65, 65]}, fp, indent=2)      #
    dizionario -> file JSON
with open("rapporto.txt", "w", encoding="utf-8") as fp:
    fp.write(f"Re = {Re}\n")                                        # testo
    libero
qui = os.path.dirname(os.path.abspath(__file__))      # cartella del programma
percorso = os.path.join(qui, "..", "figure", "nome.pdf")
```

`with open(...) as fp`: apre il file e lo *chiude automaticamente* alla fine del blocco, anche in caso di errore: è l'unica forma da usare. `np.savetxt/np.loadtxt` per le tabelle da leggere con altri programmi (fogli di calcolo, gnuplot), `np.save/np.load`

per salvare esattamente un campo e rileggerlo (una soluzione di riferimento calcolata una volta su griglia fine); JSON per i parametri di un calcolo, che così restano documentati accanto ai risultati. I percorsi si costruiscono con `os.path.join` relativi alla cartella del programma (`__file__`), non alla cartella corrente, così il programma funziona da qualunque posizione venga lanciato: è il meccanismo di `salva` in `stile_grafici.py`. Un file di coordinate di un profilo (Esercizio 7) si legge in tre righe con `np.loadtxt`; con `skiprows` e `usecols` si saltano intestazioni e colonne.

## 9.5 Tempi e profilo

`time.perf_counter()` è il cronometro ad alta risoluzione: si legge prima e dopo, e la differenza è il tempo trascorso in secondi. La prima esecuzione di una funzione può essere più lenta (caricamento di librerie, allocazioni): si cronometra la seconda, o si fa una chiamata "di riscaldamento" come nel Programma 0.6. `cProfile` esegue il programma registrando quanto tempo passa in ogni funzione e quante volte è chiamata: `python3 -m cProfile -s cumulative programma.py` o, dall'interno, `cProfile.Profile()` con `pstats`. Nel Programma 0.9 il profilo attribuisce a `laplaciano_lento` un tempo venti o trenta volte maggiore che a `laplaciano_veloce`: il collo di bottiglia è uno, ed è il doppio ciclo. La regola: misurare *prima* di ottimizzare, perché l'intuizione su dove va il tempo è spesso sbagliata, e ottimizzare solo ciò che il profilo indica.

## 9.6 Il Programma 0.9

prog0_9_verifiche.py

```python
"""
prog0_9_verifiche.py -- Programma 0.9: errori frequenti, eccezioni, verifiche e
    file.

1) Errori che il calcolatore segnala (eccezioni): come leggere un traceback,
   try/except, raise; gli errori piu' comuni nei codici di calcolo (indici fuori
   intervallo, forme incompatibili, divisione per zero, nomi non definiti).
2) Errori che il calcolatore NON segnala: lo scostamento di un passo (off by one),
   l'accumulo del tempo in un ciclo while, l'uguaglianza fra float, le viste
   NumPy modificate per sbaglio, gli argomenti predefiniti mutabili, la divisione
   intera, il ciclo che non copia (u_vecchio = u), l'overflow silenzioso (inf, nan).
3) Verifiche automatiche con assert: i metodi del modulo metodi_numerici.py
   (Thomas contro numpy.linalg.solve, RK4 del quarto ordine, SOR contro la
   soluzione esatta) messi alla prova in poche righe.
4) File: salvare e rileggere array (np.savetxt/np.loadtxt, np.save), parametri
   (json), risultati in testo; i percorsi con os.path.
5) Misurare il tempo (time.perf_counter) e trovare il collo di bottiglia (cProfile).
"""
import os
import json
import time
import cProfile
import pstats
import io
import numpy as np
from metodi_numerici import thomas, rk4_integra, sor_poisson, omega_ottimo

QUI = os.path.dirname(os.path.abspath(__file__))

```

```python
# ------------------------------------------------------------------ 1) eccezioni
print("1) Eccezioni: errori segnalati")
prove = [
    ("indice fuori intervallo", lambda: np.zeros(5)[5]),
    ("forme incompatibili", lambda: np.zeros(3) + np.zeros(4)),
    ("divisione per zero (float Python)", lambda: 1.0 / 0.0),
    ("nome non definito", lambda: velocita_inesistente + 1),
    ("tipo sbagliato", lambda: "2" + 2),
    ("chiave assente", lambda: {"aria": 1.2}["elio"]),
    ("conversione impossibile", lambda: float("dodici")),
]
for nome, azione in prove:
    try:
        azione()
    except Exception as e:                          # 'Exception' cattura tutto; in pratica si cattura il tipo specifico
        print(f"   {nome:34s} -> {type(e).__name__}: {e}")
with np.errstate(divide="ignore", invalid="ignore", over="ignore"):
    print(f"   NumPy invece NON si ferma: 1/0 = {np.float64(1.0) / 0.0}, 0/0 = {np.float64(0.0) / 0.0}, exp(1000) = {np.exp(1000.0)}"
          f"  (da controllare con np.isfinite)")


def velocita_limite(m, rho, A, cd):
    """Esempio di controllo esplicito degli argomenti con raise."""
    if m <= 0 or rho <= 0 or A <= 0 or cd <= 0:
        raise ValueError(f"tutti i parametri devono essere positivi: m={m}, rho={rho}, A={A}, cd={cd}")
    return np.sqrt(2 * m * 9.80665 / (rho * A * cd))


try:
    velocita_limite(1.0, 1.225, -0.01, 0.5)
except ValueError as e:
    print(f"   raise in una funzione propria            -> ValueError: {e}")

# ------------------------------------------------------------------ 2) errori silenziosi
print("\n2) Errori silenziosi (nessun messaggio, risultato sbagliato)")
# a) off by one
x = np.linspace(0, 1, 11)                           # 11 nodi, 10 intervalli: h = 0,1
h_sbagliato = 1.0 / len(x)                          # 1/11 !
print(f"   a) passo h: corretto {x[1] - x[0]:.4f} = 1/(len(x)-1), sbagliato 1/len(x) = {h_sbagliato:.4f}")
# b) accumulo del tempo
t, n, t10 = 0.0, 0, None
while t < 1.0:
    t += 0.1; n += 1
    if n == 10:
        t10 = t
print(f"   b) while t < 1 con t += 0.1: {n} passi (attesi 10) perche' dopo 10 passi t = {t10!r} < 1")
# c) uguaglianza fra float
print(f"   c) 0.1*3 == 0.3 e' {0.1 * 3 == 0.3}; usare abs(a - b) < tol oppure np.isclose: {np.isclose(0.1 * 3, 0.3)}")
# d) viste
u = np.zeros(5); u_vecchio = u                      # NON e' una copia
u[0] = 1.0
print(f"   d) u_vecchio = u, poi u[0] = 1: u_vecchio = {u_vecchio} (stesso oggetto); serve u.copy()")
```

```
# e) argomento predefinito mutabile
def aggiungi(valore, lista=[]):                    # la lista e' creata UNA volta sola
    lista.append(valore); return lista
print(f"   e) argomento predefinito mutabile: aggiungi(1) -> {aggiungi(1)}, aggiungi(2) -> {aggiungi(2)} (la lista persiste!)")
def aggiungi_bene(valore, lista=None):
    lista = [] if lista is None else lista
    lista.append(valore); return lista
print(f"      forma corretta: aggiungi_bene(1) -> {aggiungi_bene(1)}, aggiungi_bene(2) -> {aggiungi_bene(2)}")
# f) interi e float
N = 100
print(f"   f) indice del nodo centrale: N/2 = {N / 2} (float, non valido come indice), N//2 = {N // 2}")
# g) il ciclo che aggiorna 'sul posto' quando non dovrebbe (Gauss-Seidel invece di Jacobi)
u = np.array([1.0, 0.0, 0.0, 0.0, 0.0])                                    # contorno sinistro = 1
uj = u.copy(); uj[1:-1] = 0.5 * (u[2:] + u[:-2])                           # Jacobi: usa i valori vecchi
ugs = u.copy()
for i in range(1, 4):
    ugs[i] = 0.5 * (ugs[i + 1] + ugs[i - 1])                               # sul posto: usa gia' i valori nuovi
print(f"   g) stessa formula, risultati diversi: vettorializzata (Jacobi) {uj}, sul posto (Gauss-Seidel) {ugs}")
# h) overflow silenzioso in un ciclo instabile
R = 0.6; v = np.sin(np.pi * np.linspace(0, 1, 21))
with np.errstate(over="ignore", invalid="ignore"):
    for k in range(3000):
        v[1:-1] = v[1:-1] + R * (v[2:] - 2 * v[1:-1] + v[:-2])
print(f"   h) schema instabile dopo 3000 passi: max|v| = {np.max(np.abs(v))} -> controllare np.isfinite(v).all() = {np.isfinite(v).all()}")

# ------------------------------------------------------------------ 3) verifiche con assert
print("\n3) Verifiche automatiche del modulo metodi_numerici.py")
n = 50
a = -np.ones(n); b = 2.5 * np.ones(n); c = -np.ones(n); f = np.random.default_rng(0).standard_normal(n)
A = np.diag(b) + np.diag(a[1:], -1) + np.diag(c[:-1], 1)
x_th = thomas(a, b, c, f)
assert np.allclose(x_th, np.linalg.solve(A, f)), "Thomas non coincide con numpy.linalg.solve"
print(f"   Thomas contro solve: differenza {np.max(np.abs(x_th - np.linalg.solve(A, f))):.1e}  [ok]")
fy = lambda t, y: -2 * y
errs = [abs(rk4_integra(fy, np.array([1.0]), 1.0, h)[1][-1, 0] - np.exp(-2)) for h in (0.1, 0.05, 0.025)]
rapp = errs[0] / errs[1], errs[1] / errs[2]
assert all(12 < r < 20 for r in rapp), f"RK4 non e' del quarto ordine: rapporti {rapp}"
print(f"   RK4: errori {errs[0]:.2e}, {errs[1]:.2e}, {errs[2]:.2e}, rapporti {rapp[0]:.1f}, {rapp[1]:.1f}  [ok]")
m = 33; xg = np.linspace(0, 1, m); X, Y = np.meshgrid(xg, xg, indexing="ij")
u_es = np.sin(np.pi * X) * np.sin(np.pi * Y)
nodi = [(i, j) for i in range(1, m - 1) for j in range(1, m - 1)]
u_sor, it = sor_poisson(np.zeros((m, m)), -2 * np.pi**2 * u_es, xg[1] - xg[0], omega_ottimo(m, m), 1e-8, nodi)
```

```
assert np.max(np.abs(u_sor - u_es)) < 5e-3, "SOR: errore troppo grande"
print(f"   SOR su Poisson: {it} iterazioni, errore {np.max(np.abs(u_sor - u_es)):.2e} (O(h^2))  [ok]")
print("   (un assert falso interrompe il programma con AssertionError e il messaggio: e' il test piu' semplice)")

# ---------------------------------------------------------------- 4) file
print("\n4) File")
cartella = os.path.join(QUI, "..", "figure")
tab = np.column_stack([xg, u_es[:, m // 2]])
percorso = os.path.join(cartella, "prog0_9_profilo.txt")
np.savetxt(percorso, tab, fmt="%.6f", header="x  u(x, 0.5)", delimiter="  ")
letto = np.loadtxt(percorso)
print(f"   np.savetxt/np.loadtxt: {os.path.basename(percorso)}, forma {letto.shape}, identico? {np.allclose(letto, tab, atol=1e-6)}")
np.save(os.path.join(cartella, "prog0_9_campo.npy"), u_es)
print(f"   np.save/np.load (binario, esatto): {np.array_equal(np.load(os.path.join(cartella, 'prog0_9_campo.npy')), u_es)}")
parametri = {"Re": 100, "griglia": [65, 65], "schema": "FTCS", "tolleranza": 1e-6}
with open(os.path.join(cartella, "prog0_9_parametri.json"), "w", encoding="utf-8") as fp:
    json.dump(parametri, fp, indent=2)
with open(os.path.join(cartella, "prog0_9_parametri.json"), encoding="utf-8") as fp:
    riletti = json.load(fp)
print(f"   json: {riletti} (riletti == parametri: {riletti == parametri})")
with open(os.path.join(cartella, "prog0_9_rapporto.txt"), "w", encoding="utf-8") as fp:
    fp.write("Rapporto di calcolo\n")
    for k, v in riletti.items():
        fp.write(f"{k:12s} = {v}\n")
print("   testo: scritto prog0_9_rapporto.txt con 'with open(...) as fp' (il file e' chiuso automaticamente)")

# ---------------------------------------------------------------- 5) tempi e profilo
print("\n5) Tempi e profilo")


def laplaciano_lento(u, h):
    out = np.zeros_like(u)
    for i in range(1, u.shape[0] - 1):
        for j in range(1, u.shape[1] - 1):
            out[i, j] = (u[i + 1, j] + u[i - 1, j] + u[i, j + 1] + u[i, j - 1] - 4 * u[i, j]) / h**2
    return out


def laplaciano_veloce(u, h):
    out = np.zeros_like(u)
    out[1:-1, 1:-1] = (u[2:, 1:-1] + u[:-2, 1:-1] + u[1:-1, 2:] + u[1:-1, :-2] - 4 * u[1:-1, 1:-1]) / h**2
    return out


def simulazione(n_passi=20):
    for _ in range(n_passi):
        laplaciano_lento(u_es, 1 / 32); laplaciano_veloce(u_es, 1 / 32)


```

```
t0 = time.perf_counter(); simulazione(); print(f"   simulazione di prova: {(time.perf_counter() - t0)*1e3:.1f} ms")
pr = cProfile.Profile(); pr.enable(); simulazione(); pr.disable()
buf = io.StringIO(); pstats.Stats(pr, stream=buf).sort_stats("cumulative").print_stats(4)
righe = [r for r in buf.getvalue().splitlines() if "laplaciano" in r]
print("   cProfile (tempo cumulativo per funzione):")
for r in righe:
    print("   " + r.strip().replace(QUI + os.sep, ""))
print("   il profilo dice dove va il tempo: qui quasi tutto nel laplaciano con i cicli, da vettorializzare")
```

Listato 9.1: Programma 0.9 – eccezioni, errori silenziosi, `assert` sul modulo `metodi_numerici`, file di dati e parametri, cronometro e profilo (`prog0_9_verifiche.py`).

**Risultati del Programma 0.9**

Le otto eccezioni della prima sezione (sette provocate di proposito e un `raise` in una funzione propria) sono prodotte e intercettate una per una con il loro messaggio; NumPy restituisce `inf` e `nan` senza fermarsi. Gli otto errori silenziosi sono riprodotti con il risultato sbagliato accanto a quello giusto. I tre `assert` su `metodi_numerici` passano (Thomas $4 \times 10^{-16}$; RK4 rapporti 17,4 e 16,7; SOR 129 iterazioni, errore $8 \times 10^{-4}$). I quattro file scritti in `figure/` sono riletti identici. Il profilo attribuisce quasi tutto il tempo al laplaciano con i cicli.

## 9.7 Approfondimenti

### 9.7.1 Riproducibilità

Un calcolo è riproducibile se chiunque, con lo stesso programma e gli stessi dati, ottiene gli stessi numeri. Per i programmi del corso questo richiede: i parametri nel programma o in un file JSON accanto ai risultati, non digitati a mano in un terminale; i numeri casuali con un seme fisso (`default_rng(0)`); le versioni delle librerie annotate (`python3 -c "import numpy, scipy, matplotlib; print(numpy.__version__, ...)"`), perché i tempi misurati e, raramente, le ultime cifre dipendono dalla versione; le figure generate dal programma, non ritoccate a mano. I risultati riportati nelle monografie sono quelli stampati dai programmi pubblicati, e il file `esegui_tutti.sh` li rigenera tutti.

### 9.7.2 Debug: come trovare un errore

Quando un risultato è sbagliato e non c'è eccezione: (1) ridurre il problema al più piccolo che mostra l'errore (griglia $5 \times 5$, due passi temporali); (2) stampare le forme e i valori intermedi (`print(u.shape, u.min(), u.max())`) nei punti chiave, o usare il debugger (`breakpoint()` ferma il programma e apre una console nel punto); (3) confrontare con un calcolo a mano sul problema ridotto, che è possibile proprio perché è piccolo; (4) misurare l'ordine di convergenza, che localizza un termine sbagliato; (5) scrivere l'`assert` che avrebbe intercettato l'errore, perché il prossimo non lo rifaccia. L'Esercizio 20 è una caccia all'errore guidata su un programma con tre bug.

### 9.7.3 Quando Python è troppo lento

I programmi del corso girano in secondi o minuti. Se un problema più grande richiede ore, nell'ordine: (1) profilare; (2) vettorializzare i cicli con NumPy (fattore 10–100); (3) usare le matrici sparse e i risolutori diretti con fattorizzazione riusata (Cap. 6); (4) compilare i cicli che non si vettorializzano (Gauss–Seidel, Thomas) con `numba` (`@njit` davanti alla funzione: fattore 50–200, senza cambiare il codice); (5) solo allora, un linguaggio compilato (Fortran, C++, Julia) o il calcolo parallelo. Con (2)–(4) un codice Python arriva entro un fattore 2–3 da uno compilato, mantenendo la leggibilità che è lo scopo del corso.

### 9.7.4 Archivio dei programmi del corso

Tutti i programmi delle cinque monografie sono pubblicati in rete all'indirizzo indicato dai QR nei margini, in cartelle `monografiaN/codici/`; ogni cartella contiene i programmi, `stile_grafici.py`, `metodi_numerici.py` dove serve, gli esercizi svolti e lo script `esegui_tutti.sh` che li esegue in sequenza e rigenera le figure. Per eseguire un programma: scaricare la cartella, aprire un terminale al suo interno, `python3 nome.py`. I pacchetti richiesti sono `numpy`, `scipy`, `matplotlib`, `sympy`: `pip install numpy scipy matplotlib sympy`.

# Appendice A

# Sintassi essenziale, listati di supporto e istruzioni

## A.1 Sintassi essenziale di riferimento

Le due pagine seguenti raccolgono, in forma di promemoria, i costrutti che i programmi del corso usano. Ogni voce rimanda al capitolo che la spiega.

Tabella A.1: Il linguaggio (Capp. 1–3).

| Costrutto | Significato e note |
|---|---|
| `x = 3, v = 2.5, z = 1+2j, ok = True` | int (illimitato), float (64 bit, $\epsilon = 2{,}2 \times 10^{-16}$), complex, bool |
| `7/2, 7//2, 7%2, 2**10, abs, round` | divisione vera (3.5), intera (3), resto (1), potenza |
| `import math; math.sqrt, math.pi, math.isclose(a, b)` | funzioni scalari; confronto fra float con tolleranza |
| `f"v = {v:8.3f} m/s, {n:5d}, {e:.2e}, {p:.1%}"` | f-string: larghezza.precisione, intero, esponenziale, percentuale |
| `s[0], s[-1], s[2:5], len(s), s.split(), " ".join(seq), s.strip(), float(s)` | stringhe come sequenze; da stringa a numero |
| `if c: ... elif c2: ... else: ...` | decisione; blocchi indentati di 4 spazi |
| `for k in range(a, b, passo):` | $k = a, \ldots, b-1$; `break`, `continue` |
| `for i, x in enumerate(seq):` | indice ed elemento; sequenze accoppiate |
| `for a, b in zip(s1, s2):` | |
| `while cond:` | finché; prevedere un limite di iterazioni |
| `def f(x, h=0.1):` | funzione, argomento predefinito, docstring, più |
| `"""doc"""; ...; return a, b` | valori (tupla) |
| `g = lambda v: f(v, d, m)` | funzione anonima di una riga (adattatore) |
| `global n` | per modificare una variabile globale da una funzione |
| `if __name__ == "__main__":` | codice eseguito solo se il file è lanciato, non importato |
| `import numpy as np; from modulo import f, G` | importazione con alias; importazione di nomi |
| `[1, 2, 3], l.append(x), l[1:-1], l[::-1], x in l, sorted(l, key=f)` | liste: modificabili, slicing, appartenenza, ordinamento |
| `(a, b), a, b = b, a, T, Z = map(np.array, (T, Z))` | tuple e spacchettamento |
| `{"aria": 1.225}, d[k], d.get(k, default), d.items(), dict(Re=Re, psi=psi)` | dizionari |
| `[f(k) for k in seq if cond], {k: v for ...}` | comprehension di lista e di dizionario |
| `b = a.copy()` (non `b = a`) | copia di liste e array; `[[0]*n for _ in range(m)]` |
| `try: ... except ValueError: ..., raise ValueError("msg")` | eccezioni; `assert cond, "msg"` per le verifiche |
| `with open(f, "w", encoding="utf-8") as fp: fp.write(...)` | file, chiuso automaticamente; `for riga in fp:` |
| `sys.argv[1], time.perf_counter(), json.dump/load` | argomenti, cronometro, parametri |

Tabella A.2: NumPy, Matplotlib, SciPy e SymPy (Capp. 4–7).

| Costrutto | Significato e note |
|---|---|
| `np.array([...])`, `np.zeros((m, n))`, `np.ones`, `np.full`, `np.zeros_like(u)` | creazione; dtype float per i campi |
| `np.linspace(a, b, n+1)`, `np.arange(a, b, passo)`, `np.meshgrid(x, y, indexing="ij")` | griglie; `X[i, j] = x[i]` |
| `u.shape`, `u.ndim`, `u.dtype`, `u.T`, `u.ravel()`, `u.reshape(m, n)` | forma e riorganizzazione |
| `u[1:-1]`, `u[2:]`, `u[:-2]`, `A[1:-1, 1:-1]`, `A[:, 0]` | viste: nodi interni, destra, sinistra, blocco, contorno |
| `u[1:-1] = u[1:-1] + R*(u[2:] - 2*u[1:-1] + u[:-2])` | lo schema FTCS in una riga |
| `u[u > 0]`, `np.where(c, a, b)`, `np.roll(u, 1)`, `np.argmax(u)`, `np.unravel_index` | selezione, condizionale, periodicità, posizione del massimo |
| `x[:, None] + y[None, :]`, `T / T.max(axis=0)` | broadcasting |
| `np.sin`, `np.exp`, `np.abs`, `np.maximum`, `np.hypot`; `u.sum()`, `u.max()`, `np.mean`, `np.linalg.norm` | funzioni elemento per elemento; riduzioni (`axis=`) |
| `A @ x`, `np.linalg.solve(A, b)`, `np.linalg.eigh(A)`, `np.linalg.cond(A)` | algebra lineare densa |
| `np.isclose`, `np.allclose`, `np.isfinite(u).all()` | confronti e controllo di `inf`/`nan` |
| `np.savetxt/loadtxt`, `np.save/load`, `np.trapezoid`, `np.polyfit(np.log(h), np.log(e), 1)` | file, integrale, pendenza (ordine di convergenza) |
| `fig, ax = plt.subplots(1, 2, figsize=(7, 3), sharex=True)` | figura e assi |
| `ax.plot(x, u, "o-", color=BLU, lw=1.4, ms=3, mfc="none", label="...")` | linee e marcatori |
| `ax.loglog`, `ax.semilogy`, `ax.set_xlabel("$x/c$")`, `ax.set_xlim`, `ax.legend`, `ax.set_aspect("equal")` | scale, etichette (LaTeX), limiti, legenda |
| `ax.contour(X, Y, psi, levels=...)`, `ax.contourf(...)`, `fig.colorbar(cf, ax=ax)` | isolinee e campi riempiti |
| `ax.quiver(X[::k, ::k], ...)`, `ax.streamplot(x, y, u.T, v.T)` | vettori diradati; linee di corrente (convenzione `xy`) |
| `ax.annotate("t", xy=(x, y), xytext=(x2, y2), arrowprops=...)`, `ax.axhline(y0)` | annotazioni e riferimenti |
| `fig.tight_layout()`; `salva(fig, "nome")` | margini e salvataggio PDF (`stile_grafici`) |
| `optimize.brentq(f, a, b)`, `optimize.fsolve(F, x0)`, | radici e minimi |

## A.2 Nomi ricorrenti nei programmi del corso

Tabella A.3: Convenzioni di nome usate in tutti i programmi delle monografie.

| Nome | Significato |
|---|---|
| `x`, `y`, `X`, `Y` | nodi della griglia (vettori) e griglie bidimensionali da `meshgrid` (`ij`) |
| `h`, `dx`, `dy`, `dt`, `tau` | passi spaziali e temporale |
| `n`, `N`, `m`, `M` | numero di intervalli o di nodi (la docstring precisa quale) |
| `u`, `v`, `p`, `psi`, `om`, `zeta`, `T` | campi: velocità, pressione, funzione di corrente, vorticità, temperatura |
| `R`, `C`, `Re`, `Pr`, `Ra` | numero di diffusione $\nu\Delta t/h^2$, di Courant $u\Delta t/h$, di Reynolds, Prandtl, Rayleigh |
| `tol`, `it_max`, `res`, `err` | tolleranza, iterazioni massime, residuo, errore rispetto a un riferimento |
| `esatta(x, t)`, `passo_*()`, `integra()`, `errore()` | soluzione esatta, un passo dello schema, marcia nel tempo, misura dell'errore |
| `salva(fig, nome)`, `BLU`, `ARANCIO`, `PALETTE` | dal modulo `stile_grafici` |
| `thomas`, `rk4_integra`, `sor_poisson`, `omega_ottimo`, `liebmann` | dal modulo `metodi_numerici` |
| `QUI`, `FIG_DIR` | cartella del programma e delle figure (`os.path`) |

## A.3 Listati di supporto

metodi_numerici.py

```python
"""
metodi_numerici.py -- strumenti comuni della monografia 3 (flussi viscosi).

Contiene:
  * trid(C)              eliminazione di Gauss per sistemi tridiagonali (forma a 4 colonne)
  * thomas(a, b, c, d)   algoritmo di Thomas in memoria compatta
  * rk4_passo(f, x, y, h) un passo di Runge-Kutta del quarto ordine per y' = f(x, y)
  * rk4_integra(...)     integrazione a passo costante con memorizzazione della soluzione
  * liebmann(F, Q, h, errmax)  Gauss-Seidel per lap(F) = Q su griglia quadrata
  * sor_poisson(...)     sovrarilassamento successivo per lap(F) = Q
  * omega_ottimo(m, n)   fattore di sovrarilassamento ottimo per un rettangolo
  * cheb(N)              matrice di differenziazione di Chebyshev e nodi di Gauss-Lobatto
  * numeri_onda(N, L)    numeri d'onda della trasformata discreta di Fourier
"""
import numpy as np


# ---------------------------------------------------------------------
# 1. Sistemi tridiagonali
# ---------------------------------------------------------------------
def trid(C):
    """Risolve il sistema tridiagonale scritto riga per riga come
         C[i,0]*f[i-1] + C[i,1]*f[i] + C[i,2]*f[i+1] = C[i,3],  i = 0..n-1
    (C[0,0] e C[n-1,2] devono essere gia' stati azzerati dalle condizioni al
```

```
    contorno). Eliminazione in avanti per combinazione di righe, poi
    sostituzione all'indietro. Restituisce il vettore f (C non viene alterata).
    """
    C = np.array(C, dtype=float)          # copia di lavoro
    n = C.shape[0]
    for i in range(1, n - 1):             # eliminazione di C[i,0], i = 1..n-2
        c2 = C[i, 1] * C[i - 1, 1] - C[i, 0] * C[i - 1, 2]
        c3 = C[i, 2] * C[i - 1, 1]
        c4 = C[i, 3] * C[i - 1, 1] - C[i, 0] * C[i - 1, 3]
        C[i, 1], C[i, 2], C[i, 3] = c2, c3, c4
    f = np.zeros(n)
    # ultime due equazioni risolte insieme
    f[n - 1] = ((C[n - 1, 3] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 3]) /
                (C[n - 1, 1] * C[n - 2, 1] - C[n - 1, 0] * C[n - 2, 2]))
    for j in range(n - 2, -1, -1):        # sostituzione all'indietro
        f[j] = (C[j, 3] - C[j, 2] * f[j + 1]) / C[j, 1]
    return f


def thomas(a, b, c, d):
    """Algoritmo di Thomas per  a[i] x[i-1] + b[i] x[i] + c[i] x[i+1] = d[i].
    a[0] e c[-1] non vengono usati. Funziona anche con coefficienti complessi."""
    n = len(d)
    tipo = np.result_type(a, b, c, d, float)
    cp = np.zeros(n, dtype=tipo)
    dp = np.zeros(n, dtype=tipo)
    cp[0] = c[0] / b[0]
    dp[0] = d[0] / b[0]
    for i in range(1, n):                 # eliminazione in avanti
        den = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / den if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / den
    x = np.zeros(n, dtype=tipo)
    x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):        # sostituzione all'indietro
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


# ---------------------------------------------------------------------------
# 2. Runge-Kutta del quarto ordine per sistemi  y' = f(x, y)
# ---------------------------------------------------------------------------
def rk4_passo(f, x, y, h):
    """Un passo RK4: restituisce y(x + h)."""
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6.0


def rk4_integra(f, y0, x_fin, h, x0=0.0):
    """Integra y' = f(x, y) da x0 a x_fin con passo h costante.
    Restituisce i vettori x (n+1) e la matrice Y (n+1, dim)."""
    n = int(round((x_fin - x0) / h))
    x = x0 + h * np.arange(n + 1)
    Y = np.zeros((n + 1, len(y0)))
    Y[0] = y0
    for i in range(n):
        Y[i + 1] = rk4_passo(f, x[i], Y[i], h)
```

```
    return x, Y


# ---------------------------------------------------------------------------
# 3. Equazione di Poisson  F_xx + F_yy = Q  su griglia di passo h.
#    F[i, j] = valore nel nodo (x_i, y_j); i valori sul contorno restano fissi.
# ---------------------------------------------------------------------------
def liebmann(F, Q, h, errmax, azzera=True):
    """Metodo di Liebmann (Gauss-Seidel) sui nodi interni di un rettangolo;
    si ferma quando la somma dei |variazioni| di un'iterazione e' <= errmax.
    Se azzera=True i nodi interni partono da zero (come la subroutine LIEBMN).
    Restituisce (F, numero di iterazioni)."""
    M, N = F.shape
    if azzera:
        F[1:-1, 1:-1] = 0.0
    it = 0
    while True:
        it += 1
        err = 0.0
        for i in range(1, M - 1):
            for j in range(1, N - 1):
                nuovo = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] +
                                F[i, j + 1] - h * h * Q[i, j])
                err += abs(nuovo - F[i, j])
                F[i, j] = nuovo
        if err <= errmax:
            return F, it


def sor_poisson(F, Q, h, omega, errmax, nodi, prima_del_nodo=None, it_max=100000):
    """Sovrarilassamento successivo sui nodi 'nodi' (lista di coppie (i, j),
    visitati nell'ordine dato). prima_del_nodo(F, i, j), se fornita, viene
    chiamata prima di aggiornare ogni nodo (serve ad aggiornare i nodi fittizi
    delle condizioni di derivata). Restituisce (F, iterazioni)."""
    for it in range(1, it_max + 1):
        err = 0.0
        for i, j in nodi:
            if prima_del_nodo is not None:
                prima_del_nodo(F, i, j)
            gs = 0.25 * (F[i - 1, j] + F[i + 1, j] + F[i, j - 1] + F[i, j + 1] - h
    * h * Q[i, j])
            nuovo = (1.0 - omega) * F[i, j] + omega * gs
            err += abs(nuovo - F[i, j])
            F[i, j] = nuovo
        if err <= errmax:
            return F, it
    raise RuntimeError("SOR: convergenza non raggiunta")


def omega_ottimo(m, n):
    """Fattore di sovrarilassamento ottimo per un rettangolo con m x n linee di
    griglia (come nei programmi originali):
    omega = (8 - 4 sqrt(4 - a^2)) / a^2,  a = cos(pi/m) + cos(pi/n)."""
    a = np.cos(np.pi / m) + np.cos(np.pi / n)
    return (8.0 - 4.0 * np.sqrt(4.0 - a * a)) / (a * a)


# ---------------------------------------------------------------------------
# 4. Metodi spettrali
# ---------------------------------------------------------------------------
```

```python
def cheb(N):
    """Nodi di Chebyshev-Gauss-Lobatto y_j = cos(j pi / N), j = 0..N (da +1 a -1)
    e matrice di differenziazione D (N+1 x N+1): (D u)_j approssima u'(y_j).
    Elementi fuori diagonale: D_jk = c_j (-1)^(j+k) / (c_k (y_j - y_k)), c_0 = c_N = 2;
    la diagonale si ottiene imponendo che la derivata di una costante sia nulla
    (somma per righe nulla), che e' anche la scelta numericamente piu' stabile."""
    if N == 0:
        return np.zeros((1, 1)), np.array([1.0])
    j = np.arange(N + 1)
    y = np.cos(np.pi * j / N)
    c = np.ones(N + 1)
    c[0] = c[-1] = 2.0
    c = c * (-1.0) ** j
    Y = np.tile(y, (N + 1, 1)).T
    dY = Y - Y.T
    D = np.outer(c, 1.0 / c) / (dY + np.eye(N + 1))
    D = D - np.diag(D.sum(axis=1))
    return D, y


def numeri_onda(N, L):
    """Numeri d'onda k_n = 2 pi n / L nell'ordine usato da numpy.fft
    (n = 0, 1, ..., N/2-1, -N/2, ..., -1)."""
    return 2.0 * np.pi * np.fft.fftfreq(N, d=L / N)
```

Listato A.1: `metodi_numerici.py`: sistemi tridiagonali, Runge–Kutta, metodi iterativi per l'equazione di Poisson e altre funzioni comuni ai moduli del corso (verificato dal Programma 0.9).

```python
"""
stile_grafici.py -- impostazioni grafiche comuni (palette "della casa").
Importato da tutti gli script della monografia.
"""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Palette coerente con raucci-aero.sty
BLU      = "#1E2761"
ARANCIO  = "#DC7633"
ROSSO    = "#C0392B"
VERDE    = "#1E8449"
GIALLO   = "#B7950B"
VIOLA    = "#6C3483"
GRIGIO   = "#566573"
PALETTE  = [BLU, ARANCIO, VERDE, VIOLA, ROSSO, GIALLO, GRIGIO]

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 9,
    "axes.labelsize": 9,
    "axes.titlesize": 9,
    "legend.fontsize": 8,
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "axes.prop_cycle": matplotlib.cycler(color=PALETTE),
    "axes.grid": True,
    "grid.alpha": 0.3,
```



stile_grafici.py

```
    "grid.linestyle": ":",
    "lines.linewidth": 1.4,
    "figure.dpi": 120,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.02,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "legend.frameon": False,
})

# Le figure vengono salvate nella cartella "figure" accanto a quella degli script
# (creata se manca): cosi' ogni programma funziona anche se scaricato da solo.
FIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "figure")


def salva(fig, nome):
    """Salva la figura in PDF nella cartella delle figure del progetto LaTeX."""
    os.makedirs(FIG_DIR, exist_ok=True)
    from matplotlib.ticker import NullFormatter
    for ax in fig.get_axes():              # niente etichette secondarie sugli assi logaritmici
        if ax.get_xscale() == "log":
            ax.xaxis.set_minor_formatter(NullFormatter())
        if ax.get_yscale() == "log":
            ax.yaxis.set_minor_formatter(NullFormatter())
    fig.savefig(os.path.join(FIG_DIR, f"{nome}.pdf"))
    plt.close(fig)
    print(f"  -> figura salvata: {nome}.pdf")
```

Listato A.2: `stile_grafici.py`: impostazioni comuni di Matplotlib, palette e funzione `salva` (Cap. 5).

esercizi_svolti.py

```
"""
esercizi_svolti.py -- codice dei venti esercizi svolti del Modulo 0.

Ogni esercizio e' la funzione es_NN(); senza argomenti il programma li esegue
tutti, con  python3 esercizi_svolti.py 3 12  esegue solo il terzo e il dodicesimo.
Le figure sono salvate in ../figure con il nome es_NN.
"""
import os
import sys
import math
import time
import json
import numpy as np
import matplotlib.pyplot as plt
from stile_grafici import salva, BLU, ARANCIO, VERDE, ROSSO, GRIGIO, PALETTE

QUI = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(QUI, "letture"))          # i programmi delle monografie usati nelle letture guidate


# ============================================================================
def es_01():
    """Conversioni di unita' e tabella formattata: velocita' in nodi, km/h, m/s; quote in piedi e metri."""
    print("Es. 1 -- conversioni e formattazione")
    NODO, PIEDE = 1852.0 / 3600.0, 0.3048                    # 1 kt = 1852 m/h; 1 ft = 0,3048 m
    print(f"   {'kt':>6} {'km/h':>8} {'m/s':>8}      {'ft':>7} {'m':>8}")
```

```
27      for kt, ft in zip((100, 250, 450, 500), (1000, 10000, 35000, 41000)):
28          v = kt * NODO
29          print(f"   {kt:6d} {v*3.6:8.1f} {v:8.2f}      {ft:7d} {ft*PIEDE:8.1f}")
30      v = 250 * NODO
31      print(f"   velocita' di {250} kt in notazione italiana:
        {f'{v:.2f}'.replace('.', ',')} m/s; in esponenziale {v:.3e}")
32
33
34  # ================================================================================
35  def es_02():
36      """Numero di Mach e pressione dinamica in quota con la funzione isa() del
        Programma 0.1."""
37      print("Es. 2 -- Mach e pressione dinamica in quota")
38      from prog0_1_primi_passi import isa                        # riuso di una
        funzione di un altro file
39
40      def mach_e_q(v, z):
41          T, p, rho, a = isa(z)
42          return v / a, 0.5 * rho * v**2
43
44      print(f"   {'z [m]':>7} {'a [m/s]':>8} {'Mach':>6} {'q [Pa]':>9}   (v = 250
        m/s)")
45      for z in (0, 3000, 6000, 9000, 11000):
46          M, q = mach_e_q(250.0, z)
47          print(f"   {z:7d} {isa(z)[3]:8.2f} {M:6.3f} {q:9.0f}")
48      print("   a parita' di velocita' vera, in quota il Mach cresce (a diminuisce) e
        la pressione dinamica diminuisce (rho diminuisce)")
49
50
51  # ================================================================================
52  def es_03():
53      """Virgola mobile: somma compensata di Kahan e formula stabile per le radici di
        un'equazione di secondo grado."""
54      print("Es. 3 -- virgola mobile")
55      valori = [1e-8] * 10**6
56      ingenua = 0.0
57      for v in valori:
58          ingenua += v
59      somma, c = 0.0, 0.0                                        # Kahan: 'c' accumula
        l'errore commesso
60      for v in valori:
61          y = v - c
62          t = somma + y
63          c = (t - somma) - y
64          somma = t
65      print(f"   un milione di 1e-8: ingenua {ingenua:.17f}, Kahan {somma:.17f},
        esatta 0.01 (math.fsum: {math.fsum(valori):.17f})")
66      a, b, cc = 1.0, 1e8, 1.0                                   # x^2 + 1e8 x + 1 =
        0: radici -1e8 e -1e-8
67      disc = math.sqrt(b * b - 4 * a * cc)
68      x1, x2 = (-b + disc) / (2 * a), (-b - disc) / (2 * a)      # formula classica: x1
        soffre di cancellazione
69      q = -0.5 * (b + math.copysign(disc, b))                    # formula stabile: x1
        = q/a, x2 = c/q
70      print(f"   radici classiche: {x1:.6e}, {x2:.6e};  stabili: {cc / q:.6e}, {q /
        a:.6e}  (esatte -1e-8, -1e8)")
71
72
73  # ================================================================================
74  def es_04():
```

```
75      """Cicli con criterio di arresto: serie di Taylor di sin(x) fino a precisione
    prefissata."""
76      print("Es. 4 -- serie di Taylor con un ciclo while")
77      for x in (0.5, 2.0, 10.0):
78          termine, somma, k = x, x, 0
79          while abs(termine) > 1e-15 * abs(somma):
80              k += 1
81              termine *= -x * x / ((2 * k) * (2 * k + 1))       # termine successivo
    per ricorrenza: niente fattoriali enormi
82              somma += termine
83          print(f"   x = {x:4.1f}: {k + 1} termini, somma {somma:.15f}, math.sin
    {math.sin(x):.15f}, errore {abs(somma - math.sin(x)):.1e}")
84      print("   per x = 10 l'errore cresce: i termini intermedi (~10^8) si cancellano
    fra loro (cancellazione catastrofica)")
85
86
87  # ============================================================================
88  def es_05():
89      """Funzioni: schema di Horner per un polinomio e la sua derivata; Newton sul
    polinomio dello spessore NACA."""
90      print("Es. 5 -- Horner e Newton")
91
92      def horner(coeff, x):
93          """Valore e derivata di p(x) = c0 + c1 x + ... + cn x^n con lo schema di
    Horner."""
94          p, dp = 0.0, 0.0
95          for c in reversed(coeff):
96              dp = dp * x + p
97              p = p * x + c
98          return p, dp
99
100     coeff = [-6.0, 11.0, -6.0, 1.0]                              # x^3 - 6x^2 + 11x -
    6 = (x-1)(x-2)(x-3), potenze crescenti
101     print(f"   p(2,5) = {horner(coeff, 2.5)[0]:.4f}, p'(2,5) = {horner(coeff,
    2.5)[1]:.4f} (esatti -0,375 e p'(2,5) = 3x^2 - 12x + 11 = {3*6.25 - 30 + 11})")
102     # spessore NACA con bordo d'uscita aperto (-0,1015): dove si annulla y_t oltre
    x = 1? (Newton in sqrt(x) = s)
103     a = [0.2969, -0.1260, -0.3516, 0.2843, -0.1015]
104     ys = [0.0, a[0], a[1], 0.0, a[2], 0.0, a[3], 0.0, a[4]]   # polinomio in s =
    sqrt(x): a0 s - a1 s^2 - ...
105     s = 1.0
106     for k in range(20):
107         p, dp = horner(ys, s)
108         s_nuovo = s - p / dp
109         if abs(s_nuovo - s) < 1e-14:
110             break
111         s = s_nuovo
112     print(f"   bordo d'uscita aperto: y_t(1) = {horner(ys, 1.0)[0] * 5 * 0.12:.5f}
    c (NACA 0012); y_t = 0 in x = s^2 = {s**2:.6f} ({k} iterazioni di Newton)")
113
114
115 # ============================================================================
116 def es_06():
117     """Liste e dizionari: elaborare una polare (efficienza massima, angolo di
    portanza nulla, ordinamenti)."""
118     print("Es. 6 -- la polare come dizionario")
119     polare = {-4: (-0.20, 0.0090), -2: (0.00, 0.0075), 0: (0.25, 0.0070), 2: (0.45,
    0.0080), 4: (0.67, 0.0100),
120               6: (0.88, 0.0130), 8: (1.08, 0.0170), 10: (1.25, 0.0230), 12: (1.35,
    0.0320), 14: (1.30, 0.0500)}
```

```
121     eff = {alfa: cl / cd for alfa, (cl, cd) in polare.items()}
122     a_max = max(eff, key=eff.get)
123     print(f"   efficienza massima E = {eff[a_max]:.1f} a alfa = {a_max} gradi (C_L
        = {polare[a_max][0]}, C_D = {polare[a_max][1]})")
124     stallo = max(polare, key=lambda alfa: polare[alfa][0])
125     print(f"   C_L massimo {polare[stallo][0]} a {stallo} gradi; C_L0 =
        {polare[0][0]}; pendenza media fra -2 e 8 gradi: "
126           f"{(polare[8][0] - polare[-2][0]) / 10:.4f} per grado")
127     sotto = [alfa for alfa, (cl, cd) in polare.items() if cd < 0.012]
128     print(f"   angoli con C_D < 0,012: {sotto}; polare ordinata per C_D
        decrescente: {sorted(polare, key=lambda a: -polare[a][1])[:4]} ...")
129
130
131 # ============================================================================
132 def es_07():
133     """Stringhe e file: scrivere un file di coordinate del profilo e rileggerlo
        analizzando le righe."""
134     print("Es. 7 -- file di coordinate")
135     from prog0_3_strutture_dati import profilo
136     dorso, ventre = profilo("2412", n=20)
137     percorso = os.path.join(QUI, "..", "figure", "es_07_naca2412.dat")
138     with open(percorso, "w", encoding="utf-8") as fp:
139         fp.write("# NACA 2412, coordinate x/c y/c (dorso dal bordo d'uscita, poi
        ventre)\n")
140         for x, y in list(reversed(dorso)) + ventre[1:]:
141             fp.write(f"{x:10.6f} {y:10.6f}\n")
142     punti = []
143     with open(percorso, encoding="utf-8") as fp:
144         for riga in fp:
145             riga = riga.strip()
146             if not riga or riga.startswith("#"):          # salta commenti e righe
        vuote
147                 continue
148             campi = riga.split()
149             punti.append((float(campi[0]), float(campi[1])))
150     print(f"   scritte e rilette {len(punti)} coppie; prima {punti[0]}, ultima
        {punti[-1]}")
151     print(f"   lo stesso file letto con NumPy: forma {np.loadtxt(percorso).shape};
        ordinata massima {max(y for x, y in punti):.4f} c")
152
153
154 # ============================================================================
155 def es_08():
156     """Vettorializzare: media mobile e derivata di un segnale con cicli e con lo
        slicing; tempi e uguaglianza."""
157     print("Es. 8 -- vettorializzazione")
158     n = 200000
159     x = np.linspace(0, 10, n)
160     f = np.sin(x) + 0.05 * np.random.default_rng(1).standard_normal(n)
161     t0 = time.perf_counter()
162     media1 = np.empty(n - 2)
163     for i in range(1, n - 1):
164         media1[i - 1] = (f[i - 1] + f[i] + f[i + 1]) / 3
165     t1 = time.perf_counter() - t0
166     t0 = time.perf_counter()
167     media2 = (f[:-2] + f[1:-1] + f[2:]) / 3
168     t2 = time.perf_counter() - t0
169     print(f"   media mobile a tre punti su {n} valori: ciclo {t1*1e3:.0f} ms,
        slicing {t2*1e3:.2f} ms, uguali? {np.allclose(media1, media2)}")
170     t0 = time.perf_counter(); d = np.gradient(f, x); t3 = time.perf_counter() - t0
```

```
    print(f"   np.gradient (differenze centrate, unilatere agli estremi): {t3*1e3:.2f} ms; np.convolve per la media: "
          f"{np.allclose(np.convolve(f, np.ones(3) / 3, mode='valid'), media2)}")


# ============================================================================
def es_09():
    """Broadcasting: tabella bidimensionale della velocita' limite per diametri e densita' diversi."""
    print("Es. 9 -- broadcasting")
    d = np.array([0.001, 0.005, 0.01, 0.05])[:, None]            # (4, 1)
    rho = np.array([1000.0, 2700.0, 7800.0])[None, :]             # (1, 3)
    m = rho * np.pi * d**3 / 6                                    # (4, 3)
    vt = np.sqrt(2 * m * 9.80665 / (1.225 * np.pi * d**2 / 4 * 0.5))  # c_d = 0,5 costante
    print("   v_t [m/s] (righe: d = 1, 5, 10, 50 mm; colonne: legno/acqua 1000, alluminio 2700, acciaio 7800 kg/m^3)")
    for riga in vt:
        print("   " + " ".join(f"{v:8.2f}" for v in riga))
    print(f"   forma di vt: {vt.shape}; v_t cresce come sqrt(rho d): rapporto fra d = 50 e 5 mm = {vt[3, 0] / vt[1, 0]:.3f} (sqrt(10) = {np.sqrt(10):.3f})")


# ============================================================================
def es_10():
    """Slicing 2D: laplaciano con condizioni di Neumann imposte con i nodi fittizi (array esteso)."""
    print("Es. 10 -- nodi fittizi")
    N = 40; h = 1.0 / N
    x = np.linspace(0, 1, N + 1)
    X, Y = np.meshgrid(x, x, indexing="ij")
    u = np.cos(np.pi * X) * np.cos(np.pi * Y)                     # du/dn = 0 su tutto il contorno
    ue = np.zeros((N + 3, N + 3))
    ue[1:-1, 1:-1] = u
    ue[0, :] = ue[2, :]; ue[-1, :] = ue[-3, :]; ue[:, 0] = ue[:, 2]; ue[:, -1] = ue[:, -3]   # riflessione: derivata normale nulla
    lap = (ue[2:, 1:-1] + ue[:-2, 1:-1] + ue[1:-1, 2:] + ue[1:-1, :-2] - 4 * ue[1:-1, 1:-1]) / h**2
    esatto = -2 * np.pi**2 * u
    err = np.max(np.abs(lap - esatto))
    print(f"   laplaciano con nodi fittizi su tutta la griglia (bordi compresi), N = {N}: errore massimo {err:.2e},"
          f" sul bordo {np.max(np.abs((lap - esatto)[0, :])):.2e}; stima teorica h^2 pi^4/6 = {h**2 * np.pi**4 / 6:.2e}")
    N2 = 2 * N; h2 = 1.0 / N2; x2 = np.linspace(0, 1, N2 + 1); X2, Y2 = np.meshgrid(x2, x2, indexing="ij")
    u2 = np.cos(np.pi * X2) * np.cos(np.pi * Y2); ue2 = np.zeros((N2 + 3, N2 + 3)); ue2[1:-1, 1:-1] = u2
    ue2[0, :] = ue2[2, :]; ue2[-1, :] = ue2[-3, :]; ue2[:, 0] = ue2[:, 2]; ue2[:, -1] = ue2[:, -3]
    lap2 = (ue2[2:, 1:-1] + ue2[:-2, 1:-1] + ue2[1:-1, 2:] + ue2[1:-1, :-2] - 4 * ue2[1:-1, 1:-1]) / h2**2
    print(f"   con N = {N2}: errore {np.max(np.abs(lap2 + 2 * np.pi**2 * u2)):.2e}, rapporto {err / np.max(np.abs(lap2 + 2 * np.pi**2 * u2)):.2f} (atteso 4)")


# ============================================================================
def es_11():
    """Matplotlib: polare e curva C_L-alfa in due pannelli con annotazioni, nello
```

```
    stile della casa."""
    print("Es. 11 -- figura a due pannelli")
    alfa = np.array([-4, -2, 0, 2, 4, 6, 8, 10, 12, 14])
    cl = np.array([-0.20, 0.00, 0.25, 0.45, 0.67, 0.88, 1.08, 1.25, 1.35, 1.30])
    cd = np.array([0.0090, 0.0075, 0.0070, 0.0080, 0.0100, 0.0130, 0.0170, 0.0230,
    0.0320, 0.0500])
    fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
    ax[0].plot(alfa, cl, "o-", color=BLU, ms=3)
    ax[0].set_xlabel("$\\alpha$ [gradi]"); ax[0].set_ylabel("$C_L$");
    ax[0].axhline(0, color=GRIGIO, lw=0.5); ax[0].axvline(0, color=GRIGIO, lw=0.5)
    k = np.argmax(cl)
    ax[0].annotate("stallo", xy=(alfa[k], cl[k]), xytext=(alfa[k] - 6, cl[k] +
    0.05), fontsize=8, arrowprops=dict(arrowstyle="->", lw=0.6))
    ax[1].plot(cd, cl, "o-", color=ARANCIO, ms=3)
    e = cl / cd; j = np.argmax(e)
    ax[1].plot([0, cd[j]], [0, cl[j]], ":", color=GRIGIO)
    ax[1].annotate(f"$E_{{\\max}}={e[j]:.0f}$", xy=(cd[j], cl[j]), xytext=(cd[j] +
    0.008, cl[j] - 0.3), fontsize=8, arrowprops=dict(arrowstyle="->", lw=0.6))
    ax[1].set_xlabel("$C_D$"); ax[1].set_ylabel("$C_L$"); ax[1].set_title("polare",
    fontsize=9)
    fig.tight_layout()
    salva(fig, "es_11")


# ============================================================================
def es_12():
    """SciPy fsolve: il sistema ridotto di Galerkin per la sfera (Modulo 4) a Re =
    100; confronto con la tabella."""
    print("Es. 12 -- fsolve sul sistema di Galerkin della sfera")
    from scipy.optimize import fsolve
    Re = 100.0
    g = lambda A1, B1: 9.931035 / Re * A1 + 0.011713 * B1 - 0.002546 * A1 * B1 +
    44.689656 / Re
    h = lambda A1, B1: 0.050148 * A1 + 2.201058 / Re * B1 + 0.002421 * A1**2 -
    0.000376 * B1**2 + 0.379018
    sol, info, ier, msg = fsolve(lambda p: [g(*p), h(*p)], [-4.0, -1.0],
    full_output=True)
    print(f"   A1 = {sol[0]:.4f}, B1 = {sol[1]:.3f} (tabella: -2,6083, -10,236);
    residui {g(*sol):.1e}, {h(*sol):.1e}; {info['nfev']} valutazioni")
    A2 = -15 / 29 * (8 + 5 * sol[0]); A3 = 9 / 29 * (17 + 7 * sol[0]); A4 = -(95 +
    34 * sol[0]) / 58
    cd = 32 / (3 * Re) * (8 * A2 + 25 * A3 + 54 * A4) + 64 / (3 * Re) * (2 * A2 + 5
    * A3 + 9 * A4)
    print(f"   C_D = {cd:.4f} (tabella: 1,0436)")


# ============================================================================
def es_13():
    """solve_ivp: periodo del pendolo non lineare in funzione dell'ampiezza,
    confrontato con l'integrale ellittico."""
    print("Es. 13 -- pendolo con solve_ivp ed eventi")
    from scipy.integrate import solve_ivp
    from scipy.special import ellipk
    print(f"   {'theta0 [gradi]':>14} {'T/T0 numerico':>14} {'T/T0 esatto':>12}")
    for th0 in (10, 45, 90, 135, 170):
        th0r = np.radians(th0)
        passaggio = lambda t, y: y[0]; passaggio.direction = -1; passaggio.terminal
    = True
        s = solve_ivp(lambda t, y: [y[1], -np.sin(y[0])], (0, 50), [th0r, 0.0],
    events=passaggio, rtol=1e-11, atol=1e-12)
```

```
259         T_num = 4 * s.t_events[0][0]                                    # un quarto di
    periodo fino al primo passaggio per zero
260         T_es = 4 * ellipk(np.sin(th0r / 2)**2)                          # T = 4 K(k), k
    = sin(theta0/2), in unita' di sqrt(l/g)
261         print(f"   {th0:14d} {T_num / (2 * np.pi):14.6f} {T_es / (2 *
    np.pi):12.6f}")
262
263
264 # ============================================================================
265 def es_14():
266     """Schema implicito per la diffusione con l'algoritmo di Thomas: nessun limite
    su R, confronto con FTCS."""
267     print("Es. 14 -- BTCS con Thomas")
268     from metodi_numerici import thomas
269     from prog0_8_anatomia import esatta, integra
270     n = 40; x = np.linspace(0, 1, n + 1); h = x[1] - x[0]
271     for R in (0.4, 2.0, 10.0):
272         dt = R * h**2; n_passi = int(round(0.1 / dt))
273         u = np.sin(np.pi * x)
274         a = np.full(n - 1, -R); b = np.full(n - 1, 1 + 2 * R); c = np.full(n - 1,
    -R)
275         for _ in range(n_passi):
276             u[1:-1] = thomas(a, b, c, u[1:-1])                          # u = 0 agli
    estremi: nessun termine noto aggiuntivo
277         err = np.max(np.abs(u - esatta(x, n_passi * dt)))
278         print(f"   R = {R:5.1f}: {n_passi:4d} passi, errore {err:.2e}" + ("" if R >
    0.5 else f" (FTCS: {np.max(np.abs(integra(n, R)[1] - esatta(x, 0.1))):.2e})"))
279     print("   l'implicito e' stabile per ogni R ma l'errore cresce con dt (primo
    ordine nel tempo)")
280
281
282 # ============================================================================
283 def es_15():
284     """SymPy: formula centrata a cinque punti per la derivata prima con il metodo
    dei coefficienti indeterminati."""
285     print("Es. 15 -- formula a cinque punti con SymPy")
286     import sympy as sp
287     h = sp.symbols("h", positive=True)
288     f = sp.symbols("f0:7")                                              # f0 = f(x), f1
    = f'(x), ...
289     taylor = lambda k: sum((k * h)**j / sp.factorial(j) * f[j] for j in range(7))
290     c = sp.symbols("c_m2 c_m1 c_0 c_1 c_2")
291     approx = sp.expand(sum(ck * taylor(k) for ck, k in zip(c, (-2, -1, 0, 1, 2))) /
    h)
292     eqs = [approx.coeff(f[0]), approx.coeff(f[1]) - 1] + [approx.coeff(f[j]) for j
    in (2, 3, 4)]
293     sol = sp.solve(eqs, c)
294     print(f"   coefficienti: {[sol[ck] for ck in c]}  -> f' = (f_{{-2}} - 8
    f_{{-1}} + 8 f_1 - f_2)/(12 h)")
295     errore = sp.expand(approx.subs(sol) - f[1])
296     print(f"   errore di troncamento: {errore.coeff(f[5]) * f[5]}  (quarto ordine)")
297
298
299 # ============================================================================
300 def es_16():
301     """Lettura guidata 1: la caduta della sfera (Programma 1.1) riscritta con
    l'integratore generico ode_solvers.integra."""
302     print("Es. 16 -- Programma 1.1 con ode_solvers")
303     from ode_solvers import integra, cd_sfera
304     from prog1_1_caduta_sfera import caduta, costanti, NU
```

```
    d = 0.01
    A, B, C = costanti(d)
    F = lambda t, y: np.array([y[1], (B - C * y[1] * abs(y[1]) * cd_sfera(y[1] * d / NU)) / A])
    T, Y = integra(F, 0.0, [0.0, 0.0], 0.1, 100, metodo="rk4")
    T1, Z1, V1, _ = caduta(d, h=0.1, tmax=10.0)
    print(f"   a t = 10 s: z = {Y[-1, 0]:.4f} m, v = {Y[-1, 1]:.4f} m/s (Programma 1.1: {Z1[-1]:.4f}, {V1[-1]:.4f});"
          f" differenza {abs(Y[-1, 1] - V1[-1]):.1e}")
    print("   stesso risultato: il Programma 1.1 scrive RK4 'a mano' per due equazioni, ode_solvers lo fa per un vettore di stato qualsiasi")


# ============================================================================
def es_17():
    """Lettura guidata 2: il problema di Rayleigh (Programma 3.4) con lo schema implicito al posto dell'esplicito."""
    print("Es. 17 -- Programma 3.4 in forma implicita")
    from metodi_numerici import thomas
    from scipy.special import erfc
    M, D, NU, U0 = 21, 1.0, 1.0e-6, 1.0
    H = D / (M - 1)
    y = np.linspace(0, D, M)

    def esatta(y, t, nmax=200):
        u = y / D
        for n in range(1, nmax + 1):
            u = u + 2 * (-1)**n / (n * np.pi) * np.sin(n * np.pi * y / D) * np.exp(-(n * np.pi)**2 * NU * t / D**2)
        return U0 * u

    for R in (0.25, 2.0, 8.0):
        tau = R * H**2 / NU
        u = np.zeros(M); u[-1] = U0
        a = np.full(M - 2, -R); b = np.full(M - 2, 1 + 2 * R); c = np.full(M - 2, -R)
        t, n = 0.0, 0
        while n * tau < 2.5e5 - 1e-9:
            rhs = u[1:-1].copy(); rhs[-1] += R * U0                 # il nodo superiore (u = U0) entra nel termine noto
            u[1:-1] = thomas(a, b, c, rhs); n += 1
        t = n * tau
        print(f"   R = {R:4.2f}: tau = {tau:7.0f} s, {n:4d} passi fino a t = {t:.3e} s, errore massimo {np.max(np.abs(u - esatta(y, t))):.2e} m/s")
    print("   il Programma 3.4 (esplicito, R = 1/4) usa 400 passi; l'implicito con R = 8 ne usa 13, con un errore 20 volte maggiore")


# ============================================================================
def es_18():
    """Lettura guidata 3: usare il Programma 4.5 (cavita' MAC) come libreria: energia cinetica e vorticita' di parete."""
    print("Es. 18 -- Programma 4.5 come libreria")
    from prog4_5_cavita_mac import cavita
    r = cavita(100, 32, 20.0, stampa=False)
    u, v, h, N = r["u"], r["v"], r["h"], r["N"]
    uc = 0.5 * (u[1:, 1:-1] + u[:-1, 1:-1]); vc = 0.5 * (v[1:-1, 1:] + v[1:-1, :-1])   # velocita' nei centri delle celle
    ek = 0.5 * np.sum(uc**2 + vc**2) * h * h
    om_cop = -(2.0 - 2 * u[:, N]) / h
```

```
        # vorticita' sul coperchio: -du/dy con il nodo fittizio
354     print(f"   Re = 100, griglia {N+1}x{N+1}: psi_min = {r['psi'].min():.4f},
        energia cinetica {ek:.5f}, vorticita' media sul coperchio
        {om_cop[1:-1].mean():.2f}")
355     print(f"   divergenza massima {np.max(np.abs((u[1:, 1:-1] - u[:-1, 1:-1]) / h +
        (v[1:-1, 1:] - v[1:-1, :-1]) / h)):.1e}: la funzione restituisce un dizionario
        con tutti i campi")
356
357
358 # ============================================================================
359 def es_19():
360     """Programma completo da zero: convezione lineare con upwind e Lax-Wendroff,
        ordine di convergenza, figura."""
361     print("Es. 19 -- programma completo: convezione lineare")
362
363     def integra(schema, nx, C, t_fin, L=1.0):
364         x = np.linspace(0, L, nx, endpoint=False); dx = x[1] - x[0]
365         u = np.exp(-200 * (x - 0.3)**2)
366         n = int(round(t_fin / (C * dx)))
367         for _ in range(n):
368             um, up = np.roll(u, 1), np.roll(u, -1)
369             if schema == "upwind":
370                 u = u - C * (u - um)
371             else:
372                 u = u - 0.5 * C * (up - um) + 0.5 * C**2 * (up - 2 * u + um)
373         return x, u, np.exp(-200 * ((x - 0.3 - n * C * dx + L / 2) % L - L / 2)**2)
374
375     print(f"   {'nx':>5} {'upwind':>10} {'Lax-W':>10}")
376     errs = {"upwind": [], "laxw": []}
377     for nx in (100, 200, 400, 800):
378         riga = f"   {nx:5d}"
379         for schema in ("upwind", "laxw"):
380             x, u, ue = integra(schema, nx, 0.5, 0.5)
381             errs[schema].append(np.max(np.abs(u - ue))); riga += f"
        {errs[schema][-1]:10.2e}"
382         print(riga)
383     for schema in errs:
384         print(f"   ordine {schema}: {-np.polyfit(np.log([100, 200, 400, 800]),
        np.log(errs[schema]), 1)[0]:.2f}")
385     print("   l'upwind tende all'ordine 1 solo su griglie fini (la gaussiana
        stretta e' mal risolta a 100 nodi); Lax-Wendroff e' gia' al secondo")
386     fig, ax = plt.subplots(figsize=(5.0, 2.8))
387     x, u1, ue = integra("upwind", 200, 0.5, 0.5); _, u2, _ = integra("laxw", 200,
        0.5, 0.5)
388     ax.plot(x, ue, color=GRIGIO, label="esatta"); ax.plot(x, u1, color=BLU,
        label="upwind"); ax.plot(x, u2, color=ARANCIO, label="Lax--Wendroff")
389     ax.set_xlabel("$x$"); ax.set_ylabel("$u$"); ax.legend(fontsize=7);
        fig.tight_layout()
390     salva(fig, "es_19")
391
392
393 # ============================================================================
394 def es_20():
395     """Caccia agli errori: una funzione con cinque errori (tre silenziosi) e la sua
        versione corretta."""
396     print("Es. 20 -- caccia agli errori")
397
398     def media_profilo_sbagliata(u, h):
399         # calcola la media di u sul dominio [0, 1] con la regola dei trapezi:
        contiene errori
```

```
400         n = len(u)
401         somma = 0
402         for i in range(1, n):                          # (1) salta il nodo 0 e pesa male
    gli estremi
403             somma += u[i]
404         h = 1 / n                                       # (2) h = 1/(n-1), non 1/n
405         return somma * h / 2                            # (3) il fattore 1/2 dei trapezi
    va solo agli estremi
406
407     def media_profilo(u, h):
408         return (0.5 * u[0] + np.sum(u[1:-1]) + 0.5 * u[-1]) * h
409
410     x = np.linspace(0, 1, 11); u = x**2
411     print(f"   integrale di x^2 su [0, 1]: sbagliato {media_profilo_sbagliata(u,
    0.1):.4f}, corretto {media_profilo(u, 0.1):.4f}, esatto 0,3333"
412           f" (trapezi: {np.trapezoid(u, x):.4f})")
413     # errori che il calcolatore segnala, da leggere nel traceback
414     for nome, azione in (("(4) indice", lambda: u[11]), ("(5) forme", lambda: u +
    np.zeros(10))):
415         try:
416             azione()
417         except (IndexError, ValueError) as e:
418             print(f"   {nome}: {type(e).__name__} -- {e}")
419     print("   gli errori 1-3 non producono messaggi: solo il confronto con un
    valore noto li rivela")
420
421
422 # ============================================================================
423 if __name__ == "__main__":
424     numeri = [int(a) for a in sys.argv[1:]] or range(1, 21)
425     for k in numeri:
426         globals()[f"es_{k:02d}"]()
427         print()
```

Listato A.3: `esercizi_svolti.py`: il codice dei venti esercizi svolti, una funzione per esercizio.

## A.4 Come riprodurre i risultati

**Istruzioni**

1. Requisiti: Python $\geq$ 3.9 con `numpy`, `scipy`, `matplotlib` e `sympy` (`pip install numpy scipy matplotlib sympy`); una distribuzione TEX con `pdflatex` e i pacchetti `tcolorbox`, `listings`, `qrcode`, `marginnote`, `babel-italian` (tutti inclusi in TEX Live / MacTeX completo).
2. Dalla cartella `codice/`: `bash esegui_tutti.sh` esegue i nove programmi e gli esercizi (circa un minuto) e rigenera le figure in `figure/`. Ogni programma si può eseguire da solo con `python3 nome.py`; gli esercizi importano funzioni dai Programmi 0.1, 0.3 e 0.8 e dai programmi delle letture guidate in `codice/letture/`, quindi vanno eseguiti dalla cartella `codice/`.
3. I tempi stampati (Programmi 0.4, 0.6, 0.9, Esercizio 8) dipendono dal calcolatore; tutti gli altri numeri sono riproducibili alla cifra.
4. Dalla cartella principale: `pdflatex main.tex` due volte (o `latexmk -pdf`

`main.tex`).

## A.5 Codici in rete e QR code

I programmi sono pubblicati all'indirizzo

`https://braucci.github.io/CFD/monografia0/codici/`

La pagina elenca tutti i file, li mostra con l'evidenziazione della sintassi e permette di scaricarli singolarmente o in un unico archivio. Il QR code nel margine accanto a ogni listato codifica l'indirizzo del file corrispondente e, nel PDF, è anche un collegamento cliccabile. Per i listati delle letture guidate (Cap. 8) il QR rimanda al file nella cartella della monografia di origine (`monografia1`, `monografia3`, `monografia4`), e l'etichetta sotto il QR lo indica.

**Per l'autore: configurazione e pubblicazione**

1. In `main.tex` la macro `\nomemonografia` (qui `monografia0`) determina tutti gli indirizzi: `\urlcodici` vale `https://braucci.github.io/CFD/<nome>/codici/` e `\urlfile{file.py}` aggiunge il nome del file.
2. Un listato completo si inserisce con `\listatopy[opzioni]{file.py}`; un estratto di un programma di un'altra monografia, copiato in `codice/letture/`, con `\listatoaltro[linerange={a-b}]{monografiaN}{file.py}`; i frammenti nel testo con `\begin{lstlisting}[style=frammento]`.
3. `python3 pubblica_github.py` crea `github/monografia0/` con il PDF, la cartella `codici/` (compresa `letture/`), la pagina `index.html` e l'archivio zip: va copiata così com'è nella cartella `CFD/` del repository, accanto alle altre monografie; il portale `index.html` della cartella `CFD/` va aggiornato con la scheda del modulo 0.

## A.6 Le figure

Tutte le figure di questa monografia sono generate dai programmi e lette dalla cartella `figure/` con la macro `\figuragenerata[larghezza]{nome}`; come nelle altre monografie, se esiste `immagini/nome.png` questo ha la precedenza sul PDF generato, e ogni grafico può essere sostituito da un'immagine esterna senza toccare il sorgente. Non ci sono figure con segnaposto.

# Appendice B

# Esercizi svolti

I venti esercizi seguono l'ordine dei capitoli: i primi cinque riguardano il linguaggio (numeri, cicli, funzioni), i successivi le collezioni e NumPy, poi Matplotlib, SciPy e SymPy; gli Esercizi 16–18 riprendono le tre letture guidate del Cap. 8 e chiedono di *usare* o *modificare* i programmi del corso; gli ultimi due sono un programma completo scritto da zero e una caccia agli errori. Ogni esercizio è svolto con lo schema delle altre monografie: *testo*, *impostazione* (che cosa ci si aspetta, prima di calcolare), *svolgimento*, *risultato*, *commento*. Il codice di tutti gli esercizi è nel file `esercizi_svolti.py` (Listato A.3), una funzione `es_NN()` per esercizio; `python3 esercizi_svolti.py` li esegue tutti, `python3 esercizi_svolti.py 3 12` solo il terzo e il dodicesimo. I numeri riportati nei riquadri dei risultati sono quelli stampati dal programma.

## B.1 Esercizio 1 – Conversioni di unità e tabella formattata

**Testo**

Scrivere un programma che stampi una tabella di velocità in nodi, km/h e m/s per 100, 250, 450, 500 kt e di quote in piedi e metri per 1000, 10 000, 35 000, 41 000 ft, con colonne allineate e un numero fissato di decimali; stampare poi una velocità con la virgola decimale italiana e in notazione esponenziale.

**Impostazione.** Servono due costanti di conversione (1 kt = 1852 m/h, 1 ft = 0,3048 m), un ciclo `for` su due sequenze accoppiate con `zip`, e le f-string con larghezza e precisione (`{v:8.1f}`): la larghezza allinea le colonne, l'intestazione si allinea con la stessa larghezza (`{'kt':>6}`).

**Svolgimento.** `NODO = 1852.0/3600.0` è la velocità di un nodo in m/s. Il ciclo `for kt, ft in zip((100, 250, 450, 500), (1000, 10000, 35000, 41000)):` spacchetta le coppie; ogni riga è `f"{kt:6d} {v*3.6:8.1f} {v:8.2f} ..."`. La virgola italiana si ottiene formattando e poi sostituendo: `f"{v:.2f}".replace(".", ",")`; la notazione esponenziale con `:.3e`.

**Risultato**

250 kt = 463,0 km/h = 128,61 m/s; 35 000 ft = 10 668,0 m; "128,61 m/s"; $1{,}286 \times 10^{2}$. La tabella ha le colonne allineate a destra con le larghezze 6, 8, 8, 7, 8.

**Commento.** Le costanti di conversione vanno scritte come *definizioni* (1852/3600) e non come decimali arrotondati (0,5144): il calcolatore fa la divisione esattamente nei limiti della precisione di macchina, e il lettore vede da dove viene il numero.

## B.2 Esercizio 2 – Mach e pressione dinamica in quota

**Testo**

Usando la funzione `isa(z)` del Programma 0.1 (importata, non copiata), calcolare per $v = 250$ m/s il numero di Mach $M = v/a$ e la pressione dinamica $q = \tfrac{1}{2}\rho v^2$ alle quote 0, 3000, 6000, 9000, 11 000 m, e commentare l'andamento.

**Impostazione.** `from prog0_1_primi_passi import isa` funziona perché il Programma 0.1 ha il blocco `__main__`: l'importazione definisce le costanti e la funzione senza eseguire le stampe. Ci si aspetta $M$ crescente (la velocità del suono cala con $T$) e $q$ decrescente (cala $\rho$).

**Svolgimento.** Una funzione `mach_e_q(v, z)` chiama `isa`, spacchetta `T, p, rho, a` e restituisce la tupla `(v/a, 0.5*rho*v**2)`; il ciclo sulle quote stampa la tabella.

**Risultato**

$z = 0$: $a = 340{,}29$ m/s, $M = 0{,}735$, $q = 38\,281$ Pa; $z = 11\,000$: $a = 295{,}07$ m/s, $M = 0{,}847$, $q = 11\,372$ Pa. A parità di velocità vera il Mach cresce del 15% e la pressione dinamica si riduce a meno di un terzo.

**Commento.** È il motivo per cui gli aeroplani volano in quota: a parità di $q$ (cioè di portanza con lo stesso $C_L$) la velocità vera è maggiore. Il riuso per importazione, invece della copia della funzione, garantisce che una correzione a `isa` si propaghi a tutti i programmi che la usano.

## B.3 Esercizio 3 – Virgola mobile: somma compensata e radici stabili

**Testo**

(a) Sommare un milione di volte $10^{-8}$ con un ciclo ingenuo e con la somma compensata di Kahan; confrontare con il valore esatto 0,01 e con `math.fsum`. (b) Risolvere $x^2 + 10^8 x + 1 = 0$ con la formula classica e con quella stabile, e confrontare con le radici esatte $-10^{-8}$ e $-10^{8}$.

**Impostazione.** (a) Ogni somma $s + 10^{-8}$ con $s \sim 10^{-2}$ arrotonda a $\epsilon s \sim 10^{-18}$; su $10^6$ somme l'errore accumulato può arrivare a $10^{-12}$ relativo, cioè $10^{-14}$ assoluto. L'algoritmo di Kahan tiene in una seconda variabile $c$ l'errore commesso a ogni passo e lo sottrae al passo successivo. (b) La radice piccola $x_1 = (-b + \sqrt{b^2 - 4ac})/2a$ è la differenza di due numeri quasi uguali ($b$ e $\sqrt{b^2-4} \approx b - 2/b$): cancellazione catastrofica. La forma stabile calcola $q = -\tfrac{1}{2}\big(b + \operatorname{sign}(b)\sqrt{b^2 - 4ac}\big)$ (una somma di termini concordi) e poi $x_1 = c/q$, $x_2 = q/a$, usando il prodotto delle radici $x_1 x_2 = c/a$.

```
somma, c = 0.0, 0.0
for v in valori:
    y = v - c               # il termine corretto dell'errore precedente
    t = somma + y           # la somma arrotondata
    c = (t - somma) - y     # l'errore commesso in questa somma (esatto in
    aritmetica float)
    somma = t
q = -0.5 * (b + math.copysign(disc, b)); x1, x2 = cc / q, q / a
```

**Risultato**

(a) Ingenua 0,00999999999994859 (errore $5 \times 10^{-14}$), Kahan 0,01000000000000000, `math.fsum` 0,01. (b) Classica: $x_1 = -7{,}45 \times 10^{-9}$ (errore del 25%!), $x_2 = -10^{8}$; stabile: $-10^{-8}$ e $-10^{8}$ esatti.

**Commento.** L'errore della somma ingenua è piccolo in assoluto, ma è dello stesso ordine delle tolleranze di convergenza che i programmi usano ($10^{-12}$): una somma lunga di contributi piccoli (un residuo, un'energia) merita `math.fsum` o `np.sum`, che usa una somma a coppie con errore $\mathcal{O}(\epsilon \log n)$. La formula stabile delle radici è un caso in cui la *matematica* è la stessa e l'*aritmetica* no: riorganizzare un calcolo per evitare sottrazioni fra quasi uguali è la prima regola dell'analisi numerica.

## B.4 Esercizio 4 – Serie di Taylor con un ciclo `while`

**Testo**

Calcolare $\sin x$ con la serie $\sum_k (-1)^k x^{2k+1}/(2k+1)!$ fermandosi quando il termine è inferiore a $10^{-15}$ della somma, per $x = 0{,}5,\ 2,\ 10$; contare i termini e confrontare con `math.sin`.

**Impostazione.** Il numero di termini non è noto in anticipo: `while`. Ogni termine si ottiene dal precedente per ricorrenza, $t_k = -t_{k-1} x^2/[(2k)(2k+1)]$, senza calcolare fattoriali e potenze enormi. Per $x = 10$ i termini intermedi arrivano a $10^{10}/10! \sim 3000$ e si cancellano fra loro: l'errore finale sarà maggiore.

```
termine, somma, k = x, x, 0
while abs(termine) > 1e-15 * abs(somma):
    k += 1
    termine *= -x * x / ((2 * k) * (2 * k + 1))
    somma += termine
```

**Risultato**

$x = 0{,}5$: 8 termini, errore nullo; $x = 2$: 12 termini, errore nullo; $x = 10$: 27 termini, errore $2 \times 10^{-14}$, dovuto alla cancellazione fra termini di modulo fino a $\sim 10^{3}$.

**Commento.** Il criterio di arresto *relativo* (`> 1e-15*abs(somma)`) è quello giusto: un criterio assoluto fallirebbe se la somma fosse grande o piccola. Il ciclo termina sempre perché i termini tendono a zero; in un ciclo iterativo di un metodo numerico, dove la convergenza non è garantita, va aggiunto un numero massimo di iterazioni (Cap. 2). La ricorrenza evita anche l'overflow di $x^{2k+1}$ e di $(2k+1)!$, entrambi oltre $10^{300}$ per $k$ grande.

## B.5 Esercizio 5 – Schema di Horner e metodo di Newton

**Testo**

Scrivere una funzione `horner(coeff, x)` che restituisca valore e derivata del polinomio $p(x) = c_0 + c_1 x + \cdots + c_n x^n$ con lo schema di Horner; verificarla su $(x-1)(x-2)(x-3)$ in $x = 2{,}5$. Usarla con il metodo di Newton per trovare dove si annulla, oltre il bordo d'uscita, la distribuzione di spessore NACA con il coefficiente $-0{,}1015$.

**Impostazione.** Lo schema di Horner valuta $p$ con $n$ moltiplicazioni e $n$ somme, partendo dal coefficiente di grado massimo: $p \leftarrow p\,x + c_k$; la derivata si accumula in parallelo, $p' \leftarrow p'x + p$, *prima* di aggiornare $p$. Lo spessore NACA è un polinomio in $s = \sqrt{x}$: $y_t \propto a_0 s - a_1 s^2 - a_2 s^4 + a_3 s^6 - a_4 s^8$; con $-0{,}1015$ il bordo d'uscita è aperto ($y_t(1) \neq 0$) e lo zero cade poco oltre $x = 1$.

**Svolgimento.** I coefficienti vanno passati in ordine di potenza *crescente* e letti con `reversed`. Il polinomio in $s$ ha coefficienti $[0, a_0, -a_1, 0, -a_2, 0, a_3, 0, -a_4]$. Newton da $s = 1$: `s_nuovo = s - p/dp` fino a $|s_{\text{nuovo}} - s| < 10^{-14}$.

**Risultato**

$p(2{,}5) = -0{,}375$, $p'(2{,}5) = -0{,}25$ (esatti). $y_t(1) = 0{,}00126\,c$ per il NACA 0012 (spessore del bordo d'uscita aperto, 0,25% della corda); $y_t = 0$ in $x = s^2 = 1{,}008930\,c$ dopo 3 iterazioni.

**Commento.** L'errore classico in questo esercizio è l'ordine dei coefficienti: passare la lista $[1, -6, 11, -6]$ (potenze decrescenti) a una funzione che li vuole crescenti dà $p(2{,}5) = -58{,}6$ senza alcun messaggio. La docstring della funzione deve dichiarare la convenzione, e un `assert` su un polinomio noto la verifica. Newton converge in tre iterazioni perché il punto iniziale è vicino e la funzione è regolare.

## B.6 Esercizio 6 – La polare come dizionario

**Testo**

Rappresentare la polare di un profilo come dizionario $\{\alpha : (C_L, C_D)\}$ e, con comprehension e funzioni con chiave, trovare: l'efficienza massima $E = C_L/C_D$ e l'angolo a cui si ha; il $C_L$ massimo; la pendenza media fra $-2^\circ$ e $8^\circ$; gli angoli con $C_D < 0{,}012$; la polare ordinata per $C_D$ decrescente.

**Impostazione.** Un dizionario con chiave intera (l'angolo in gradi) e valore una tupla. `{alfa: cl/cd for alfa, (cl, cd) in polare.items()}` costruisce il dizionario delle efficienze; `max(eff, key=eff.get)` restituisce la *chiave* con il valore massimo; `max(polare, key=lambda a: polare[a][0])` la chiave con il $C_L$ massimo.

**Svolgimento.** Le cinque richieste sono cinque righe: due `max` con chiave, una differenza di valori, una comprehension con `if`, un `sorted` con chiave negativa.

**Risultato**

$E_{\max} = 67{,}7$ a $\alpha = 6°$ ($C_L = 0{,}88$, $C_D = 0{,}013$); $C_{L,\max} = 1{,}35$ a $12°$; $C_{L0} = 0{,}25$; pendenza media 0,108 per grado (6,19 per radiante); $C_D < 0{,}012$ per $\alpha \in \{-4, -2, 0, 2, 4\}$; ordinata per $C_D$ decrescente: $14, 12, 10, 8, \ldots$

**Commento.** Il pattern `max(dizionario, key=dizionario.get)` ("la chiave del massimo") e `sorted(seq, key=lambda ...)` coprono quasi tutte le interrogazioni su una tabella senza scrivere cicli. Per calcoli numerici su tutta la polare (interpolazione, derivate) si passa agli array, come nel Programma 0.6; il dizionario serve finché i dati sono pochi e si interrogano per chiave.

## B.7 Esercizio 7 – Un file di coordinate

**Testo**

Con la funzione `profilo` del Programma 0.3 scrivere su file le coordinate del NACA 2412 nel formato usuale (una riga di commento, poi coppie $x/c$, $y/c$ dal bordo d'uscita lungo il dorso e ritorno lungo il ventre); rileggere il file riga per riga saltando commenti e righe vuote, e confrontare con `np.loadtxt`.

**Impostazione.** Scrittura con `with open(..., "w")` e `fp.write(...)` di una f-string per riga; il dorso va rovesciato (`reversed(dorso)`) e concatenato con il ventre senza ripetere il bordo d'attacco (`ventre[1:]`). Lettura: `for riga in fp:` dà una riga alla volta; `strip`, controllo di `startswith("#")`, `split`, `float`.

**Svolgimento.** Vedi `es_07`. Il file è scritto in `figure/es_07_naca2412.dat`.

**Risultato**

41 coppie scritte e rilette (20 punti per lato più il bordo d'attacco), prima $(1, 0)$ e ultima $(1, 0)$; `np.loadtxt` dà forma $(41, 2)$; ordinata massima $0{,}0792\,c$.

**Commento.** Il ciclo esplicito mostra che cosa `np.loadtxt` fa dietro le quinte, e serve quando il formato è irregolare (intestazioni di più righe, colonne separate da virgole, decimali con la virgola: `riga.replace(",", ".")`). Per i file regolari `np.loadtxt` con `comments="#"`, `skiprows` e `delimiter` basta sempre.

## B.8 Esercizio 8 – Vettorializzare

**Testo**

Calcolare la media mobile a tre punti di un segnale di 200 000 valori con un ciclo `for` e con lo slicing; misurare i tempi, verificare l'uguaglianza dei risultati, e confrontare con `np.convolve` e `np.gradient`.

**Impostazione.** La media $m_i = (f_{i-1} + f_i + f_{i+1})/3$ nei nodi interni è `(f[:-2] + f[1:-1] + f[2:])/3`: tre viste sfalsate. `np.convolve(f, np.ones(3)/3, mode="valid")` è la stessa operazione come convoluzione; `np.gradient(f, x)` calcola la derivata con differenze centrate all'interno e unilatere agli estremi.

**Svolgimento.** Due cronometri con `perf_counter`, `np.allclose` per il confronto.

**Risultato**

Ciclo $\sim$ 80 ms, slicing $\sim$ 1,5 ms (rapporto 50–60), risultati uguali; `np.convolve` coincide; `np.gradient` 6,6 ms.

**Commento.** Il rapporto cresce con $n$ e con la complessità dell'espressione; per il laplaciano bidimensionale del Programma 0.4 era 40–50. La regola di traduzione: ogni `f[i+k]` dentro un ciclo su `i` diventa una vista `f[a+k:b+k]` dove `a:b` è l'intervallo del ciclo, e il ciclo sparisce. Non si vettorializza quando l'elemento $i$ dipende dall'elemento $i-1$ *appena calcolato* (Esercizio 20, Programma 0.9 g).

## B.9 Esercizio 9 – Broadcasting

**Testo**

Costruire con il broadcasting, senza cicli, la tabella della velocità limite $v_t = \sqrt{2mg/(\rho_a A c_d)}$ (con $c_d = 0{,}5$) per sfere di diametro 1, 5, 10, 50 mm e densità 1000, 2700, 7800 kg/m$^3$; verificare che $v_t \propto \sqrt{\rho d}$.

**Impostazione.** I diametri come colonna, forma `(4, 1)`, le densità come riga, forma `(1, 3)`: ogni espressione che li combina ha forma `(4, 3)`. `d[:, None]` aggiunge l'asse; equivalentemente `d.reshape(4, 1)`.

```
d = np.array([0.001, 0.005, 0.01, 0.05])[:, None]        # (4, 1)
rho = np.array([1000.0, 2700.0, 7800.0])[None, :]         # (1, 3)
m = rho * np.pi * d**3 / 6                                # (4, 3)
vt = np.sqrt(2 * m * G / (1.225 * np.pi * d**2 / 4 * 0.5))
```

La tabella si stampa riga per riga con `" ".join(f"{v:8.2f}" for v in riga)`.

**Risultato**

Da 4,62 m/s (1 mm, legno) a 91,2 m/s (50 mm, acciaio); forma $(4, 3)$; rapporto fra $d = 50$ e 5 mm pari a 3,162 $= \sqrt{10}$.

**Commento.** $m \propto d^3$ e $A \propto d^2$ danno $v_t \propto \sqrt{\rho d}$: un chicco di grandine da 5 mm cade a $\sim 10$ m/s, uno da 5 cm a $\sim 33$ m/s. Il broadcasting è il modo naturale di fare *studi parametrici* su due parametri: la stessa idea produce la mappa $(Re, \text{griglia}) \to$ errore o il campo $f(x_i, y_j)$ senza `meshgrid`.

## B.10 Esercizio 10 – Nodi fittizi e condizioni di Neumann

**Testo**

Calcolare il laplaciano discreto di $u = \cos \pi x \cos \pi y$ su tutta la griglia $[0, 1]^2$, *bordi compresi*, usando un array esteso di una riga di nodi fittizi per lato riempiti per riflessione ($\partial u/\partial n = 0$ sul contorno); confrontare con $-2\pi^2 u$, stimare l'errore e verificare l'ordine raddoppiando $N$.

**Impostazione.** `ue = np.zeros((N+3, N+3))` e `ue[1:-1, 1:-1] = u`; la riflessione `ue[0, :] = ue[2, :]` realizza $u_{-1} = u_1$, cioè la derivata centrata nulla sul bordo. Il laplaciano a cinque punti applicato a `ue[1:-1, 1:-1]` ha la forma di `u`. L'errore atteso sul bordo: per $u = \cos \pi x$ la riflessione è *esatta* (la funzione è pari intorno a $x = 0$), quindi l'errore è solo quello di troncamento $h^2(u_{xxxx} + u_{yyyy})/12 = h^2\pi^4/6$ nel massimo.

**Svolgimento.** Vedi `es_10`; il calcolo è ripetuto con $N = 80$.

**Risultato**

$N = 40$: errore massimo $1{,}01 \times 10^{-2}$, uguale sul bordo e all'interno, uguale alla stima $h^2\pi^4/6 = 1{,}01 \times 10^{-2}$; $N = 80$: $2{,}54 \times 10^{-3}$, rapporto 4,00.

**Commento.** È la tecnica che il Programma 4.5 usa per l'aderenza sulla griglia MAC (`u[:, 0] = -u[:, 1]`: riflessione *antisimmetrica* per $u = 0$ sulla parete) e che `matrice_pressione` incorpora nella matrice per $\partial p/\partial n = 0$. Con i nodi fittizi lo *stesso* slicing vale all'interno e sul bordo, senza casi speciali; il prezzo è un array più grande di due righe e due colonne e l'attenzione agli indici (il nodo fisico $(i, j)$ è `ue[i+1, j+1]`).

## B.11 Esercizio 11 – Una figura a due pannelli

**Testo**

Disegnare, nello stile della casa, la curva $C_L(\alpha)$ e la polare $C_L(C_D)$ della tabella dell'Esercizio 6 in due pannelli affiancati, con gli assi di riferimento, un'annotazione dello stallo e la retta dall'origine tangente alla polare nel punto di efficienza massima con la sua etichetta.

**Impostazione.** `plt.subplots(1, 2)`; `axhline/axvline` per gli assi; `np.argmax(cl)` per lo stallo e `annotate` con freccia; l'efficienza massima è dove la retta dall'origine è tangente alla polare: `j = np.argmax(cl/cd)` e un segmento da $(0,0)$ a $(C_{D,j}, C_{L,j})$.

**Svolgimento.** Vedi `es_11`; la figura è salvata con `salva(fig, "es_11")`.

**Risultato**

Fig. B.1: stallo a 12°, $E_{\max} = 68$ con la retta tangente.

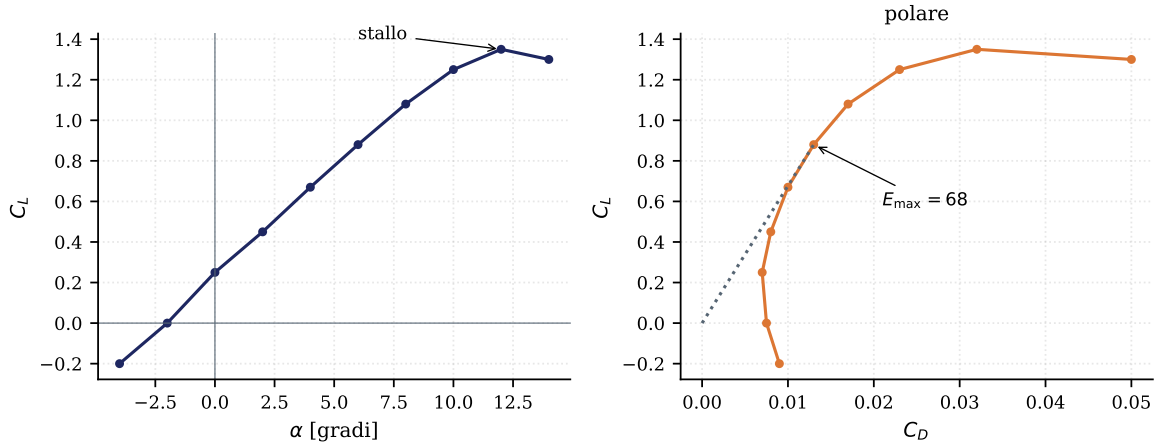



Figura B.1: Esercizio 11: curva di portanza e polare con le annotazioni.

**Commento.** Le annotazioni calcolate dai dati (`xy=(alfa[k], cl[k])`) seguono i dati se cambiano; le etichette con una f-string dentro la formula (`f"$E_{{\\max}}={e[j]:.0f}$"`: le graffe di LaTeX vanno raddoppiate dentro una f-string) evitano di scrivere a mano un numero che potrebbe cambiare.

## B.12 Esercizio 12 – `fsolve` sul sistema di Galerkin della sfera

**Testo**

Il Modulo 4 risolve con il metodo di Newton a due variabili il sistema ridotto di Galerkin per la sfera, $g(A_1, B_1) = 0$, $h(A_1, B_1) = 0$, con

$$g = \frac{9{,}931035}{Re} A_1 + 0{,}011713\, B_1 - 0{,}002546\, A_1 B_1 + \frac{44{,}689656}{Re},$$
$$h = 0{,}050148\, A_1 + \frac{2{,}201058}{Re} B_1 + 0{,}002421\, A_1^2 - 0{,}000376\, B_1^2 + 0{,}379018.$$

Risolverlo con `scipy.optimize.fsolve` per $Re = 100$ a partire da $(-4, -1)$ e confrontare con i valori tabulati $A_1 = -2{,}6083$, $B_1 = -10{,}236$; calcolare $C_D$ dalle formule del modulo ($A_2 = -\frac{15}{29}(8 + 5A_1)$, $A_3 = \frac{9}{29}(17 + 7A_1)$, $A_4 = -(95 + 34A_1)/58$, $C_D = \frac{32}{3Re}(8A_2 + 25A_3 + 54A_4) + \frac{64}{3Re}(2A_2 + 5A_3 + 9A_4)$) e confrontare con 1,0436.

**Impostazione.** `fsolve` vuole una funzione che riceve un vettore e restituisce una lista di residui: `lambda p: [g(*p), h(*p)]`, dove `*p` spacchetta le due componenti negli argomenti di `g`. `full_output=True` dà anche il numero di valutazioni e lo stato.

**Svolgimento.** Vedi `es_12`.

**Risultato**

$A_1 = -2{,}6083$, $B_1 = -10{,}236$, residui $\sim 10^{-13}$ in 13 valutazioni; $C_D = 1{,}0436$: tutto coincide con la tabella del Modulo 4.

**Commento.** `fsolve` fa ciò che `newtn2` del Modulo 4 fa a mano, con lo jacobiano stimato per differenze finite (per questo 13 valutazioni invece di $\sim 5$ iterazioni). Il punto di partenza $(-4, -1)$ è nel bacino giusto; da $(0, 0)$ si trova la stessa soluzione, ma per $Re$ grandi il sistema ha più soluzioni e la continuazione dal $Re$ precedente, come nel modulo, è più sicura.

## B.13 Esercizio 13 – Il pendolo con `solve_ivp` ed eventi

**Testo**

Integrare $\ddot{\theta} = -\sin\theta$ (tempo in unità di $\sqrt{l/g}$) con `solve_ivp` da $\theta_0 = 10°$, $45°$, $90°$, $135°$, $170°$ con velocità iniziale nulla; usare un evento sul passaggio per $\theta = 0$ per misurare un quarto di periodo e confrontare $T/T_0$ ($T_0 = 2\pi$) con il valore esatto $T = 4K(k)$, $k = \sin(\theta_0/2)$, con $K$ da `scipy.special.ellipk`.

**Impostazione.** Lo stato è $[\theta, \dot{\theta}]$. L'evento è la funzione `passaggio = lambda t, y: y[0]` con `direction = -1` (solo attraversamenti decrescenti) e `terminal = True`; il primo tempo di evento è $T/4$. `ellipk` di SciPy prende come argomento il *parametro* $m = k^2$, non il modulo $k$: una fonte classica di errori.

**Svolgimento.** Vedi `es_13`, con `rtol=1e-11`.

**Risultato**

$T/T_0 = 1{,}001907$, $1{,}039973$, $1{,}180341$, $1{,}527948$, $2{,}439363$ per i cinque angoli, identici a sei cifre ai valori di $4K(k)/2\pi$.

**Commento.** Gli eventi di `solve_ivp` localizzano l'istante con la precisione della soluzione densa, non del passo: il metodo è adattivo e il passaggio per zero è trovato per interpolazione e raffinamento. È lo strumento per "la sfera tocca il suolo", "il proiettile raggiunge l'apice", "la

traiettoria attraversa il piano", che il Modulo 1 gestisce con una funzione di arresto a passo fisso.

## B.14 Esercizio 14 – Lo schema implicito con l'algoritmo di Thomas

**Testo**

Risolvere il problema del Programma 0.8 con lo schema implicito BTCS, $-Ru_{i-1}^{n+1} + (1 + 2R)u_i^{n+1} - Ru_{i+1}^{n+1} = u_i^n$, usando `thomas` di `metodi_numerici.py`, per $R = 0{,}4,\ 2,\ 10$; confrontare l'errore a $t = 0{,}1$ con quello dello schema esplicito.

**Impostazione.** A ogni passo si risolve un sistema tridiagonale con diagonale $1 + 2R$ e codiagonali $-R$, costanti nel tempo: i tre vettori si costruiscono una volta con `np.full`. Con $u = 0$ agli estremi il termine noto è semplicemente `u[1:-1]`. Lo schema è incondizionatamente stabile, ma del primo ordine nel tempo: l'errore crescerà con $R$.

**Svolgimento.** Vedi `es_14`: `integra` ed `esatta` sono importate dal Programma 0.8.

**Risultato**

$R = 0{,}4$: 400 passi, errore $6{,}4 \times 10^{-4}$ (esplicito: $2{,}6 \times 10^{-4}$); $R = 2$: 80 passi, $2{,}5 \times 10^{-3}$; $R = 10$: 16 passi, $1{,}1 \times 10^{-2}$. Stabile per ogni $R$, con errore $\propto \Delta t$.

**Commento.** A parità di $R$ l'implicito è meno accurato dell'esplicito (il cui errore di troncamento $\Delta t$ e $h^2$ hanno segni opposti e si compensano in parte), ma può fare passi 25 volte più lunghi: il rapporto costo/accuratezza va valutato caso per caso, e Crank–Nicolson (Modulo 3) unisce stabilità incondizionata e secondo ordine. L'Esercizio 17 applica lo stesso schema al problema di Rayleigh.

## B.15 Esercizio 15 – La formula a cinque punti con SymPy

**Testo**

Con il metodo dei coefficienti indeterminati del Programma 0.7, ricavare la formula centrata a cinque punti per $f'$, $f' \approx \sum_{k=-2}^{2} c_k f(x + kh)/h$, e il suo errore di troncamento.

**Impostazione.** Cinque incognite, cinque condizioni: coefficiente di $f_0$ nullo, di $f_1$ unitario, di $f_2, f_3, f_4$ nulli. Lo sviluppo di Taylor va spinto fino a $f_6$ per leggere l'errore. Per simmetria ci si aspetta $c_{-k} = -c_k$ e $c_0 = 0$, e un errore in $h^4 f^{(5)}$.

**Svolgimento.** `f = sp.symbols("f0:7")` crea sette simboli; lo sviluppo e la combinazione sono

```
taylor = lambda k: sum((k * h)**j / sp.factorial(j) * f[j] for j in range(7))
approx = sp.expand(sum(ck * taylor(k) for ck, k in zip(c, (-2, -1, 0, 1, 2))) / h)
eqs = [approx.coeff(f[0]), approx.coeff(f[1]) - 1] + [approx.coeff(f[j]) for j in
    (2, 3, 4)]
```

```
sol = sp.solve(eqs, c);  errore = sp.expand(approx.subs(sol) - f[1])
```

**Risultato**

$c = \left(\frac{1}{12}, -\frac{2}{3}, 0, \frac{2}{3}, -\frac{1}{12}\right)$, cioè $f' = (f_{-2} - 8f_{-1} + 8f_1 - f_2)/(12h)$, con errore $-h^4 f^{(5)}/30$: quarto ordine.

**Commento.** Il codice è generale: cambiando la tupla dei nodi `(-2, -1, 0, 1, 2)` in `(0, 1, 2, 3)` si ottiene la formula unilatera a quattro punti del terzo ordine; chiedendo il coefficiente di $f_2$ unitario invece di $f_1$ si ottengono le formule per la derivata seconda. Tutte le formule del Modulo 2 si generano così in un ciclo.

## B.16 Esercizio 16 – Il Programma 1.1 con l'integratore generico

**Testo**

Riscrivere la caduta della sfera del Programma 1.1 (lettura guidata 1) usando `ode_solvers.integra` con lo stato $\mathbf{y} = [z, v]$ invece del Runge–Kutta scritto a mano, e verificare che a $t = 10$ s i risultati coincidano.

**Impostazione.** `integra(f, t0, y0, h, n_passi, metodo="rk4")` vuole `f(t, y)` che restituisce un array della forma di `y`: `F = lambda t, y: np.array([y[1], (B - C*y[1]*abs(y[1])*cd_sfera(y[1]*d/NU))/A])`, con `A, B, C` da `costanti(d)` del programma originale (importato). Con $h = 0{,}1$ e 100 passi si arriva a $t = 10$.

**Svolgimento.** Vedi `es_16`: le due versioni sono eseguite e confrontate.

**Risultato**

$z(10) = 294{,}2477$ m, $v(10) = 40{,}6718$ m/s con entrambe, differenza nulla: le due implementazioni di RK4 eseguono esattamente le stesse operazioni in virgola mobile.

**Commento.** La versione con `integra` è più corta e vale per qualunque sistema (il pendolo, l'oscillatore, le equazioni di un proiettile con tre componenti) senza riscrivere i quattro stadi; quella originale mostra i quattro stadi, che è lo scopo didattico del Modulo 1. Scrivere entrambe e confrontarle è una verifica reciproca.

## B.17 Esercizio 17 – Il problema di Rayleigh in forma implicita

**Testo**

Modificare il Programma 3.4 (lettura guidata 2) sostituendo lo schema esplicito con BTCS risolto con `thomas`; integrare fino a $2{,}5 \times 10^5$ s con $R = 0{,}25$, 2, 8 e confrontare errore e numero di passi con l'originale (400 passi, $R = 0{,}25$).

**Impostazione.** A differenza dell'Esercizio 14, qui il nodo superiore ha $u = U_0 \neq 0$: nell'equazione dell'ultimo nodo interno il termine $-Ru_{M-1}^{n+1}$ è noto e passa al secondo membro, `rhs[-1] += R*U0`. È l'unico cambiamento rispetto al caso omogeneo, ed è il punto in cui si sbaglia più spesso.

**Svolgimento.** Vedi `es_17`; il numero di passi è `while n*tau < TMAX`, con $\tau = RH^2/\nu$.

**Risultato**

$R = 0{,}25$: $\tau = 625$ s, 400 passi, errore $5{,}7 \times 10^{-4}$ m/s; $R = 2$: 50 passi, $3{,}4 \times 10^{-3}$; $R = 8$: $\tau = 20\,000$ s, 13 passi (fino a $2{,}6 \times 10^{5}$ s), $1{,}2 \times 10^{-2}$ m/s, venti volte l'errore dell'esplicito con un trentesimo dei passi.

**Commento.** Il problema di Rayleigh ha una scala di tempo lunga ($d^2/\pi^2\nu \approx 10^5$ s) e il limite $R \leq 1/2$ dell'esplicito impone passi di pochi minuti: è il caso tipico in cui l'implicito conviene, accettando un errore maggiore o passando a Crank–Nicolson. La modifica ha toccato tre righe del programma originale: la costruzione dei tre vettori, la riga dello schema e il termine noto.

## B.18 Esercizio 18 – Il Programma 4.5 come libreria

**Testo**

Importare `cavita` dal Programma 4.5 (lettura guidata 3) e, senza modificarlo, calcolare per $Re = 100$ su griglia $33 \times 33$: $\psi_{\min}$, l'energia cinetica $E_k = \frac{1}{2}\sum(u^2 + v^2)h^2$ sui centri delle celle, la vorticità media sul coperchio, e la divergenza massima del campo finale.

**Impostazione.** `r = cavita(100, 32, 20.0, stampa=False)` restituisce il dizionario; `u` e `v` sono sulle facce della griglia MAC, e per l'energia vanno mediati ai centri con le stesse due righe del programma (`uc = 0.5*(u[1:, 1:-1] + u[:-1, 1:-1])`). La vorticità sul coperchio è $-\partial u/\partial y$, con il nodo fittizio: $(u_{N+1} - u_N)/h$ dove $u_{N+1} = 2u_{\text{cop}} - u_N$.

**Svolgimento.** Vedi `es_18`.

**Risultato**

$\psi_{\min} = -0{,}1021$ (Ghia et al.: $-0{,}1034$ su griglia $129 \times 129$), $E_k = 0{,}03274$, vorticità media sul coperchio $-14{,}9$, divergenza massima $1{,}4 \times 10^{-14}$: il campo è solenoidale a meno dell'arrotondamento.

**Commento.** Un programma con funzioni e blocco `__main__` è anche una libreria: l'esercizio ha calcolato tre grandezze nuove senza toccare le 174 righe del Programma 4.5, e il dizionario restituito ha reso disponibili tutti i campi con il loro nome. È il motivo della struttura descritta nel Cap. 8.

## B.19 Esercizio 19 – Un programma completo: convezione lineare

**Testo**

Scrivere da zero, con la struttura del programma modello, un programma che risolva $u_t + u_x = 0$ su $[0,1]$ periodico con condizione iniziale gaussiana $e^{-200(x-0{,}3)^2}$ fino a $t = 0{,}5$ con $C = 0{,}5$, con gli schemi upwind e Lax–Wendroff; misurare l'ordine di convergenza con $n_x = 100, 200, 400, 800$ e disegnare le soluzioni.

**Impostazione.** Griglia periodica con `endpoint=False` ($n_x$ nodi, il nodo $n_x$ coincide con lo 0); i vicini con `np.roll(u, 1)` ($u_{i-1}$) e `np.roll(u, -1)` ($u_{i+1}$). Upwind: $u_i - C(u_i - u_{i-1})$; Lax–Wendroff: $u_i - \frac{C}{2}(u_{i+1} - u_{i-1}) + \frac{C^2}{2}(u_{i+1} - 2u_i + u_{i-1})$. La soluzione esatta è la gaussiana traslata di $t$ (modulo la periodicità). Una funzione `integra(schema, nx, C, t_fin)` restituisce griglia, soluzione numerica ed esatta; il blocco principale fa la tabella degli errori, l'ordine con `np.polyfit` e la figura.

**Svolgimento.** Vedi `es_19`.

**Risultato**

Errori upwind 0,29, 0,18, 0,11, 0,057 (ordine misurato 0,79, che tende a 1 raffinando); Lax–Wendroff $6{,}8 \times 10^{-2}$, $1{,}7 \times 10^{-2}$, $4{,}3 \times 10^{-3}$, $1{,}1 \times 10^{-3}$ (ordine 2,00). Fig. B.2: l'upwind attenua e allarga la gaussiana, Lax–Wendroff la conserva con una piccola oscillazione in coda.

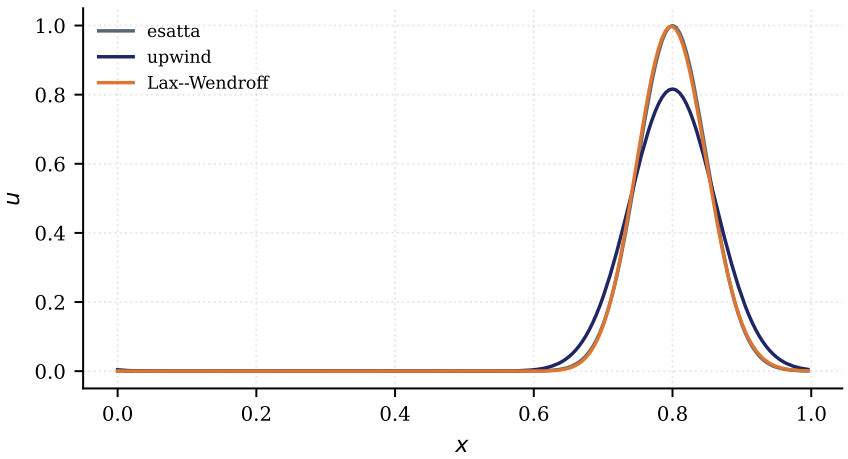



Figura B.2: Esercizio 19: gaussiana trasportata con upwind e Lax–Wendroff ($n_x = 200$, $C = 0{,}5$, $t = 0{,}5$) e soluzione esatta.

**Commento.** L'ordine misurato dell'upwind è 0,79 invece di 1 perché la gaussiana con $\sigma = 0{,}05$ è coperta da pochi nodi a $n_x = 100$ (regime pre-asintotico): l'ordine asintotico si legge solo quando la griglia risolve la soluzione, e misurarlo con più griglie, non con due, mostra la tendenza. Il programma è lungo trenta righe e ha tutte le parti del modello: è la forma che ogni esercizio di programmazione del corso dovrebbe avere.

## B.20 Esercizio 20 – Caccia agli errori

**Testo**

La funzione seguente dovrebbe calcolare $\int_0^1 u\,\mathrm{d}x$ con la regola dei trapezi da un vettore `u` su griglia uniforme di passo `h`, ma contiene tre errori silenziosi. Trovarli con il confronto con un caso noto ($u = x^2$, integrale $1/3$), correggerla, e provocare poi due errori segnalati (indice fuori intervallo, forme incompatibili) leggendone il messaggio.

```
def media_profilo_sbagliata(u, h):
    n = len(u)
    somma = 0
    for i in range(1, n):
        somma += u[i]
    h = 1 / n
    return somma * h / 2
```

**Impostazione.** La regola dei trapezi è $h\left(\frac{1}{2}u_0 + u_1 + \cdots + u_{n-2} + \frac{1}{2}u_{n-1}\right)$. Si confronti termine a termine con il codice: il ciclo salta $u_0$ e include $u_{n-1}$ con peso pieno; `h` ricevuto come argomento viene sovrascritto con $1/n$ invece di $1/(n-1)$; il fattore $1/2$ è applicato a tutta la somma.

**Svolgimento.** Con $u = x^2$ su 11 nodi la funzione dà 0,1750 contro 0,3333: l'errore è grossolano e immediatamente visibile, ma *solo* perché si conosce la risposta. La versione corretta, vettorializzata: `(0.5*u[0] + np.sum(u[1:-1]) + 0.5*u[-1])*h`. Poi `u[11]` su un array di 11 elementi e `u + np.zeros(10)`.

**Risultato**

Sbagliato 0,1750; corretto 0,3350 (uguale a `np.trapezoid`; la differenza da $1/3$ è l'errore $h^2/12 \cdot \int u'' = 1/600$ della regola dei trapezi). `IndexError: index 11 is out of bounds for axis 0 with size 11`; `ValueError: operands could not be broadcast together with shapes (11,) (10,)`.

**Commento.** I tre errori (scostamento di uno, $h = 1/n$, fattore applicato male) sono fra gli otto del Programma 0.9, e nessuno produce un messaggio: il programma restituisce un numero con la forma giusta. L'unica difesa è la verifica contro un caso noto, codificata in un `assert` che resta nel programma: `assert abs(media_profilo(x**2, h) - 1/3) < h**2`. I due errori segnalati, al contrario, si diagnosticano leggendo il messaggio: "index 11 … size 11" dice che l'ultimo indice valido è 10, "shapes (11,) (10,)" che due array di lunghezze diverse sono stati combinati.

# Bibliografia


### Testi di riferimento del corso

[1] S. Biringen, C.-Y. Chow, *An Introduction to Computational Fluid Mechanics by Example*, Wiley, Hoboken, 2011. Il testo di riferimento dei Moduli 1–4; i suoi programmi MATLAB sono stati tradotti in Python nelle monografie.

[2] C.-Y. Chow, *An Introduction to Computational Fluid Mechanics*, Wiley, New York, 1979 (edizione originale, con programmi FORTRAN).

[3] L. A. Barba, G. F. Forsyth, *CFD Python: the 12 steps to Navier–Stokes equations*, Journal of Open Source Education **1**(9) (2018) 21; `https://github.com/barbagroup/CFDPython`. Il corso in notebook che ha ispirato l'impostazione didattica (Python, NumPy, slicing per le differenze finite) dell'intero corso.

### Python

[4] *Introduction to Python Programming*, OpenStax, Rice University, Houston, 2024 (manuale libero; `https://openstax.org/details/books/introduction-python-programming`). Capp. 1–9: tipi, controllo del flusso, funzioni, liste, dizionari, stringhe, file, eccezioni.

[5] *The Python Tutorial* e *The Python Language Reference*, Python Software Foundation, `https://docs.python.org/3/`. In particolare il tutorial (Sez. 3–9) e la sezione *Floating Point Arithmetic: Issues and Limitations*.

[6] G. van Rossum, B. Warsaw, A. Coghlan, *PEP 8 – Style Guide for Python Code*, 2001–, `https://peps.python.org/pep-0008/`.

[7] M. Lutz, *Learning Python*, 5ª ed., O'Reilly, Sebastopol, 2013 (riferimento esteso sul linguaggio: oggetti mutabili e immutabili, ambito delle variabili, moduli).

### Virgola mobile

[8] D. Goldberg, *What every computer scientist should know about floating-point arithmetic*, ACM Computing Surveys **23**(1) (1991) 5–48. Rappresentazione IEEE 754, arrotondamento, cancellazione, somma compensata.

[9] IEEE Computer Society, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019.

[10] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2ª ed., SIAM, Philadelphia, 2002. Cap. 1 (principi), Cap. 4 (sommazione: Kahan, a coppie), Sez. 1.8 (radici dell'equazione di secondo grado).

### NumPy, SciPy, Matplotlib, SymPy

[11] C. R. Harris et al., *Array programming with NumPy*, Nature **585** (2020) 357–362. Il modello degli array: forma, viste, broadcasting, vettorializzazione.

[12] *NumPy User Guide*, `https://numpy.org/doc/stable/user/`; in particolare *Indexing on ndarrays*, *Broadcasting*, *Copies and views*.

[13] P. Virtanen et al., *SciPy 1.0: fundamental algorithms for scientific computing in Python*, Nature Methods **17** (2020) 261–272.

[14] *SciPy User Guide*, `https://docs.scipy.org/doc/scipy/tutorial/`; sezioni *Optimization*, *Integration*, *Linear algebra*, *Sparse arrays*, *Interpolation*.

[15] J. D. Hunter, *Matplotlib: a 2D graphics environment*, Computing in Science & Engineering **9**(3) (2007) 90–95; *Matplotlib User Guide*, `https://matplotlib.org/stable/users/`.

[16] A. Meurer et al., *SymPy: symbolic computing in Python*, PeerJ Computer Science **3** (2017) e103; *SymPy Documentation*, `https://docs.sympy.org/`.

[17] J. VanderPlas, *Python Data Science Handbook*, O'Reilly, Sebastopol, 2016 (Cap. 2: NumPy; Cap. 4: Matplotlib); disponibile in rete.

### Metodi numerici e verifica

[18] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, Philadelphia, 2007. Cap. 1 (formule alle differenze con i coefficienti indeterminati), App. A (misura dell'ordine di convergenza, estrapolazione di Richardson).

[19] J. H. Ferziger, M. Perić, *Computational Methods for Fluid Dynamics*, 3ª ed., Springer, Berlin, 2002. Sez. 3.9 (stima dell'errore di discretizzazione), Cap. 7 (soluzione delle equazioni di Navier–Stokes, griglia sfalsata).

[20] P. J. Roache, *Verification and Validation in Computational Science and Engineering*, Hermosa, Albuquerque, 1998. Soluzioni manifatturate, ordine di convergenza osservato, indice di convergenza di griglia.

[21] W. H. Press, S. A. Teukolsky, W. T. Vetterling, B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3ª ed., Cambridge University Press, 2007. Cap. 2 (sistemi tridiagonali e a banda), Cap. 5.3 (Horner), Cap. 9 (bisezione, Brent, Newton), Cap. 17 (Runge–Kutta adattivo, problemi stiff).

[22] U. Ghia, K. N. Ghia, C. T. Shin, *High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method*, Journal of Computational Physics **48** (1982) 387–411 (dati di riferimento per la cavità, Esercizio 18).

[23] I. H. Abbott, A. E. von Doenhoff, *Theory of Wing Sections*, Dover, New York, 1959 (profili NACA a quattro cifre: distribuzione di spessore e linea media, Programma 0.3).

[24] ISO 2533:1975, *Standard Atmosphere*, International Organization for Standardization, Ginevra (atmosfera standard, Programma 0.1).